ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ-VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: KỸ THUẬT ĐIỆN TỬ-VIỄN THÔNG
CHUYÊN NGÀNH: ĐIỆN TỬ VIỄN THÔNG

ĐỀ TÀI:

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG CẢNH BÁO CHÁY SỬ DỤNG CÔNG NGHỆ LORAWAN

Người hướng dẫn: **TS. TRẦN VĂN LÍC**
TS. NGUYỄN THỊ BÍCH NGỌC
Sinh viên thực hiện: **DƯƠNG TẤN THỊNH**
Số thẻ sinh viên: **106210100**
Lớp**: 21DT2**

Đà Nẵng, 1/2026

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ-VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

NGÀNH: KỸ THUẬT ĐIỆN TỬ-VIỄN THÔNG
CHUYÊN NGÀNH: ĐIỆN TỬ-VIỄN THÔNG

ĐỀ TÀI:

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG CẢNH BÁO CHÁY SỬ DỤNG CÔNG NGHỆ LORAWAN

Người hướng dẫn: **TS. TRẦN VĂN LÍC**
TS. NGUYỄN THỊ BÍCH NGỌC
Sinh viên thực hiện: **DƯƠNG TẤN THỊNH**
Số thẻ sinh viên: **106210100**
Lớp**: 21DT2**

Đà Nẵng, 1/2026

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA……

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP

1. Thông tin chung:

1. Họ và tên sinh viên: …….
2. Lớp: …… Số thẻ SV: ..……
3. Tên đề tài: ……....
4. Người hướng dẫn: ……..…… Học hàm/ học vị: ……

II. Nhận xét, đánh giá đồ án tốt nghiệp:

1. Về tính cấp thiết, tính mới, khả năng ứng dụng của đề tài: (điểm tối đa là 2đ)

   ……..

   ……..

2. Về kết quả giải quyết các nội dung nhiệm vụ yêu cầu của đồ án: (điểm tối đa là 4đ)

   ……..

   ……..

3. Về hình thức, cấu trúc, bố cục của đồ án tốt nghiệp: (điểm tối đa là 2đ)

   ……..

   ……..

4. Đề tài có giá trị khoa học/ có bài báo/ giải quyết vấn đề đặt ra của doanh nghiệp hoặc nhà trường: (điểm tối đa là 1đ)

   ……..

   ……..

5. Các tồn tại, thiếu sót cần bổ sung, chỉnh sửa:

   ……..

   ……..

III. Tinh thần, thái độ làm việc của sinh viên: (điểm tối đa 1đ)

……..

IV. Đánh giá:

1. Điểm đánh giá: ……../10 (lấy đến 1 số lẻ thập phân)
2. Đề nghị: ☐ Được bảo vệ đồ án ☐ Bổ sung để bảo vệ ☐ Không được bảo vệ

Đà Nẵng, ngày tháng năm 201

Người hướng dẫn

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ……

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

NHẬN XÉT PHẢN BIỆN ĐỒ ÁN TỐT NGHIỆP

I. Thông tin chung:

1. Họ và tên sinh viên: ……

2. Lớp: …… Số thẻ SV: ..……

3. Tên đề tài: ……....

4. Người phản biện: ..…… Học hàm/ học vị: ……

II. Nhận xét, đánh giá đồ án tốt nghiệp:

TT	Các tiêu chí đánh giá	Điểm tối đa	Điểm đánh giá
1	**Sinh viên có phương pháp nghiên cứu phù hợp, giải quyết đủ nhiệm vụ đồ án được giao**	**80**	
1a	- Tính mới (nội dung chính của ĐATN có những phần mới so với các ĐATN trước đây). - Đề tài có giá trị khoa học, công nghệ; có thể ứng dụng thực tiễn.	15	
1b	- Kỹ năng giải quyết vấn đề; hiểu, vận dụng được kiến thức cơ bản, cơ sở, chuyên ngành trong vấn đề nghiên cứu. - Chất lượng nội dung ĐATN (thuyết minh, bản vẽ, chương trình, mô hình,…).	50	
1c	- Có kỹ năng vận dụng thành thạo các phần mềm ứng dụng trong vấn đề nghiên cứu; - Có kỹ năng đọc, hiểu tài liệu bằng tiếng nước ngoài ứng dụng trong vấn đề nghiên cứu; - Có kỹ năng làm việc nhóm;	15	
2	**Kỹ năng viết:**	**20**	
2a	- Bố cục hợp lý, lập luận rõ ràng, chặt chẽ, lời văn súc tích	15	
2b	- Thuyết minh đồ án không có lỗi chính tả, in ấn, định dạng	5	
3	**Tổng điểm đánh giá theo thang 100:**		
	Quy về thang 10 (lấy đến 1 số lẻ)		

- Các tồn tại, thiếu sót cần bổ sung, chỉnh sửa: ……..
  ……...
- Câu hỏi đề nghị sinh viên trả lời trong buổi bảo vệ: ……
  ……...
  ……...
- Đề nghị: ☐ Được bảo vệ đồ án ☐ Bổ sung để bảo vệ ☐ Không được bảo vệ

Đà Nẵng, ngày tháng năm 201…

Người phản biện

TÓM TẮT

Tên đề tài: Thiết kế và triển khai hệ thống cảnh báo cháy sử dụng công nghệ LoraWAN

Sinh viên thực hiện: Dương Tấn Thịnh

Số thẻ SV: 106210100 Lớp: 21DT2

Trong bối cảnh các sự cố hỏa hoạn tại khu dân cư và chung cư cao tầng diễn biến phức tạp, nhu cầu về một hệ thống cảnh báo cháy tin cậy, phạm vi phủ sóng rộng và tiêu thụ năng lượng thấp là vô cùng cấp thiết. Đồ án này tập trung nghiên cứu và phát triển một giải pháp IoT toàn diện để giám sát nguy cơ cháy, khắc phục các hạn chế về khoảng cách truyền tin của các công nghệ cũ (như Wi-Fi, Zigbee).

Cấu trúc đồ án bao gồm 4 phần:

Chương 1: Trình bày tổng quan về kiến trúc IOT của hệ thống; những yêu cầu chức năng, phi chức năng của toàn hệ thống cũng như riêng từng phân hệ trong hệ thống.

Chương 2: Trình bày về cách thức triển khai phân hệ Device từ việc thiết kế kiến trúc, xây dựng mạch PCB, xây dựng chương trình nhúng, hoàn thiện Device trong thực tế v.v.
Chương 3: Trình bày cách thức xây dựng MobiApp của hệ thống; một số bước chính để cấu hình, giao tiếp giữa MobiApp với Server thông qua MQTT.

Chương 4: Trình bày nội dung của một số trường hợp kiểm thử, từ đó giúp đánh giá khả năng hoạt động của hệ thống trong một số điều kiện khác nhau.

Chương 5: Trình bày về tổng quan những gì mà hệ thống đã đạt được, nêu bật những ưu điểm lẫn khuyết điểm, từ đó đưa ra kết luận lẫn hướng phát triển cho đề tài.

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ-VIỄN THÔNG

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: Dương Tấn Thịnh Số thẻ sinh viên: 106210100

Lớp: 21DT2 Khoa: Kỹ thuật Điện tử-Viễn thông Ngành: Điện tử-Viễn thông

1. *Tên đề tài đồ án:*

Thiết kế và triển khai hệ thống báo cháy sử dụng công nghệ LoRaWAN

2. *Đề tài thuộc diện:* ☐ *Có ký kết thỏa thuận sở hữu trí tuệ đối với kết quả thực hiện*
3. *Các số liệu và dữ liệu ban đầu:*
   - Giao thức truyền thông: Chuẩn LoRaWAN v1.0.3, băng tần AS923-2 (923.2 MHz - 923.4 MHz).
   - Vi điều khiển & Module RF: RAK3172 (dựa trên SoC STM32WLE5) hỗ trợ truyền tin tầm xa công suất thấp.
   - Cảm biến:
     +Cảm biến khói: Buồng quang điện (Photoelectric Smoke Chamber) hoạt động theo nguyên lý tán xạ ánh sáng.
     +Cảm biến nhiệt độ/độ ẩm: SHTC3 (giao tiếp I2C, độ chính xác ±0.2°C)
   - Nguồn cấp: Pin dung lượng 1000mAh, IC ổn áp LDO RT9013-33GB (3.3V, 500mA).
   - Hệ thống phần mềm: Nền tảng ChirpStack (Network/Application Server), Framework Flutter (Mobile App), giao thức MQTT và HTTP GET/POST.
4. *Nội dung các phần thuyết minh và tính toán:*
   - **Chương 1:** Tổng quan kiến trúc hệ thống IoT LoRaWAN.Tổng quan kiến trúc và phân tích yêu cầu chức năng và phi chức năng của hệ thống chính.
   - **Chương 2:** Thiết kế chi tiết phần cứng Device:
     - Thiết kế sơ đồ nguyên lý và mạch in PCB trên phần mềm Altium Designer.
     - Thi công mạch và lắp ráp mô hình hoàn chỉnh.
     - Xây dựng Firmware 3 trạng thái: Giám sát, Xác nhận và Báo động.
   - **Chương 3:** Triển khai Mobile Application: Xây dựng dịch vụ chạy ngầm, tích hợp giao thức MQTT-HTTP GET/POST. Thiết kế luồng giải mã dữ liệu JSON.
   - **Chương 4:** Kiểm thử và đánh giá:
     - **Tính toán năng lượng:** Tính toán dòng tiêu thụ trung bình và ước tính

thời gian sử dụng pin

- Thực nghiệm đo đạc chỉ số cường độ tín hiệu (RSSI) và tỷ số tín hiệu trên nhiễu (SNR) trong các môi trường khác nhau.

5. *Các bản vẽ, đồ thị (ghi rõ các loại và kích thước bản vẽ):*
   - **Bản vẽ sơ đồ khối hệ thống:** Kiến trúc End-to-End từ Device đến Mobile App.
   - **Bản vẽ sơ đồ nguyên lý (Schematic):** Chi tiết các khối MCU, Nguồn, Cảm biến, Cảnh báo.
   - **Bản vẽ thiết kế PCB:** Layout 2D và mô hình 3D của thiết bịi.
   - **Lưu đồ thuật toán:** Lưu đồ logic máy trạng thái của Firmware và lưu đồ điều hướng của Mobile App.
   - **Đồ thị dòng điện tiêu thụ:** Biểu đồ phân tích dòng điện tức thời thu được từ máy đo chuyên dụng Otii Arc.
   - **Bảng biểu kết quả thực nghiệm:** Bảng biểu thể hiện tỉ lệ lỗi của các kịch bản thực nghiệm.
6. *Họ tên người hướng dẫn:* TS. Trần Văn Líc
   TS. Nguyễn Thị Bích Ngọc
7. *Ngày giao nhiệm vụ đồ án: ……/……/202…..*
8. *Ngày hoàn thành đồ án: ……/……/202…..*

Đà Nẵng, ngày tháng năm 202

Trưởng Bộ môn …….. **Người hướng dẫn**

LỜI NÓI ĐẦU

Đồ án tốt nghiệp này là kết quả của quá trình học tập, rèn luyện và nghiên cứu không ngừng nghỉ trong sut thời gian tôi theo học tại Trường Đại học Bách khoa Đà Nẵng. Để hoàn thành đồ án này, bên cạnh sự nỗ lực của bản thân, tôi đã nhận được rất nhiều sự quan tâm, động viên và giúp đỡ quý báu.

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc nhất đến **TS. Nguyễn Thị Bích Ngọc**, **TS. Trần Văn Líc** là những người đã tận tình hướng dẫn, định hướng đề tài và chỉ bảo những kiến thức chuyên môn vô cùng quý giá cho tôi trong suốt quá trình thực hiện đồ án. Sự tâm huyết và nghiêm túc trong khoa học của Thầy và Cô là tấm gương sáng để tôi noi theo.

Tôi xin chân thành cảm ơn các Quý Thầy, Cô giáo trong Khoa Điện tử-Viễn thông đã trang bị cho tôi nền tảng kiến thức vững chắc và tạo mọi điều kiện thuận lợi về cơ sở vật chất, phòng thí nghiệm để tôi có thể hoàn thành tốt nhiệm vụ của mình.

Cuối cùng, tôi xin gửi lời tri ân sâu sắc đến gia đình, bố mẹ - chỗ dựa tinh thần vững chắc nhất, cùng bạn bè đã luôn bên cạnh động viên, chia sẻ khó khăn và ủng hộ tôi trên con đường học tập.

Dù đã rất cố gắng, nhưng do hạn chế về thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn không tránh khỏi những thiếu sót. Tôi rất mong nhận được những ý kiến đóng góp quý báu của Quý Thầy, Cô và các bạn để đề tài được hoàn thiện hơn.

Xin chân thành cảm ơn!

CAM ĐOAN

Tôi là Dương Tấn Thịnh, Mã số sinh viên 106210100, đang học lớp 21DT2, Trường Đại học Bách khoa Đà Nẵng. Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi dưới sự hướng dẫn khoa học của TS. Trần Văn Líc và TS. Nguyễn Thị Bích Ngọc. Các nội dung nghiên cứu, kết quả thử nghiệm và số liệu trong đồ án này là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.

Các tài liệu tham khảo, số liệu trích dẫn đều được chỉ rõ nguồn gốc và tuân thủ các quy định về sở hữu trí tuệ. Giải pháp phần mềm và phần cứng được thiết kế và lập trình bởi chính tác giả (hoặc nhóm tác giả), không sao chép nguyên bản từ bất kỳ mã nguồn có sẵn nào mà không qua chỉnh sửa, phát triển.

Đà Nẵng, ngày tháng năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Dương Tấn Thịnh

MỤC LỤC

TÓM TẮT
NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP
LỜI NÓI ĐẦU ..... **i**
CAM ĐOAN ..... **ii**
DANH SÁCH CÁC BẢNG, HÌNH VẼ ..... **vi**
DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT ..... **ix**
MỞ ĐẦU **1**
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT **4**
1.1. Tổng quan kiến trúc của hệ thống LoraWAN: **4**
1.1.1. Các thành phần trong hệ thống LoraWAN 4
1.1.2. Phân loại lớp hoạt động 4
1.1.3. Các phương thức kích hoạt: 5
1.1.4. Cơ chế bảo mật: 5
1.2. Tổng quan kiến trúc của hệ thống báo cháy **5**
1.3. Phân tích yêu cầu hệ thống: **6**
1.3.1 Yêu cầu chức năng 6
1.3.2 Phân tích yêu cầu phi chức năng 7
1.3.3. Yêu cầu về bảo mật 8
1.3.4. Yêu cầu về tính khả dụng 8
1.4. Tiến độ thực hiện và phân định khối lượng công việc **8**
1.4.1. Các nội dung đã hoàn thành từ giai đoạn trước 8
1.4.2. Khối lượng công việc thực hiện trong đồ án tốt nghiệp: 8
1.5. Kết luận chương **9**
CHƯƠNG 2. THIẾT KẾ VÀ TRIỂN KHAI PHÂN HỆ THIẾT BỊ (DEVICE) **10**
2.1. Sơ đồ khối tổng quát **10**
2.2. Triển khai phân hệ phần cứng **10**
2.2.1. Sơ đồ khối tổng quan của phần cứng: 10
2.2.2 Thiết kế nguyên lý mạch in 11
2.2.3 Thiết kế bảng mạch in (PCB) 18
2.3.4 Hàn mạch và hoàn thiện phần cứng 20
2.3. Chương trình nhúng Firmware của Device **21**
2.3.1. Thiết các kế luồng hoạt động 21
2.3.2. Logic phát hiện sự kiện 26

2.3.3. Triển khai Stack LoRaWAN và Cơ chế kích hoạt OTAA.....27
2.4 Kết luận chương.....29
CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI PHÂN HỆ MOBILE APPLICATION 30
3.1. Tổng quan và công cụ phát triển.....30
3.1.1. Vai trò của phân hệ ứng dụng.....30
3.1.2. Môi trường và Công nghệ phát triển.....30
3.1.3. Các thư viện chính sử dụng.....30
3.2. Kiến trúc phần mềm.....30
3.3. Thiết kế cơ sở và mô hình dữ liệu.....32
3.4. Thiết kế luồng hoạt động một số dịch vụ.....33
3.4.1. Luồng hoạt động điều hướng màn hình khi mở ứng dụng.....33
3.4.2. Luồng chung của một RestfulAPI.....35
3.4.3. Luồng hoạt động khác.....36
3.5. Thiết kế giao diện người dùng (UI/UX).....37
3.5.1. Màn hình giám sát trung tâm (Dashboard): 37
3.5.2. Màn hình chi tiết thiết bị.....38
3.5.3. Tính năng quét QR.....39
3.5.4. Màn hình điều khiển thiết bị (Downlink LoRaWAN) 40
3.5.5. Màn hình cài đặt hệ thống.....41
3.5.6. Lịch sử và Truy xuất dữ liệu 41
3.6. Xây dựng các module cốt lõi.....42
3.6.1. Dịch vụ chạy ngầm.....42
3.6.2. Hệ thống cảnh báo.....43
3.6.3. Tự động phát hiện thiết bị 44
3.7. Kết luận chương.....44
CHƯƠNG 4: KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ.....45
4.1. Kiểm thử tính năng cơ bản 45
4.1.1. Cấu hình phần cứng và thiết bị đo lường.....45
4.1.2. Môi trường kiểm thử tính năng cơ bản 45
4.1.4. Kết quả kiểm thử tính năng cơ bản 46
4.1.5 Nhận xét kết quả kiểm thử 47
4.2. Kiểm thử năng lượng tiêu thụ 47
4.2.1. Cấu hình phần cứng và dụng cụ đo lường 47
4.2.2. Các hoạt động chính của thiết bị.....47

4.2.3. Tính thời gian sử dụng pin 50
4.2.4. Nhận xét kết quả kiểm thử 50
4.4. Kết luận chương 51
KẾT LUẬN 52
1. Đánh giá hệ thống 52
2. Kết luận chung 53
3. Hướng phát triển đề tài 53
TÀI LIỆU THAM KHẢO 55
PHỤ LỤC 1
Phụ luc 1. Hệ thống IOT 1
Phụ lục 2. JSON DATA 2
Phụ lục 3. Tiêu chuẩn JWT 2
Phụ lục 4. Tiêu chuẩn REST và RESTful API 3
Phụ lục 5. Mô hình giao tiếp Client – Server 5
Phụ lục 6: Giao thức truyền thông MQTT & Mô hình MQTT Broker 6
Phụ lục 7: Cross-platform Mobile Application 7
Phụ lục 8: Ngôn ngữ lập trình Dart 7
Phụ lục 9: Framework Flutter 8
Phụ lục 10: Xử lý tác vụ nền (Background Services) 8
Phụ lục 11: Lưu trữ cục bộ & Caching 9
Phụ lục 12: Thông báo cục bộ (Local Notification) 10
Phụ lục 13: Thao tác Bit (Bitwise Operations) 11
Phụ lục 14: Giao thức truyền thông UART 12
Phụ lục 15: Giao thức truyền thông I2C 12
Phụ lục 16: Bộ chuyển đổi tương tự - số (ADC) 13
Phụ lục 17: Giao thức OTAA (Over-The-Air Activation) 14
Phụ lục 18: Module RAK3172 WisDuo LoRaWAN 15
Phụ lục 19: Tổng quan buồng quang cảm biến khói 16
Phụ lục 20: Cảm biến nhiệt ẩm SHTC3 17
Phụ lục 21: IC ổn áp 9013-33GB 18
Phụ lục 22: BJT BC847B-NPN 19
Phụ lục 23: BJT 2SA1015LT1-PNP 20
Phụ lục 24: Cách cấu hình một node trên ChirpStack Server 20

DANH SÁCH CÁC BẢNG, HÌNH VẼ

Hình 1. 1 Mô hình tổng quan kiến trúc hệ thống LoraWAN 4
Hình 1. 2 Tổng quan kiến trúc của hệ thống 6
Hình 2. 1 Sơ đồ tổng quát của Device 10
Hình 2. 2 Sơ đồ tổng quan phần cứng 11
Hình 2. 3 Sơ đồ tổng quan chi tiết của phần cứng 12
Hình 2. 4 Nguyên lý mạch khối MCU 12
Hình 2. 5 Nguyên lý mạch khối nguồn 13
Hình 2. 6 Nguyên lý mạch khối cảm biến 14
Hình 2. 7 Nguyên lý mạch khối cảnh báo 15
Hình 2. 8 Nguyên lý mạch khối đo dung lượng pin 16
Hình 2. 9 Nguyên lý mạch khối nút bấm thao tác 17
Hình 2. 10 Nguyên lý mạch khối LED cảnh báo 17
Hình 2. 11 Nguyên lý mạch khối nạp và gỡ lỗi 18
Hình 2. 12 Mạch PCB thiết kế bằng Altium 18
Hình 2. 13 Mặt trên của PCB chế độ 2D 19
Hình 2. 14 Mặt dưới của PCB chế độ 2D 19
Hình 2. 15 Lớp sơn trên mặt PCB chế độ 2D 20
Hình 2. 16 Tổng thể hình 3D PCB với hình 3D của linh kiện 20
Hình 2. 17 Mạch sau khi đã hàn các linh kiện, mô-đun 21
Hình 2. 18 Mạch sau khi lắp và hoàn thiện 21
Hình 2. 19 Lưu đồ luồng hoạt động Giám sát 22
Hình 2. 20 Lưu đồ luồng hoạt động Xác nhận 24
Hình 2. 21 Lưu đồ luồng hoạt động Báo động 24
Hình 2. 22 Serial log khi có cháy 25
Hình 2. 23 Lưu đồ luồng hoạt động Downlink 25
Hình 2. 24 Lưu đồ luồng hoạt động Test 26
Hình 2. 25 Serial log sau khi thiết bị kết nối thành công NetworkServer 27
Hình 2. 26 Các khóa bảo mật cần thiết 28
Hình 2. 27 Cơ chế thử kết nối lại 28
Hình 2. 28 Serial log gửi tin thành công 29
Hình 2. 29 Serial log khi nhận downlink 29

Hình 3.1 Kiến trúc dự án flutter 31
Hình 3. 3 Dữ liệu JSON server trả về.....32
Hình 3. 4 Mô hình hóa dữ liệu JSON.....33
Hình 3. 5 Luồng điều hướng màn hình khi mở ứng dụng.....34
Hình 3. 6 Luồng chung của một RestfulAPI.....35
Hình 3. 7 Luồng hoạt động của Background Service.....36
Hình 3. 8 Luồng hoạt động của chế độ điều khiển.....37
Hình 3. 9 Màn hình giám sát trung tâm.....38
Hình 3. 10 Màn hình chi tiết thiết bị 38
Hình 3. 11 Màn hình chế độ quét QR.....39
Hình 3. 12 Màn hình chế độ điều khiển 40
Hình 3. 13 Server nhận downlink.....40
Hình 3. 14 Màn hình chế độ cài đặt.....41
Hình 3. 15 Màn hình lịch sử.....42
Hình 3. 16 Dịch vụ ngầm (Background Service) 43
Hình 3. 17 Màn hình cảnh báo cháy/test 43
Hình 4. 1 Kết quả của ĐKKT 1 46
Hình 4. 2 Kết quả của ĐKKT 2 47
Hình 4. 3 Dòng tiêu thụ khi ở chế độ đọc cảm biến.....48
Hình 4. 4 Dòng tiêu thụ khi ở chế độ gửi tin định kỳ.....48
Hình 4. 5 Dòng tiêu thụ khi ở chế độ mở cửa sổ RX 49
Hình 0. 1 Các lớp trong hệ thống IOT.....1
Hình 0. 3 API key được sử dụng trong dự án.....3
Hình 0. 5 Mô hình Client-Server.....6
Hình 0. 6 Mô hình MQTT 6
Hình 0. 7 Khởi tạo tiến trình chạy ngầm 9
Hình 0. 8 Khởi tạo chức năng lưu lịch sử.....10
Hình 0. 9 Tạo và hiển thị thông báo 11
Hình 0. 10 Dùng phép dịch bit 12
Hình 0. 11 Bản tin truyền nhận trong UART 12
Hình 0. 12 Mô hình I2C 13
Hình 0. 13 Mô hình hoạt động của ADC.....14
Hình 0. 14 Mô hình giao thức OTAA.....15
Hình 0. 15 Module RAK3172 16
Hình 0. 16 Cấu tạo buồng quang khói.....17

Hình 0. 17 Module SHTC3 18
Hình 0. 19 BJT BC847B.....20
Hình 0. 20 BJT 2SA1015LT1.....20
Hình 0. 21 Tạo mục Application 21
Hình 0. 22 Tạo Device.....21
Hình 0. 23 Các thông số cần thiết của Device.....22
Hình 0. 24 Các thông số được gán vào phần code firmware 22

DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT

Ký hiệu/Chữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
IoT	Internet of Things	Mạng lưới vạn vật kết nối
IP	Internet Protocol	Giao thức mạng
RF	Radio Frequency	Tần số vô tuyến
MQTT	Message Queuing Telemetry Transport	Giao thức Truyền tải Dữ liệu Viễn thông)
MobiApp	Mobile Application	Ứng dụng di động
OTAA	Over-The-Air Activation	Kích hoạt thiết bị qua mạng vô tuyến
LoraWAN	Long Range Wide Area Network	Giao thức mạng diện rộng công suất thấp dựa trên công nghệ LoRa
LPWAN	Low Power Wide Area Network	Mạng diện rộng công suất thấp
LoRa	Long Range	Công nghệ truyền thông vô tuyến tầm xa
MCU	Microcontroller Unit	Vi điều khiển
PCB	Printed Circuit Board	Mạch in
SNR	Signal-to-Noise Ratio	Tỷ số tín hiệu trên nhiễu
RSSI	Received Signal Strength Indicator	Chỉ số cường độ tín hiệu thu được
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tín hiệu tương tự sang số
API	Application Programming Interface	Giao diện lập trình ứng dụng
ID	Identifier	Định danh / Mã số nhận dạng
QR Code	Quick Response Code	Mã phản hồi nhanh
UI	User Interface	Giao diện người dùng
UX	User Experience	Trải nghiệm người dùng
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu đối tượng JavaScript
JS	Javascript	Tên một ngôn ngữ lập trình
UART	Universal Asynchronous Receiver/Transmitter	Bộ thu phát không đồng bộ phổ thông
I/O	Input/Ouput	Đầu vào/Đầu ra
IDE	Intergrated Development Environment	Môi trường phát triển tích hợp
REST	Representational State Transfer	Chuyển đổi trạng thái đại diện
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
GPS	Global Positioning System	Hệ thống định vị toàn cầu
TX	Transmitter	Bộ phát
RX	Receiver	Bộ thu
VIN	Input Voltage	Điện áp vào

Ký hiệu/Chữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
GND	Ground	Đất (mức 0V)
AppEUI	Application Extended Unique Identifier	Mã định danh duy nhất mở rộng của ứng dụng (trong LoRaWAN)
AppKey	Application Key	Khóa ứng dụng
DevEUI	Device Extended Unique Identifier	Mã định danh duy nhất mở rộng của thiết bị
AES	Advanced Encryption Standard	Tiêu chuẩn mã hóa nâng cao
I2C	Inter-Integrated Circuit	Giao thức truyền thông nối tiếp đồng bộ
LDO	Low-Dropout Regulator	Bộ ổn áp tuyến tính có độ sụt áp thấp
RH	Relative Humidity	Độ ẩm tương đối
ABS	Acrylonitrile Butadiene Styrene	Một loại nhựa kỹ thuật
ĐKKT		Điều kiện kiểm thử
JWT	JSON Web Token	Chuỗi mã
IC	Integrated circuit	Mạch tích hợp
URL	Uniform Resource Locator	Địa chỉ duy nhất
URI	Uniform Resource Identifier	Mã định danh duy nhất
PSRR	Power Supply Rejection Ratio	Tỷ lệ khử nhiễu nguồn cấp
ALARM_ON_THRESHOLD	Alarm on threshold	Ngưỡng báo động
	Device/Node	Thiết bị
	Gateway	Cổng mạng
	Broker	Môi giới
	Cache Strategy	Chiến lược bộ nhớ cache
	Key-Value Store	Kho lưu trữ dạng Khóa – Giá trị
	Perception Layer	Tầng nhận thức
	Network Layer	Tầng mạng
	Processing Layer	Tầng xử lý
	Application Layer	Tầng ứng dụng
	Manipulating	Điều khiển
	Module	Mô-đun
	Sensor	Cảm biến
	Power	Nguồn điện
	Communicate	Giao tiếp
	Alert	Báo động
	Realtime	Thời gian thực
	Trigger	Kích hoạt
	Snoozed	Hoãn báo động

MỞ ĐẦU

1. Mục đích thực hiện đề tài

Trong bối cảnh đô thị hóa và công nghiệp hóa diễn ra mạnh mẽ tại Việt Nam, vấn đề an toàn phòng cháy chữa cháy (PCCC) đang trở thành mối quan tâm cấp thiết.

Trong tháng 11/2025, toàn quốc ghi nhận 211 vụ cháy, khiến 4 người tử vong, 4 người bị thương và thiệt hại tài sản ước tính khoảng 30,28 tỷ đồng; đồng thời xảy ra 01 vụ nổ làm 01 người bị thương. Tỷ lệ cháy chủ yếu tập trung ở nhà dân (37,9%), kho, cơ sở sản xuất – kinh doanh và phương tiện giao thông. Phần lớn nguyên nhân được xác định là do sự cố hệ thống, thiết bị điện (85,8%) và sơ suất trong sử dụng nguồn lửa, nguồn nhiệt.

Tiếp đó vào tháng 12/2025, số vụ cháy tiếp tục tăng với 287 vụ, gây 7 người tử vong, 22 người bị thương và thiệt hại tài sản lên tới hơn 229,7 tỷ đồng; đồng thời có 01 vụ nổ làm 01 người bị thương. Tình hình cháy nổ diễn biến phức tạp với tỷ lệ cháy tại nhà dân vẫn cao nhất (38,3%). Trong các vụ đã làm rõ nguyên nhân, phần lớn đều do sự cố hệ thống điện và sơ suất con người.

Tuy nhiên, với số liệu đáng lo ngại như vậy thì các giải pháp báo cháy hiện nay đang bộc lộ nhiều hạn chế về mặt kỹ thuật truyền thông:

- **Hệ thống có dây:** Tin cậy nhưng chi phí triển khai hạ tầng quá lớn, khó thi công tại các công trình cũ hoặc nhà cao tầng đã hoàn thiện.
- **Hệ thống không dây tầm ngắn (Wi-Fi/Zigbee/Bluetooth):** Phụ thuộc nhiều vào nguồn điện lưới (do tiêu thụ năng lượng cao), khoảng cách truyền tin ngắn và khả năng xuyên vật cản (tường bê tông, hầm kín) kém, không đảm bảo duy trì kết nối trong các tòa nhà phức tạp.

Để giải quyết bài toán "Phủ sóng rộng - Xuyên vật cản tốt - Tiết kiệm năng lượng", công nghệ LoRaWAN (Long Range Wide Area Network) nổi lên như một giải pháp tối ưu cho mạng diện rộng công suất thấp (LPWAN). Với đặc tính kỹ thuật vượt trội: khả năng truyền tin hàng kilomet trong đô thị, sử dụng điều chế trải phổ Chirp (CSS) để chống nhiễu và tối ưu hóa năng lượng cho thiết bị chạy pin lên đến hàng năm.

Xuất phát từ nhu cầu thực tiễn và ưu điểm công nghệ, đồ án "Thiết kế và triển khai hệ thống cảnh báo cháy sử dụng công nghệ LoRaWAN" tập trung nghiên cứu ứng dụng giao thức này để xây dựng một mạng lưới giám sát cháy tin cậy, khắc phục "điểm mù" về khoảng cách và năng lượng mà các công nghệ cũ chưa giải quyết được.

2. Mục tiêu đề tài

Mục tiêu tổng quát của đồ án là nghiên cứu, thiết kế và chế tạo hoàn chỉnh một hệ thống giám sát hỏa hoạn end-to-end, từ nút cảm biến đến ứng dụng người dùng. Các mục tiêu cụ thể bao gồm:

- **Nghiên cứu lý thuyết:** Tìm hiểu sâu về giao thức LoRaWAN, chuẩn giao tiếp OTAA và băng tần hoạt động AS923 phù hợp với quy chuẩn Việt Nam.
- **Thiết kế phần cứng:** Xây dựng nút cảm biến hoạt động bằng pin, tích hợp module thu phát LoRa và buồng đo khói quang học. Mạch cần đảm bảo tiêu chí tiết kiệm năng lượng tối đa.

- **Xây dựng thuật toán:** Lập trình vi điều khiển với cơ chế máy trạng thái để xử lý tín hiệu khói, lọc nhiễu, gửi gói tin Uplink định kỳ hoặc khẩn cấp, và khả năng nhận lệnh Downlink từ server.
- **Phát triển ứng dụng di động:** Xây dựng ứng dụng trên nền tảng Flutter (Android/iOS) cho phép người dùng nhận cảnh báo tức thời, giám sát nồng độ khói, dung lượng pin và điều khiển cài đặt thiết bị từ xa.

3. Đối tượng và phạm vi nghiên cứu:

3.1. Đối tượng nghiên cứu:

- Công nghệ truyền thông không dây LoRa/LoRaWAN.
- Kỹ thuật đo lường các thông số môi trường (nồng độ khói) bằng cảm biến điện tử.
- Kỹ thuật lập trình ứng dụng di động và giao tiếp với Application server.

3.2. Phạm vi nghiên cứu:

- **Về không gian:** Hệ thống được thiết kế để hoạt động trong môi trường trong nhà như hộ gia đình, văn phòng, nhà kho vừa và nhỏ.
- **Về kỹ thuật:**
  - Tập trung phát hiện khói và nhiệt là những dấu hiệu chính của đám cháy.
  - Giao tiếp LoRaWAN Class A (tối ưu cho thiết bị chạy pin).
  - Ứng dụng di động thực hiện trọn vẹn các chức năng giám sát và cảnh báo.
- **Giới hạn:** Đồ án tập trung vào việc thiết kế Node thiết bị và Ứng dụng. Phần hạ tầng mạng (Gateway, Network Server) sử dụng các nền tảng có sẵn hoặc thiết bị thương mại để giả lập môi trường mạng.

3.3. Đối tượng hướng đến và Phạm vi ứng dụng thực tiễn

Hệ thống được thiết kế tối ưu cho các nhóm đối tượng có đặc thù khó khăn trong việc thi công hệ thống báo cháy có dây truyền thống:

a. **Hộ gia đình và nhà ở đã hoàn thiện:** Giải quyết nhu cầu lắp đặt nhanh, đảm bảo thẩm mỹ do không cần đục tường đi dây tín hiệu.

b. **Chung cư mini và khu nhà trọ:** Tận dụng khả năng xuyên vật cản vượt trội của sóng LoRa để duy trì kết nối ổn định từ các phòng kín, tầng hầm về thiết bị trung tâm.

c. **Kho bãi và nhà xưởng vừa/nhỏ:** Tận dụng ưu thế vùng phủ sóng rộng (Wide Area) của LoRaWAN để giám sát diện tích lớn với chi phí hạ tầng mạng thấp nhất.

4. Phương pháp nghiên cứu:

Để đạt được các mục tiêu đề ra, đồ án áp dụng các phương pháp nghiên cứu và triển khai sau:

- **Phương pháp nghiên cứu tài liệu:** Tổng hợp tài liệu kỹ thuật của các module LoRa (RAK3172/RAK4630...), cảm biến quang và tài liệu chuẩn LoRaWAN Specification.
- **Phương pháp thiết kế hệ thống nhúng:**
  - Sử dụng nguyên lý cảm biến khói dạng quang điện: Kết hợp LED phát hồng ngoại và Photodiode để phát hiện mật độ khói dựa trên hiện tượng tán xạ ánh sáng.
  - Tối ưu hóa năng lượng: Sử dụng các chế độ ngủ (Sleep Mode) của vi điều khiển và lập lịch gửi tin hợp lý.
- **Phương pháp lập trình ứng dụng:** Sử dụng Flutter Framework và ngôn ngữ Dart để xây dựng ứng dụng đa nền tảng. Sử dụng kiến trúc MQTT và RestfulAPI để quản lý trạng thái ứng dụng.

- **Phương pháp thực nghiệm:** Triển khai mô hình thực tế, sử dụng khói tạo mẫu để kiểm thử độ nhạy, đo đạc cường độ tín hiệu (RSSI/SNR) và độ trễ gói tin trong môi trường đô thị.

5. Tiêu chí đánh giá hệ thống

a. Độ chính xác (Accuracy): Hệ thống phải có khả năng phát hiện khói chính xác khi mật độ khói vượt ngưỡng cài đặt, đồng thời giảm thiểu tối đa báo động giả (False Alarm) thông qua thuật toán xác thực đa bước (Confirming State).

b. Độ trễ truyền tin (Latency): Thời gian từ lúc cảm biến phát hiện nguy cơ cháy đến khi ứng dụng di động hiển thị cảnh báo phải nằm trong ngưỡng chấp nhận được (thường < 10 giây đối với mạng LoRaWAN Class A trong điều kiện tiêu chuẩn).

c. Chất lượng kết nối (Connectivity): Đảm bảo tỷ lệ mất gói tin (Packet Loss Rate) dưới 5% trong phạm vi phủ sóng thiết kế. Các chỉ số RSSI (Cường độ tín hiệu) và SNR (Tỷ số tín hiệu trên nhiễu) phải đảm bảo ngưỡng thu của Gateway.

d. Hiệu quả năng lượng (Energy Efficiency): Thiết bị phải hoạt động ổn định ở chế độ tiết kiệm năng lượng (Sleep Mode) với dòng tiêu thụ thấp (cỡ µA) để đảm bảo tuổi thọ pin kéo dài, hạn chế việc phải thay pin thường xuyên.

e. Tính ổn định của Ứng dụng: Mobile App phải hoạt động mượt mà, nhận Push Notification ngay cả khi chạy ngầm (Background) và cập nhật trạng thái thời gian thực từ Server.

f. Tính quản lý: Có khả năng đồng thời quản lý nhiều thiết bị cảnh báo cùng lúc

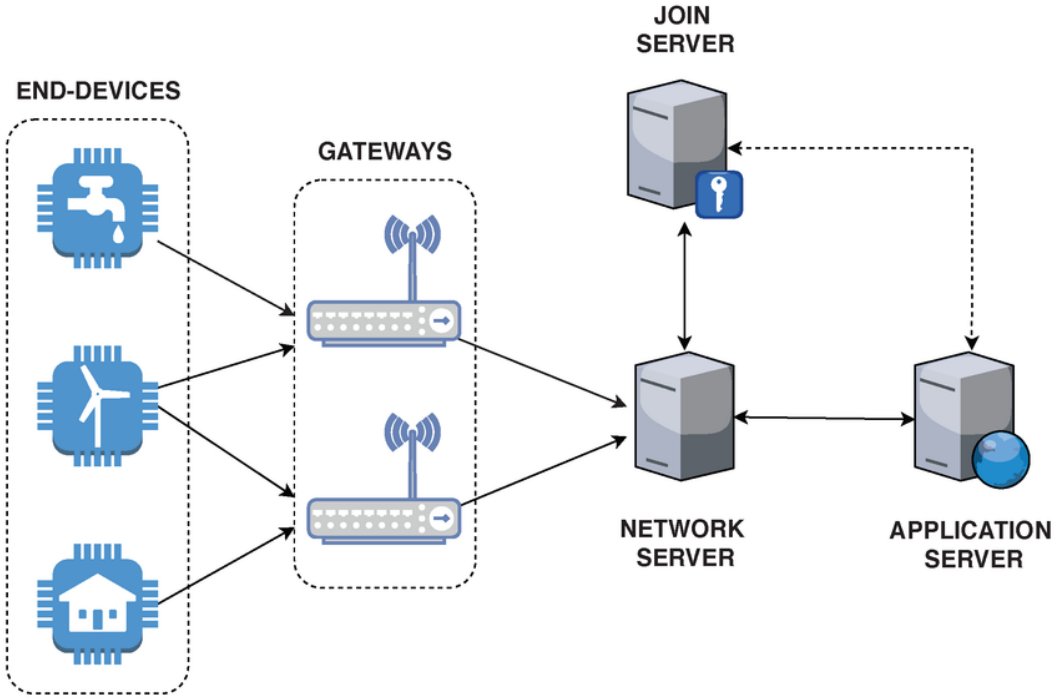
6. Cấu trúc đồ án: Ngoài phần mở đầu và kết luận, đồ án được kết cấu thành 4 chương chính như sau:

- Chương 1: Cơ sở lý thuyết. Trình bày tổng quan về kiến trúc IOT của hệ thống; những yêu cầu chức năng, phi chức năng của toàn hệ thống cũng như riêng từng phân hệ trong hệ thống.
- Chương 2: Thiết kế và triển khai phân hệ thiết bị (Device). Trình bày về cách thức triển khai phân hệ Device từ việc thiết kế kiến trúc, xây dựng mạch PCB, xây dựng chương trình nhúng, hoàn thiện Device trong thực tế v.v.
- Chương 3: Thiết kế và triển khai phân hệ Mobile Application. Trình bày cách thức xây dựng MobiApp của hệ thống; một số bước chính để cấu hình, giao tiếp giữa MobiApp với Server thông qua MQTT.
- Chương 4: Kiểm thử và đánh giá kết quả. Trình bày nội dung của một số trường hợp kiểm thử, từ đó giúp đánh giá khả năng hoạt động của hệ thống trong một số điều kiện khác nhau.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan kiến trúc của hệ thống LoraWAN:

Một hệ thống LoraWAN tiêu chuẩn gồm những thành phần sau: Lớp End-Devices (Thiết bị, cảm biến,...), Lớp Gateway, Lớp Network Server, Lớp Application Server [2].



Hình 1. 1 Mô hình tổng quan kiến trúc hệ thống LoraWAN

Mô tả chi tiết các phân hệ trong kiến trúc:

1.1.1. Các thành phần trong hệ thống LoraWAN

a. Lớp End-Devices (Thiết bị đầu cuối): Là các nút cảm biến hoạt động bằng pin, có nhiệm vụ thu thập dữ liệu từ môi trường, xử lý sơ bộ tín hiệu để lọc nhiễu và đóng gói dữ liệu theo giao thức LoRaWAN.

b. Lớp Gateway: Đóng vai trò cầu nối trung gian, nhận tín hiệu RF từ End-Device và chuyển tiếp gói tin IP về Network Server.

c. Lớp Network Server: Quản lý xác thực thiết bị (Join Server), loại bỏ các gói tin trùng lặp và điều phối luồng dữ liệu (Uplink/Downlink).

d. Lớp Application Server: Là nơi giải mã dữ liệu từ Network Server.

1.1.2. Phân loại lớp hoạt động

Trong chuẩn LoRaWAN, các thiết bị đầu cuối (end devices) được phân loại theo cách thức giao tiếp với mạng, nhằm đáp ứng các yêu cầu về năng lượng, độ trễ và tính sẵn sàng nhận lệnh trong các ứng dụng IoT khác nhau. Có ba lớp thiết bị: Class A, Class B và Class C.

a. Lớp A (Class A):

- **Nguyên lý:** Thiết bị chỉ mở các cửa sổ nhận (RX) sau khi gửi uplink, với hai cửa sổ ngắn RX1 và RX2 để nhận lệnh từ mạng sau khi gửi dữ liệu. Network Server chỉ có thể gửi lệnh trong hai cửa sổ nhận sau uplink; nếu không, server phải đợi uplink tiếp theo.

- **Đặc điểm nổi bật:** Tiêu thụ năng lượng thấp nhất vì thiết bị dành phần lớn thời gian ở chế độ ngủ.

b. Lớp B (Class B):

- **Nguyên lý:** Bổ sung thêm các cửa sổ nhận theo lịch định kỳ ngoài các cửa sổ sau uplink của Class A. Thiết bị nhận beacon đồng bộ từ gateway để biết chính xác thời điểm mở các ping slot, cho phép Network Server gửi lệnh tại các thời điểm đó.
- **Đặc điểm nổi bật:** Độ trễ lệnh thấp hơn Class A do có thêm khe nhận theo thời gian, nhưng tiêu thụ năng lượng cao hơn Class A.

c. Lớp C (Class C):

- **Nguyên lý:** Thiết bị mở liên tục cửa sổ nhận (RX) khi không đang gửi uplink, giúp Network Server có thể gửi lệnh gần như bất cứ lúc nào.
- **Đặc điểm nổi bật:** Cho phép độ trễ lệnh thấp nhất trong ba lớp, nhưng tiêu thụ năng lượng **cao nhất** do thu sóng liên tục. Sử dụng nguồn trực tiếp là chủ yếu.

1.1.3. Các phương thức kích hoạt:

a. Phương thức ABP:

Khóa phiên và địa chỉ thiết bị được gán cố định vào firmware trước khi triển khai, bỏ qua bước Join Request/Join Accept.

Đây là phương thức ít an toàn hơn do khóa tĩnh, khó thay đổi và dễ bị khai thác nếu thiết bị bị truy cập vật lý.

b. Phương thức OTAA.

Thiết bị gửi **Join Request** đến mạng, Server phản hồi **Join Accept** và tại thời điểm này hai khóa phiên mới là **NwkSKey** và **AppSKey** được phát sinh ngẫu nhiên và phân phối cho thiết bị và server (Phụ lục 17).

Nhờ việc tạo khóa phiên tại mỗi lần join, hệ thống có thể hạn chế nguy cơ bị tấn công khi khóa bị lộ và cho phép cập nhật lại khóa theo chu kỳ.

1.1.4. Cơ chế bảo mật:

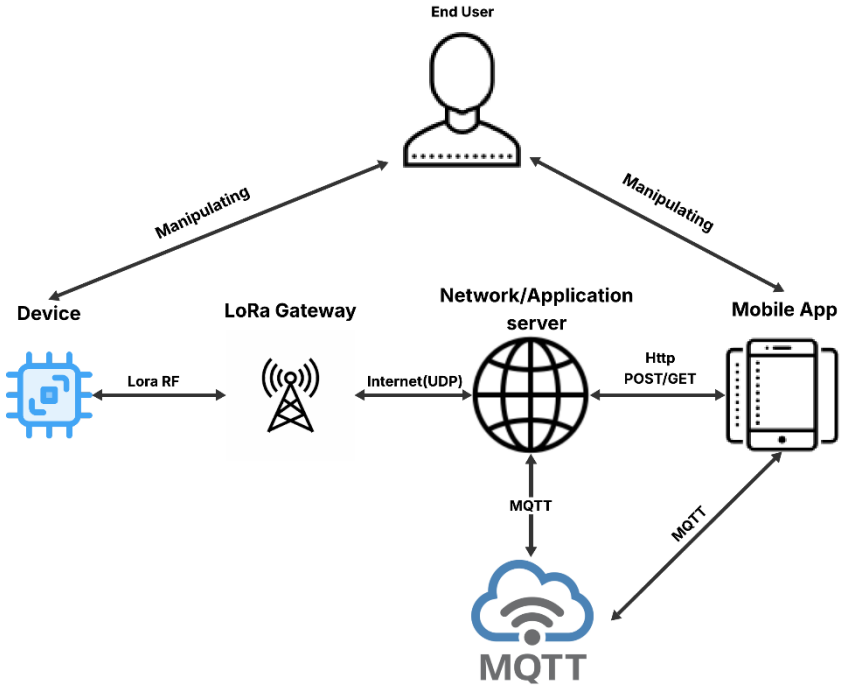
a. Khóa và mã hóa: LoRaWAN sử dụng các khóa 128-bit theo tiêu chuẩn AES-128 để bảo vệ dữ liệu và xác thực.

- AppKey: khóa gốc dùng trong OTAA để tạo khóa phiên.
- NwkSKey: khóa phiên mạng dùng để kiểm tra tính toàn vẹn của mỗi gói
- AppSKey: khóa phiên ứng dụng dùng để mã hóa và giải mã payload giữa thiết bị và Application Server

Việc phân tách khóa giữa lớp mạng và lớp ứng dụng giúp bảo vệ dữ liệu người dùng ngay cả khi một phần hệ thống bị truy cập trái phép.

b. Ngăn chặn phát lại: LoRaWAN sử dụng **frame counter** độc lập cho uplink và downlink. Nếu một gói có số bộ đếm thấp hơn giá trị hiện tại, nó bị loại bỏ, giúp chống tấn công tái phát gói.

1.2. Tổng quan kiến trúc của hệ thống báo cháy



Hình 1. 2 Tổng quan kiến trúc của hệ thống

Hệ thống được xây dựng dựa theo kiến trúc LoraWAN với 3 phân hệ chính là: Device, Network/Application Server và Mobile App.được ứng dụng vào các lớp như sau:

- Lớp End-Device (Thiết bị đầu cuối): Đây là phân hệ phần cứng (Device) hoạt động tại hiện trường. Nhiệm vụ chính là thu thập các thông số môi trường thời gian thực như nồng độ khói và nhiệt độ. Dữ liệu sau khi thu thập sẽ được xử lý sơ bộ và đóng gói để gửi lên lớp tiếp theo thông qua thiết bị trung gian là Gateway.
- Lớp Network Server: Đóng vai trò là trung tâm quản lý kết nối. Lớp này chịu trách nhiệm xác thực thiết bị gia nhập mạng, tiếp nhận các gói tin dữ liệu (Uplink) từ End-Device chuyển đến. Đồng thời, đây cũng là luồng trung gian quan trọng để truyền tải các lệnh điều khiển (Downlink) từ Application Server xuống thiết bị.
- Lớp Application Server: Tại đây, các gói tin thô từ Network Server sẽ được giải mã và chuyển đổi sang định dạng tiêu chuẩn (JSON) để xử lý nghiệp vụ. Đây là cổng giao tiếp chính với ứng dụng di động (Mobile App) thông qua các giao thức mạng phổ biến như HTTP RESTful API (cho các tác vụ cấu hình) và MQTT (cho việc nhận cảnh báo thời gian thực).

1.3. Phân tích yêu cầu hệ thống:

Để đảm bảo tính thực tiễn và độ tin cậy, hệ thống cần thỏa mãn các nhóm yêu cầu chức năng và phi chức năng dưới đây.

1.3.1 Yêu cầu chức năng

Đây là các chức năng cụ thể mà hệ thống phải thực hiện được, chia theo từng phân hệ.

a. Đối với phân hệ Device (End-Node)

- **Đo lường tín hiệu khói-nhiệt:** Thiết bị phải đọc được giá trị điện áp từ cảm biến

quang điện (Photodiode) và cảm biến nhiệt để xác định mật độ khói và nhiệt độ trong không khí.

- **Cơ chế xác thực báo động:** Để tránh báo giả, khi phát hiện khói vượt ngưỡng, thiết bị không báo cháy ngay mà chuyển sang trạng thái "Chờ xác nhận" (Confirming State). Nếu tín hiệu khói vẫn duy trì sau khoảng thời gian kiểm tra lại, mới chính thức kích hoạt báo động.
- **Giao tiếp LoraWAN:**
  - Hỗ trợ kích hoạt thiết bị qua phương thức OTAA (Over-The-Air Activation).
  - Gửi bản tin Uplink định kỳ chứa thông tin: Mức khói, nhiệt, level cảnh báo và dung lượng pin.
  - Gửi bản tin Uplink khẩn cấp ngay lập tức khi có cháy.
- **Xử lý lệnh điều khiển:** Thiết bị phải có khả năng nhận và giải mã bản tin Downlink từ Server để thay đổi ngưỡng báo động (ALARM_ON_THRESHOLD) mà không cần nạp lại code.
- **Cảnh báo tại chỗ:** Kích hoạt còi (Buzzer) và đèn LED cảnh báo khi phát hiện cháy.

b. Đối với phân hệ Ứng dụng di động (Mobile Application)

- **Giám sát thời gian thực:** Hiển thị danh sách các thiết bị, trạng thái kết nối, mức pin, nồng độ khói và nhiệt độ hiện tại dưới dạng trực quan.
- **Nhận thông báo:** Ứng dụng phải hiển thị thông báo đẩy (Push Notification) có âm thanh cảnh báo ngay cả khi điện thoại đang ở chế độ khóa hoặc ứng dụng chạy ngầm
- **Tra cứu lịch sử:** Cho phép xem lại các nhật ký các lần báo động trước đó.
- **Điều khiển từ xa:** Cung cấp gia điện dể người dùng gửi lệnh cài đặt ngưỡng báo động xuống thiết bị
- **Thêm/Thay đổi nhóm thiết bị giám sát:** Ứng dụng có thể quét QR code để thêm nhóm thiết bị muốn giám sát.

c. Đối với phân hệ Hạ tầng mạng và Backend (Cấu hình và tích hợp)

Hệ thống sử dụng nền tảng ChirpStack mã nguồn mở làm Network Server

- **Cấu hình Gateway và Server:** Thiết lập thông số Gateway để kết nối ổn định với ChirpStack Server. Cấu hình Profile thiết bị (Device Profile) trên Server tương thích với phần cứng (chuẩn LoraWAN 1.0.3, Class A).
- **Quản lý xác thực (Join Server):** Hệ thống phải cấp phát và quản lý các khóa bảo mật (AppKey, NwkSKey) chính xác để thiết bị có thể Join mạng thành công.
- **Tích hợp MQTT Broker:** Cấu hình Chirpstack để xuất bản dữ liệu giải mã rra MQTT topic chuẩn, giúp Mobile App có thể đăng ký và nhận dữ liệu ngay lập tức.

1.3.2 Phân tích yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống phảo thỏa mãn các tiêu chuẩn kỹ thuật khắc khe sau:

a. Yêu cầu về hiệu năng

- Độ trễ cảnh báo: Thời gian từ lúc cảm biến xác nhận có khói đến khi điện thoại nhận thông báo không được vượt quá 10 giây trong điều kiện mạng tiêu chuẩn.
- Độ nhạy: Cảm biến phải phát hiện được khói và nhiệt độ ở giai đoạn đầu trước

khi bùng thành ngọn lửa lớn.

b. Yêu cầu về năng lượng

Đây là yêu cầu quan trọng nhất đối với thiết bị chạy pin.

- Thiết bị phải duy trì dòng tiêu thụ ở mức thấp (cỡ μA) trong trạng thái ngủ (sleep mode)
- Tuổi thọ pin phải đảm bảo hoạt động liên tục tối thiểu 8 tháng tới 1 năm với điều kiện hoạt động ổn định.

c. Yêu cầu về độ tin cậy

- Cơ chế xác thực: Hệ thống phải giảm thiểu tối đa tỷ lệ gửi tin giả, gây phiền toái cho người dùng.
- Khả năng tự phục hồi kết nối: Thiết bị phải có khả năng tự kết nối lại mạng nếu như bị mất kết nối tạm thời.

1.3.3. Yêu cầu về bảo mật

Hệ thống sử dụng mã hóa AES-128 chuẩn giao thức LoRaWAN cho cả hai lớp Network Session và Application Session, đảm bảo dữ liệu không bị nghe lén hoặc giả mạo.

1.3.4. Yêu cầu về tính khả dụng

Giao diện ứng dụng phải đơn giản, dễ hiểu, ưu tiên màu sắc cảnh báo rõ ràng (Đỏ cho nguy hiểm, Xanh cho an toàn) để người dùng phản ứng nhanh trong tình huống khẩn cấp.

1.4. Tiến độ thực hiện và phân định khối lượng công việc

1.4.1. Các nội dung đã hoàn thành từ giai đoạn trước

a) ***Về phần cứng*:**

- Đã hoàn thiện thiết kế sơ ddoogf nguyên lý và sơ đồ mạch in.
- Đã hoàn tất việc gia công mạch in, hàn linh kiện và lắp ráp phần cứng hoàn chỉnh

b) Về kiểm thử cơ bản:

- Đã viết code kiểm tra (Test Code) để đảm bảo các khối chức năng hoạt động tốt: Khối nguồn, khối đo khói và vi điều khiển RAK3172.
- Đã chạy thử nghiệm hệ thống ở mức cơ bản (đọc giá trị cảm biến và hiển thị lên Serial Monitor).

1.4.2. Khối lượng công việc thực hiện trong đồ án tốt nghiệp:

a) Phát triển Firmware chuyên sâu (Phần lõi)

- Thay thế các đoạn code test đơn giản bằng Giải thuật Máy trạng thái hoàn chỉnh với cơ chế "Xác thực kép" (Monitoring - Confirming - Alarming) để lọc nhiễu.
- Lập trình giao thức LoRaWAN OTAA (Over-The-Air Activation) để thiết bị có thể gia nhập mạng bảo mật và gửi dữ liệu đi xa (thay vì chỉ test cục bộ).
- Tối ưu hóa năng lượng để thiết bị có thể hoạt động bằng pin lâu dài.
- Tích hợp thêm cảm biến SHTC3 để thiết bị có thể nhận biết nhiều loại đám cháy.

b) Xây dựng và Hoàn thiện Ứng dụng di động (Mobile App):

- Thiết kế toàn bộ giao diện người dùng (UI/UX) trên nền tảng Flutter.
- Xử lý kỹ thuật Dịch vụ chạy ngầm (Background Service) để nhận cảnh báo ngay cả khi tắt ứng dụng – Đây là tính năng phức tạp nhất của App.
- Tích hợp kết nối MQTT và tính năng quét mã QR để quản lý thiết bị.

c) Đo đạc và Đánh giá hệ thống:

- Thực hiện các bài đo kiểm thực tế: Đo dòng tiêu thụ, đo khoảng cách truyền tin (Range Test) và đo độ trễ cảnh báo (Latency),…

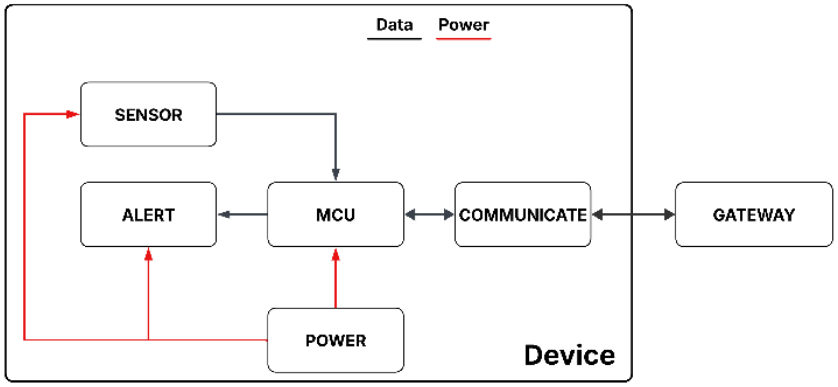
- Đóng gói sản phẩm và viết báo cáo thuyết minh.

1.5. Kết luận chương

Chương 1 đã trình bày tổng quan về kiến trúc hệ thống dựa trên nền tảng IoT và công nghệ LoRaWAN cũng như kiến trúc của hệ thống. Các yêu cầu chức năng và phi chức năng đã được phân tích chi tiết làm cơ sở định hướng cho việc thiết kế phần cứng và phần mềm trong chương tiếp theo. Phân hệ thiết bị sẽ tập trung vào tối ưu năng lượng và thuật toán xử lý tín hiệu, trong khi phân hệ ứng dụng tập trung vào trải nghiệm người dùng và tốc độ cảnh báo.

CHƯƠNG 2. THIẾT KẾ VÀ TRIỂN KHAI PHÂN HỆ THIẾT BỊ (DEVICE)

2.1. Sơ đồ khối tổng quát



Hình 2. 1 Sơ đồ tổng quát của Device

Phân hệ device gồm các khối sau:
-MCU: khối đóng vai trò trung tâm của phân hệ, nơi sẽ tính toán, phát hiện sự cố, điều khiển mọi hoạt động thu, nhận dữ liệu
-SENSOR: Bao gồm hai thành phần:

- Cảm biến khói quang điện (Photoelectric Smoke Sensor).
- Cảm biến Nhiệt độ - Độ ẩm kỹ thuật số SHTC3 (giao tiếp I2C).

-POWER: Ổn áp LDO và mạch giám sát pin.
-ALERT: khối phát cảnh báo cho phân hệ.
-COMMUNICATE: khối giúp Device giao tiếp với hệ thống (các phân hệ khác)

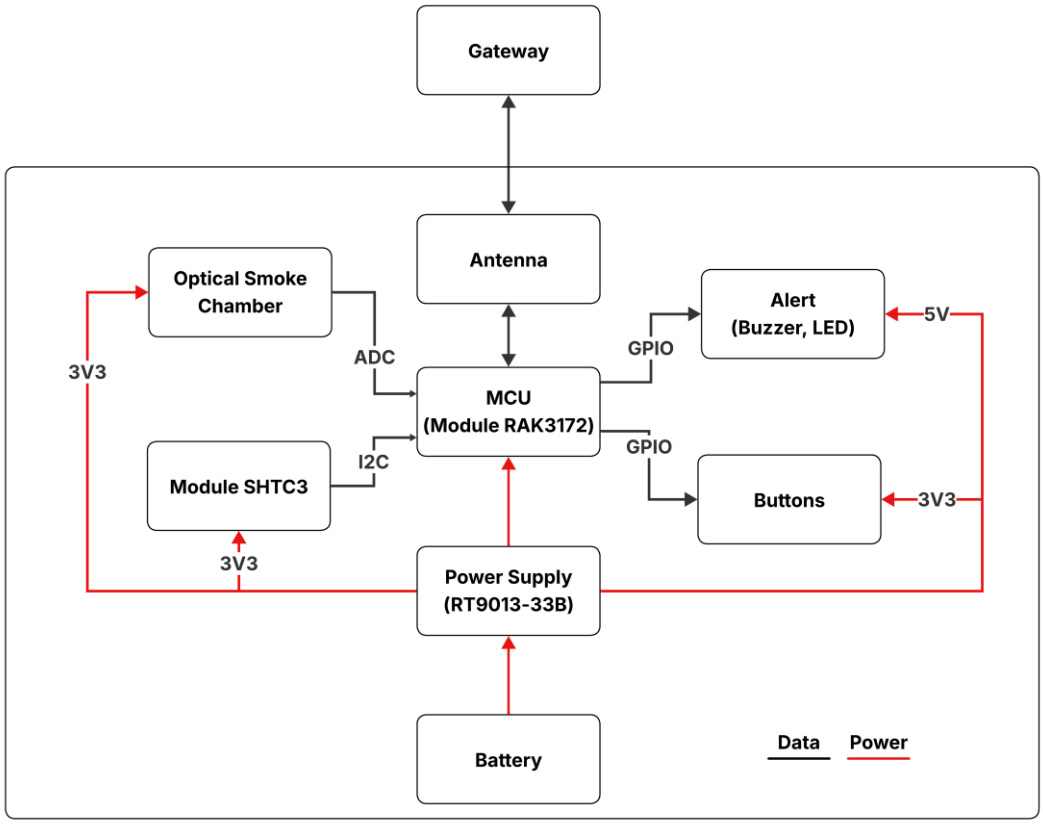
2.2. Triển khai phân hệ phần cứng

2.2.1. Sơ đồ khối tổng quan của phần cứng:

Dựa vào sơ đồ tổng quan của Device, phần cứng sẽ được triển khai với sơ đồ khối, linh kiện của từng khối, cách thức giao tiếp giữa các khối như sau:

-Khối MCU: MCU RAK3172
-Khối POWER: 3 Pin 3A, mô-đun ổn áp RT9013-33B.
-Khối SENSOR: mô-đun cảm biến khói buồng quang, mô-đun cảm biến nhiệt-ẩm SHTC3.
-Khối COMMUNICATE: Antenna.
-Khối ALERT: Còi, đèn led.
-Nút bấm thao tác với Device.

Thông tin chung, thông số kỹ thuật, đặc điểm ,… của các linh kiện được đề cập tại phụ lục

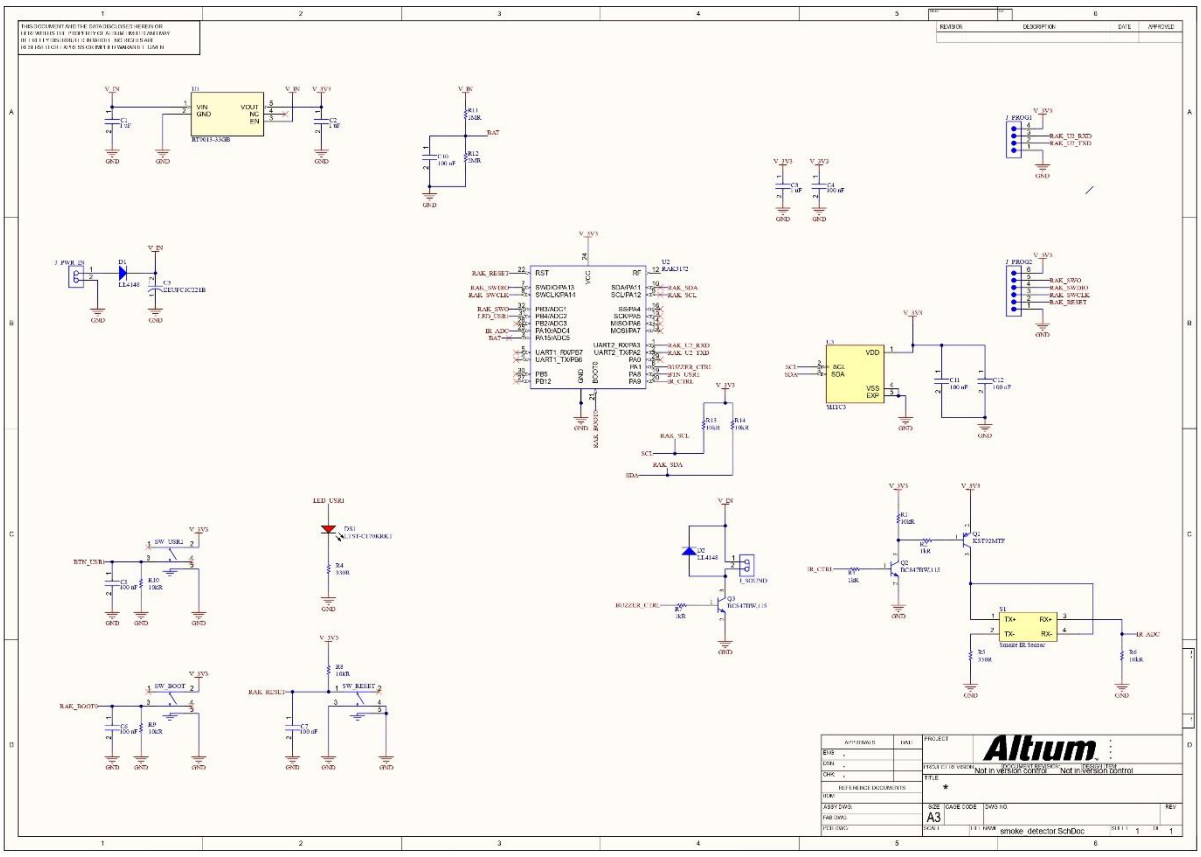


Hình 2. 2 Sơ đồ tổng quan phần cứng

2.2.2 Thiết kế nguyên lý mạch in

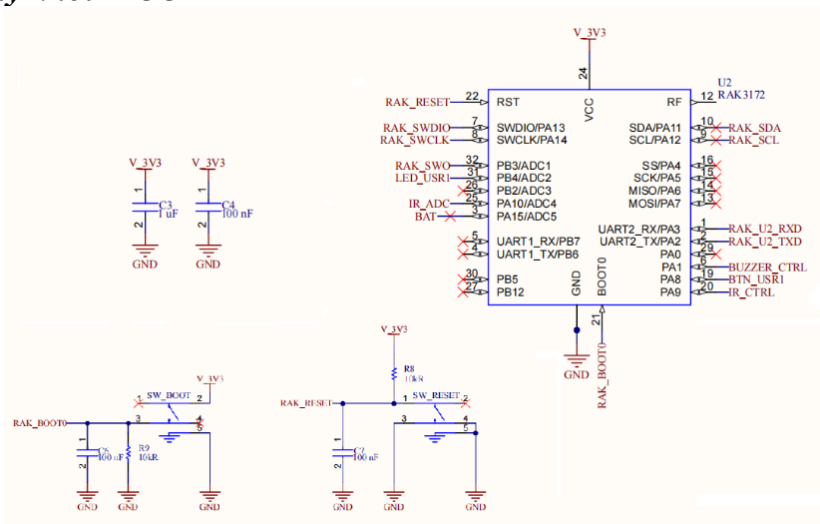
Nguyên lý mạch in của phần cứng được thiết kế trên phần mềm thiết kế mạch điện tử chuyên dụng Altium Designer (Phiên bản 23.9.2).

a. Tổng quan kiến trúc nguyên lý mạch in



Hình 2. 3 Sơ đồ tổng quan chi tiết của phần cứng

b. Nguyên lý khối MCU



Hình 2. 4 Nguyên lý mạch khối MCU

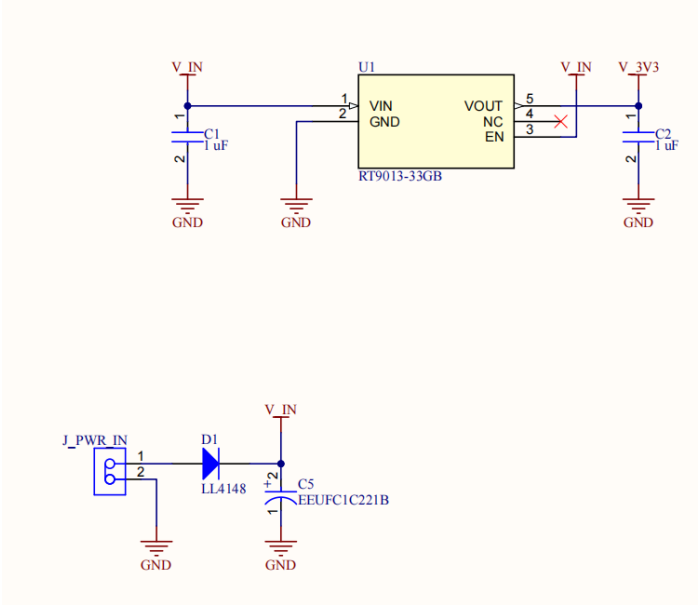
- Module **RAK3172** là "bộ não" của toàn bộ hệ thống. Nó là một module tích hợp chip SoC STM32WLE5, hỗ trợ giao thức LoRaWAN để truyền tin đi xa và xử lý các tín hiệu từ cảm biến. Nguồn được cấp qua chân VCC, bên cạnh được lắp 2 tụ chống nhiễu 100nF (Tụ cho dòng điện nhiễu đi qua, đi xuống GND)
- **Phần nút Reset:** Chân NRST của MCU được treo lên mức cao (3.3V) thông qua điện trở kéo lên và nối với đất qua một tụ điện. Khi cấp nguồn, tụ điện nạp

dần tạo ra độ trễ (Time constant $\tau = R \times C$), giữ chân NRST ở mức thấp trong một khoảng thời gian ngắn đủ để điện áp nguồn ổn định trước khi MCU bắt đầu chạy chương trình. Điều này ngăn chặn lỗi khởi động do nhiễu nguồn.

Phần nút Boot

- RAK_BOOT0 quyết định chế độ khởi động của MCU. Bình thường nó được kéo xuống Mass qua điện trở R9 (10kΩ) để chạy chương trình chính
- Khi cần nạp firmware mới (bootloader mode), người dùng nhấn giữ SW_BOOT (kéo chân này lên 3.3V) và nhấn Reset.

c. Nguyên lý khối nguồn



Hình 2. 5 Nguyên lý mạch khối nguồn

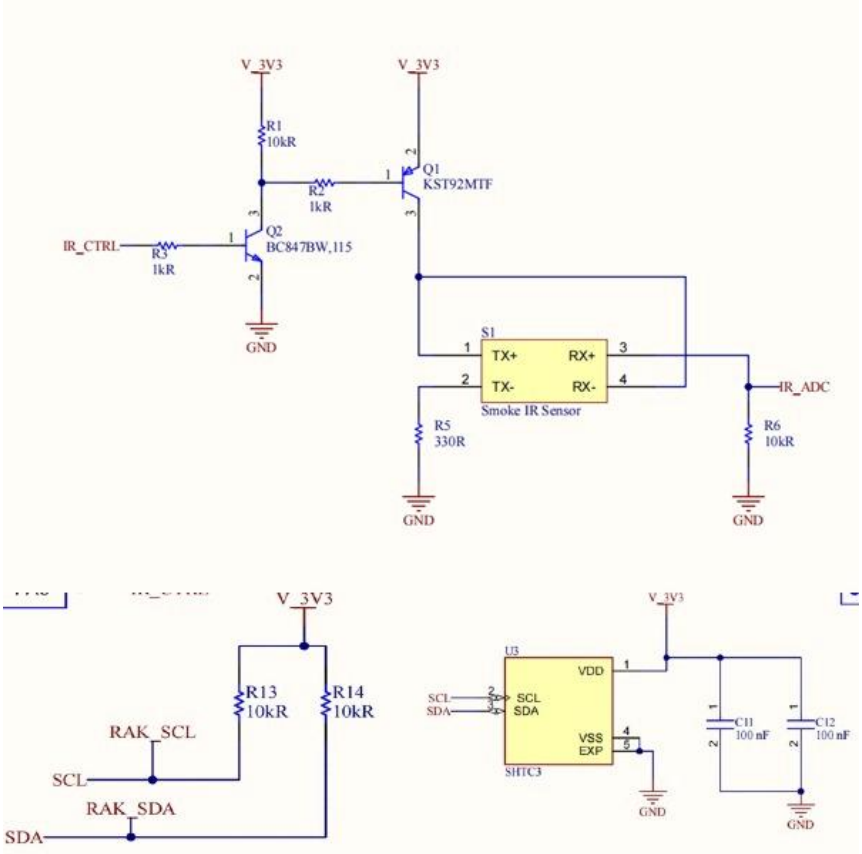
***Tầng đầu vào và bảo vệ**

- J_PWR_IN: Đây là Jack/Connector để cắm nguồn vào (ví dụ: pin Li-Po, pin 9V hoặc Adapter). Chân 1 là dương (+), chân 2 là âm (GND).
- Diode D1 (LL4148): Đóng vai trò bảo vệ chống ngược cực (Reverse Polarity Protection). Trong trường hợp người dùng cắm nhầm cực tính của nguồn, Diode D1 sẽ phân cực ngược và ngăn dòng điện chạy vào mạch, bảo vệ các linh kiện phía sau khỏi hư hỏng.
- Tụ điện C5: Có nhiệm vụ tích trữ năng lượng và xả điện bù vào khi nguồn cấp dao động, giúp điện áp V_IN tại chân vào của IC ổn áp luôn ổn định, tránh hiện tượng sụt áp tức thời gây sập nguồn hệ thống.

***Tầng ổn áp**

- IC RT9013 so sánh điện áp ngõ ra với một điện áp tham chiếu nội, từ đó điều chỉnh dòng điện qua transistor công suất bên trong để giữ V_{IN} luôn bằng 3.3V bất chấp sự thay đổi của V_{OUT} (khi pin sụt áp).
- Tụ lọc C_{IN}, C_{OUT}: Tụ đầu vào (C_{IN}) lọc các gai nhiễu tần số thấp từ pin. Tụ đầu ra (C_{OUT}) đóng vai trò bù dòng tức thời khi tải thay đổi đột ngột, ngăn hiện tượng sụt áp khiến MCU reset.

d. Nguyên lý khối cảm biến



Hình 2. 6 Nguyên lý mạch khối cảm biến

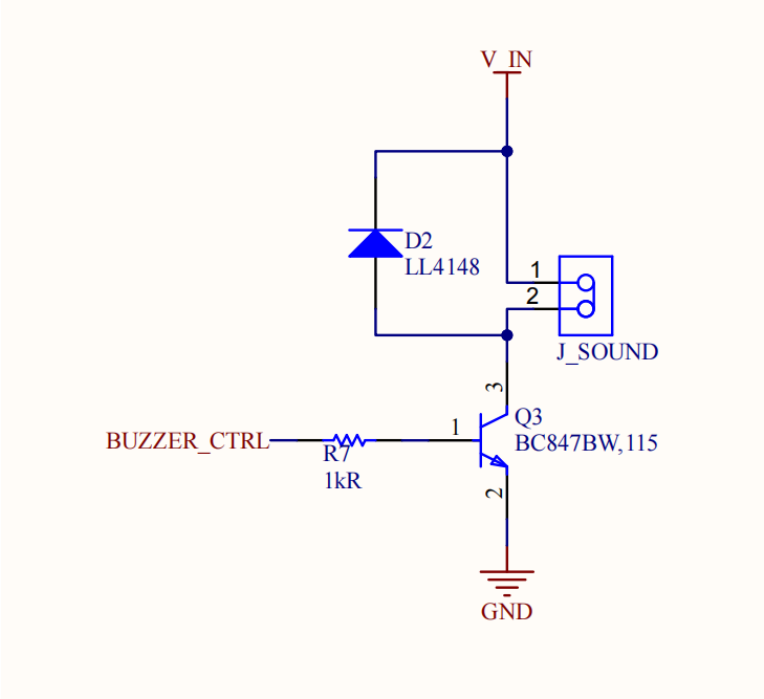
***Phần cảm biến khói**

- **Nguyên lý tán xạ ánh sáng:** Buồng khói được thiết kế là một buồng tối, chứa một LED phát hồng ngoại và một Photodiode thu đặt lệch góc (không nhìn thấy nhau trực tiếp).
- **Hoạt động:**
  - *Trạng thái bình thường:* Ánh sáng từ LED hồng ngoại đi thẳng và bị hấp thụ bởi vách buồng, Photodiode không nhận được ánh sáng → Dòng điện ngõ ra xấp xỉ 0.
  - *Trạng thái có cháy:* Các hạt khói bay vào buồng sẽ làm tán xạ tia hồng ngoại theo nhiều hướng. Một phần ánh sáng bị tán xạ sẽ đi vào Photodiode, sinh ra dòng quang điện (I_{ph}). Dòng điện này đi qua điện trở chuyển đổi thành điện áp ($V= I_{ph} \times R$) để ADC của MCU đọc được.
  - MCU không bật LED hồng ngoại liên tục mà chỉ phát xung ngắn chu kỳ 10ms mỗi lần đo để tiết kiệm pin.

***Phần cảm biến nhiệt:**

- Sử dụng giao thức I2C (Phụ lục 16). Hai đường dây SDA (Dữ liệu) và SCL (Xung nhịp) được kéo lên nguồn (Pull-up) bằng điện trở 10kΩ.
- MCU đóng vai trò Master, gửi địa chỉ của SHTC3 (Slave) lên bus. Khi khớp địa chỉ, SHTC3 thực hiện chuyển đổi ADC nội bộ từ cảm biến điện dung (cho độ ẩm) và bán dẫn (cho nhiệt độ), sau đó trả về dữ liệu số cho MCU.

e. Nguyên lý khối cảnh báo



Hình 2. 7 Nguyên lý mạch khối cảnh báo

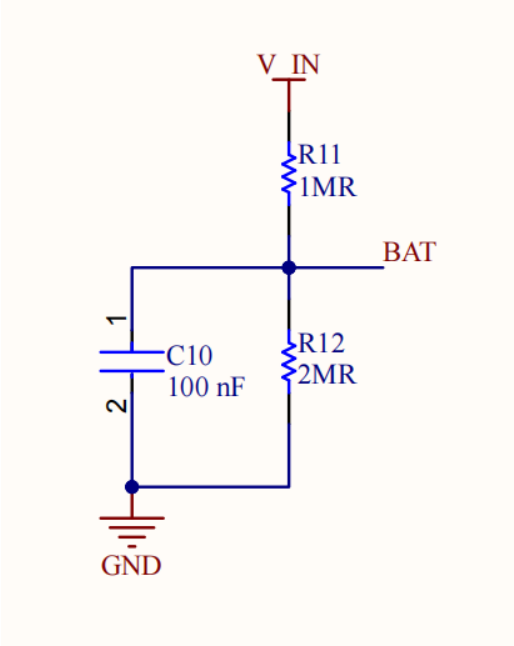
Chân PA1 có nhãn BUZZER_CTRL. Khi phát hiện khói, MCU xuất mức Logic cao (High) ra chân này. Tín hiệu này sẽ kích dẫn một Transistor để cấp dòng lớn cho còi kêu, vì chân MCU không đủ dòng để kéo trực tiếp còi.

Mạch điều khiển (Driver Circuit): Sử dụng một Transistor BJT (NPN) hoặc MOSFET kênh N hoạt động ở chế độ khóa (Switching).

Đóng ngắt: Khi GPIO xuất mức cao, Transistor dẫn bão hòa ($V_{CE} \approx 0V$), dòng điện chạy từ nguồn qua còi xuống đất→ Còi kêu. Khi GPIO mức thấp, Transistor ngắt→Còi tắt.

Diode xả (Flyback Diode): Một diode mắc ngược cực song song với còi (nếu dùng còi từ tính) để triệt tiêu dòng điện cảm ứng sinh ra khi ngắt đột ngột, bảo vệ Transistor khỏi bị đánh thủng.

f. Nguyên lý khối đo dung lượng pin



Hình 2. 8 Nguyên lý mạch khối đo dung lượng pin

Thiết kế Vpin (điện áp từ nguồn pin) đi qua mạch cầu phân áp ($R11$ và $R12$) để giảm điện áp xuống nằm trong khoảng điện áp mà vi điều khiển có thể đọc được: từ 0V đến +3.3V.

Áp dụng định luật phân áp, điện áp VBAT thu được tại điểm giữa sẽ tỉ lệ thuận với điện trở R12 phía dưới.

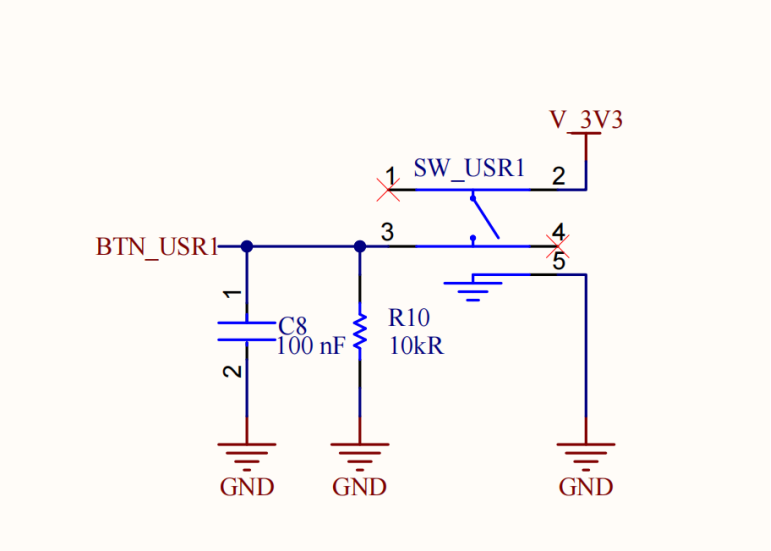
$$V_{BAT}=\frac{R12}{R12+R11}\times V_{IN}$$

Chọn giá trị R11 = 1M và R12 = 2M vào công thức trên, ta được:

$$V_{BAT}=\frac{2}{2+1}\times V_{IN}$$
$$\rightarrow V_{BAT}=0.667\times V_{IN}$$
$$\rightarrow V_{IN}\leq 4.95V$$

Kết luận về tỉ lệ: Điện áp đọc được tại chân ADC sẽ bằng **2/3** điện áp thực tế của pin. Do R11 và R12 có trở kháng rất lớn để tiết kiệm năng lượng (giảm dòng rò) nên ta phải có tụ C10 giúp giảm nhiễu khi đọc giá trị.

g. Nguyên lý khối nút bấm thao thác

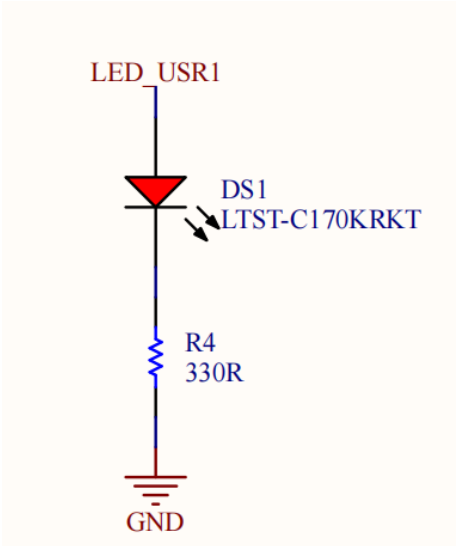


Hình 2. 9 Nguyên lý mạch khối nút bấm thao tác

Thiết kế mạch nút nhấn sử dụng logic kích hoạt mức cao (Active High), tương tự như cấu trúc của nút BOOT:

- **Trạng thái nghỉ:** Chân GPIO được ghim xuống mức thấp (GND) thông qua điện trở kéo xuống 10kΩ.
- **Trạng thái kích hoạt:** Khi nhấn nút, dòng điện từ nguồn +3.3V được dẫn trực tiếp vào chân GPIO, chuyển trạng thái logic từ 0 lên 1 để kích hoạt ngắt đánh thức vi điều khiển.
- **Khử rung:** Tụ điện 100nF mắc song song đóng vai trò lọc nhiễu và triệt tiêu hiện tượng rung phím cơ học, đảm bảo tín hiệu đầu vào ổn định.

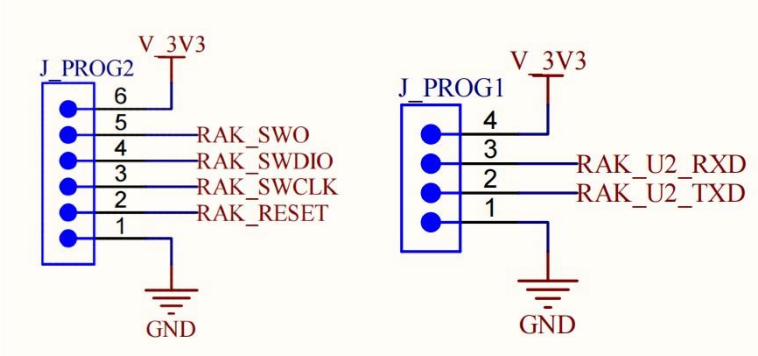
h. Nguyên lý khối led cảnh báo



Hình 2. 10 Nguyên lý mạch khối LED cảnh báo

Đèn báo trạng thái (LED) Chân PB4 nối với LED_USR1 nhận tín hiệu theo chu kỳ từ MCU để báo hiệu trạng thái hoạt động

i. Nguyên lý khối nạp và gỡ lỗi:



Hình 2. 11 Nguyên lý mạch khối nạp và gỡ lỗi

Hệ thống sử dụng chuẩn giao tiếp SWD (Serial Wire Debug), là chuẩn rút gọn của JTAG dành cho các dòng chip ARM Cortex để tiết kiệm chân.

- **SWDIO (Serial Wire Data I/O):** Đường truyền dữ liệu hai chiều, dùng để nạp code vào bộ nhớ Flash và đọc giá trị thanh ghi/biến số khi Debug.
- **SWCLK (Serial Wire Clock):** Đường xung nhịp đồng bộ do mạch nạp (ST-Link/J-Link) cấp cho MCU.
- **Nguyên lý:** Mạch nạp nắm quyền điều khiển lõi CPU, có thể tạm dừng (Halt), chạy từng bước (Step) hoặc đặt điểm dừng (Breakpoint) để kiểm tra logic chương trình mà không ảnh hưởng đến phần cứng đang chạy.

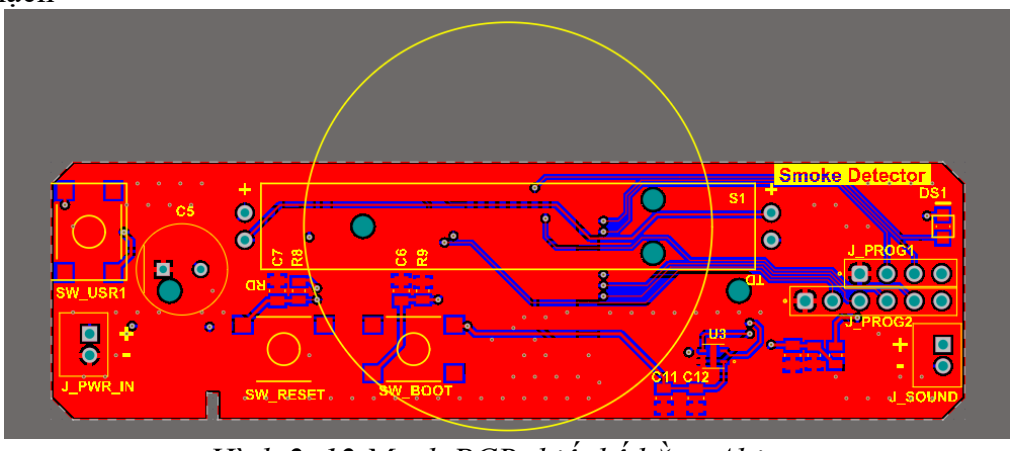
2.2.3 Thiết kế bảng mạch in (PCB)

Sau khi đã thiết kế xong nguyên lý mạch in của toàn mạch, tiến hành chuyển từ nguyên lý thành dạng 2D, 3D của linh kiện, phục vụ thiết kế mạch in PCB.

a. Mạch PCB thiết kế ở chế độ 2D

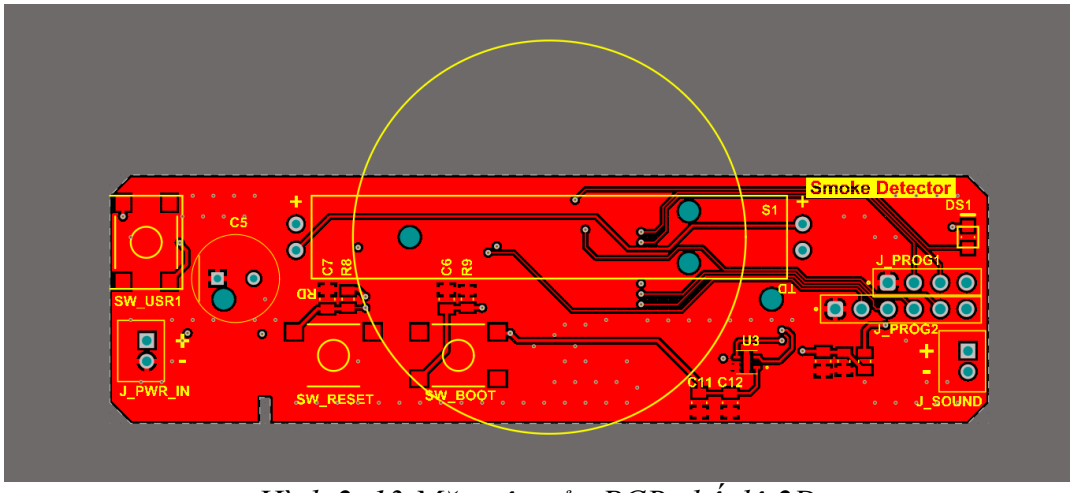
Kích thước mạch (dài x rộng x cao):8.5cm x 2.39cm x 1.8mm

Toàn mạch



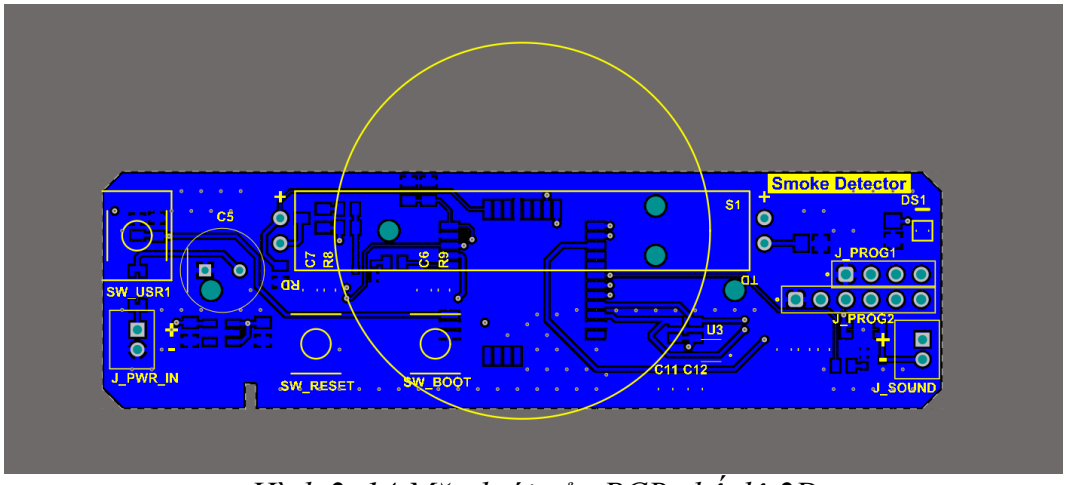
Hình 2. 12 Mạch PCB thiết kế bằng Altium

-Mặt trên:



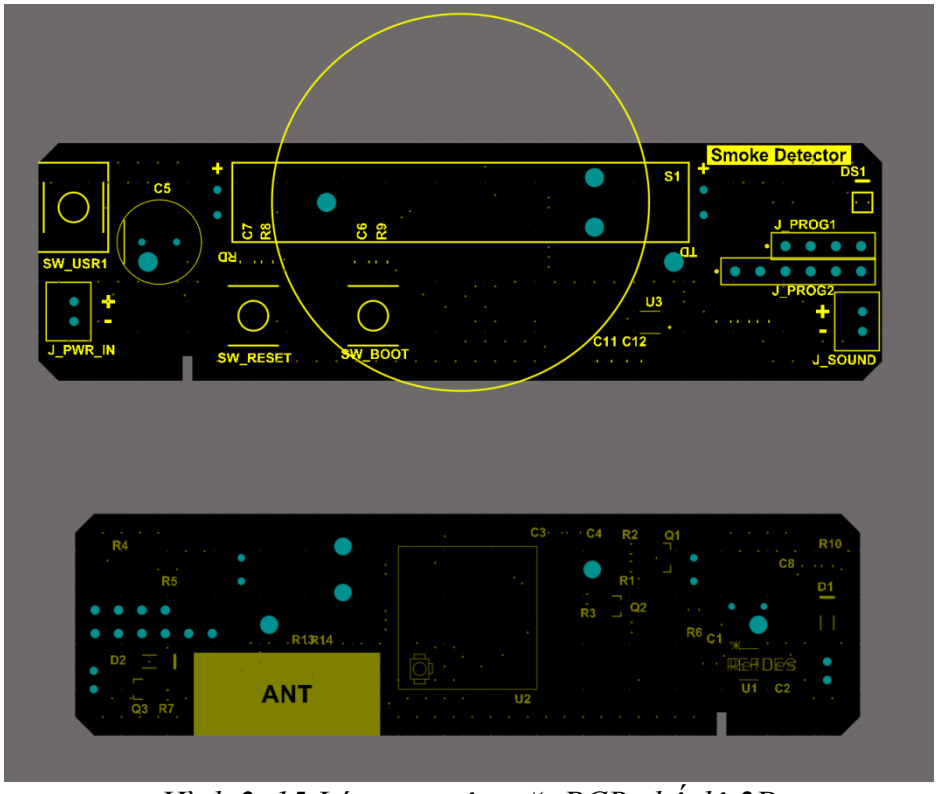
Hình 2. 13 Mặt trên của PCB chế độ 2D

-Mặt dưới:



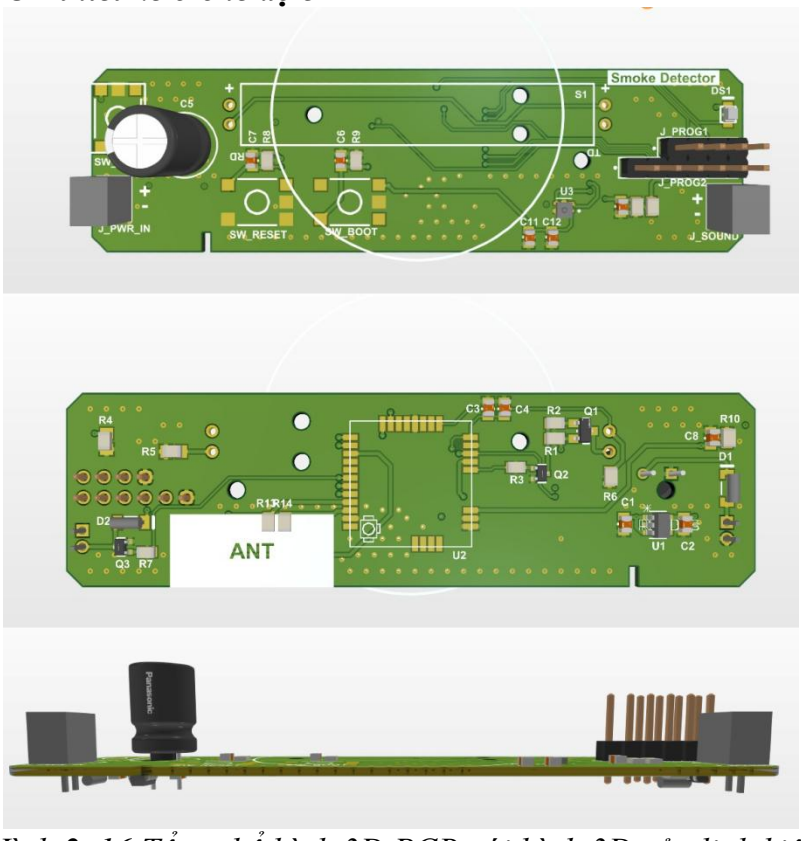
Hình 2. 14 Mặt dưới của PCB chế độ 2D

-Lớp sơn chỉ dẫn của mặt trên và mặt dưới:



Hình 2. 15 Lớp sơn trên mặt PCB chế độ 2D

b. Mạch PCB thiết kế ở chế độ 3D



Hình 2. 16 Tổng thể hình 3D PCB với hình 3D của linh kiện

c. Mạch PCB trong thực tế

2.3.4 Hàn mạch và hoàn thiện phần cứng

Kích thước mạch cơ bản sau khi hàn các linh kiện, mô-đun:
Mạch sau khi đã hàn các linh kiện, mô-đun:



Hình 2. 17 Mạch sau khi đã hàn các linh kiện, mô-đun

Mạch sau khi lắp vào hộp và hoàn thiện:



Hình 2. 18 Mạch sau khi lắp và hoàn thiện

2.3. Chương trình nhúng Firmware của Device

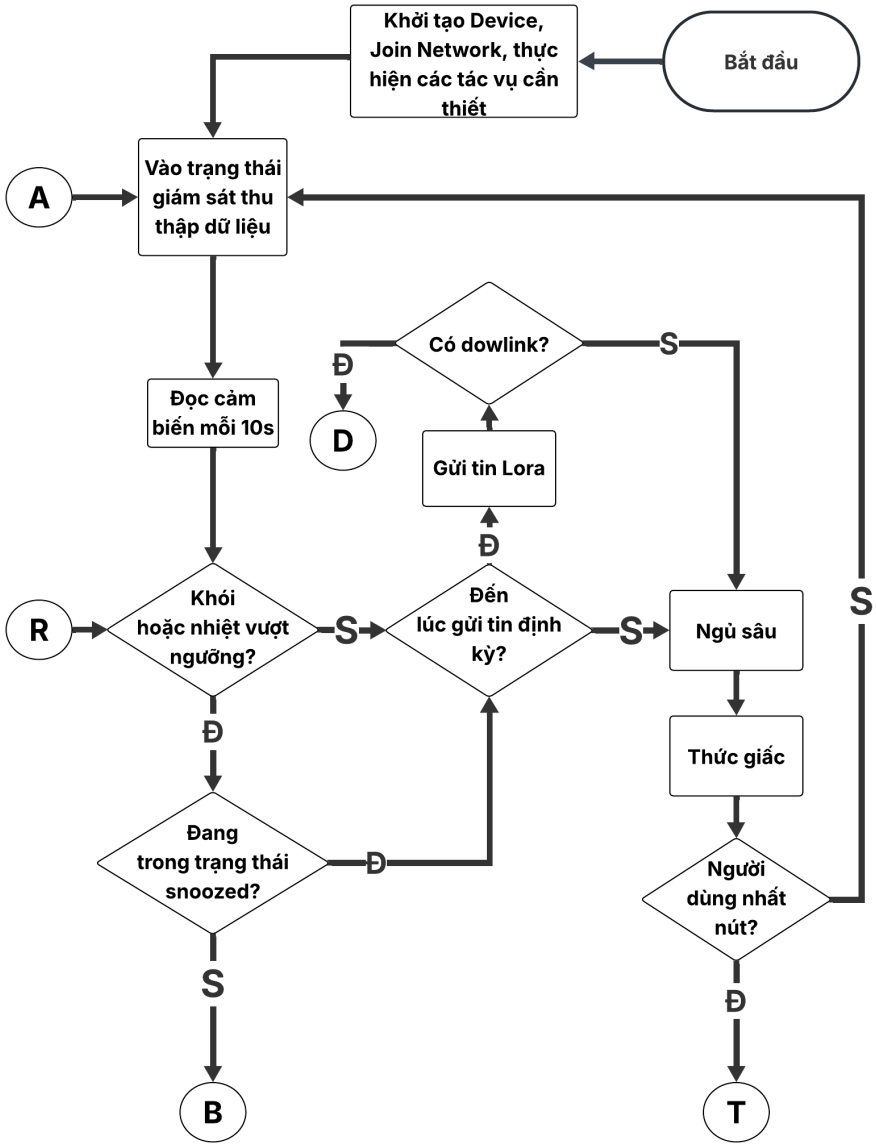
Các chương trình được thiết kế bằng ngôn ngữ lập trình C/C++ cùng với các thư viện hỗ trợ. Chương trình sữ được nạp vào vi điều khiển RAK3172 thông qua giao tiếp UART để thực hiện các tính toán logic của các dịch vụ trên Device

2.3.1. Thiết các kế luồng hoạt động

Để đảm bảo thiết bị hoạt động ổn định, tiết kiệm năng lượng và đặc biệt là giảm thiểu tối đa tỷ lệ cảnh báo giả, chương trình điều khiển cho vi xử lý RAK3172 được thiết kế dựa trên mô hình Máy trạng thái hữu hạn. Hệ thống hoạt động luân phiên giữa ba trạng thái chính: Giám sát (Monitoring), Xác thực (Confirming) và Báo động (Alarming).

Hệ thống hoạt động theo 3 trạng thái:

a. Luồng hoạt động Giám sát (MONITORING)



Hình 2. 19 Lưu đồ luồng hoạt động Giám sát

Các điểm khoanh tròn là các mốc hoạt động của luồng. Các luồng tính năng khác sử dụng các mốc này làm đầu vào và đầu ra.

Phần "Khởi tạo Device, Join NetWork (Tham gia mạng), thực hiện các tác vụ cần thiết" bao gồm những phần sau:

-Khởi tạo Device: Khởi tạo các chân tín hiệu INPUT, OUTPUT

-Các tác vụ cần thiết:

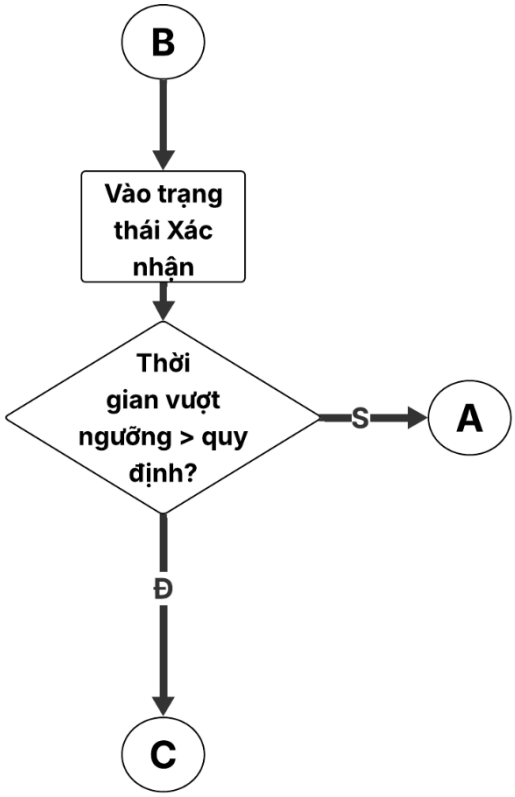
+ Đọc ngưỡng từ bộ nhớ flash, nếu thành công thì gán ngưỡng lần thay đổi gần nhất, ngược lại thì gán ngưỡng cố định trong Firmware.

+Cấu hình, thiết lập các thông số LoraWAN: OTAAKey (AppEui, Appkey), DevEui.

-Join mạng: Nếu Join thành công thì cấu hình các tham số (ADR, Retry CFM) và đăng ký các hàm Callbacks còn nếu ngược lại thì sẽ phải Join lại.

- Đây là trạng thái hoạt động mặc định nhằm tối ưu hóa năng lượng. Thiết bị thực hiện chu kỳ "Ngủ (Sleep) - Thức (Wake-up)" định kỳ. Tại mỗi chu kỳ thức, vi xử lý thu thập dữ liệu từ cảm biến nhiệt độ/độ ẩm SHTC3 và cảm biến khói quang.
- Nếu các giá trị đo nằm trong ngưỡng an toàn: Thiết bị gửi gói tin định kỳ (theo chu kỳ dài, ví dụ: 15 phút/lần) để báo cáo trạng thái hoạt động về Gateway và quay lại ngủ.
- Nếu phát hiện giá trị vượt ngưỡng (khói hoặc nhiệt> ngưỡng): Hệ thống sẽ chuyển nhay sang trạng thái CONFIRMING.

b. Luồng hoạt động Xác nhận (COMFIRMING):

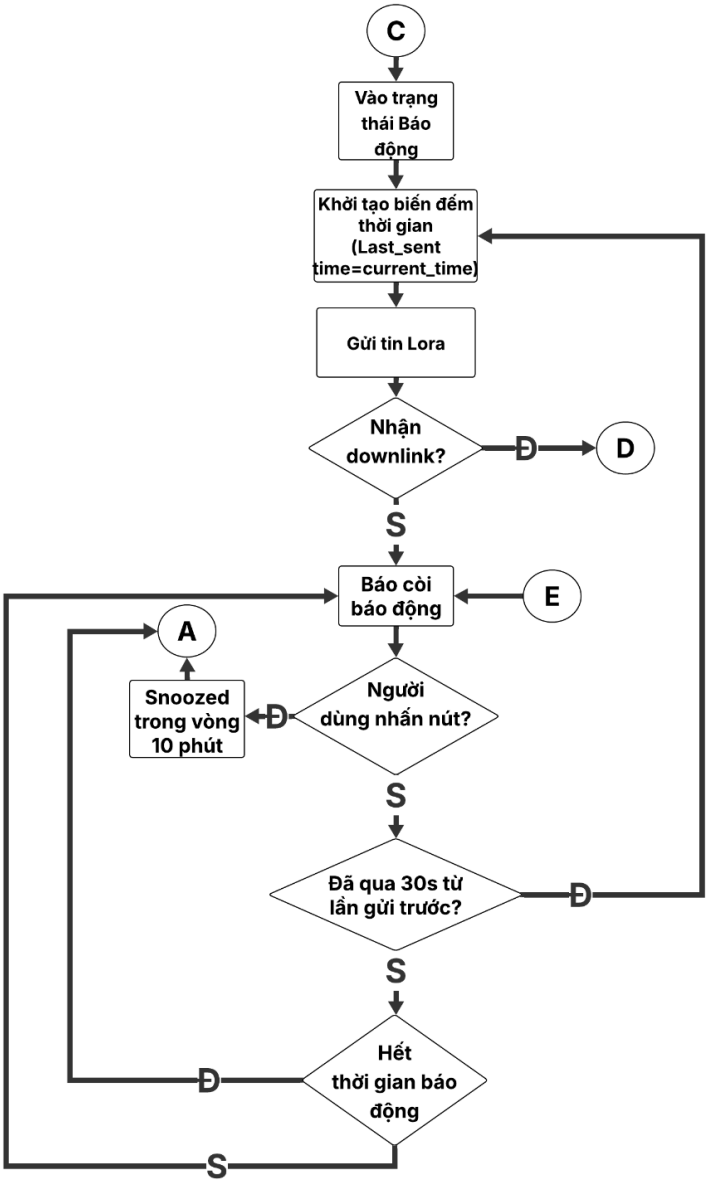


Hình 2. 20 Lưu đồ luồng hoạt động Xác nhận

Đây là bước đệm quan trọng nhằm loại bỏ nhiễu tín hiệu tức thời (do bụi bẩn hoặc dao động điện áp).

- Tại trạng thái này, tần suất lấy mẫu của cảm biến được tăng lên.
- Nếu dữ liệu cảm biến liên tục vượt ngưỡng trong một thời gian xác định (Confirm_time), sự cố được xác nhận là thực tế. Hệ thống chuyển sang trạng thái ALARMING
- Ngược lại, nếu tín hiệu giảm xuống dưới ngưỡng trong quá trình xác thực, hệ thống coi đây là nhiễu và quay về trạng thái Monitoring.

c. Luồng hoạt động Báo động (ALARMING):



Hình 2. 21 Lưu đồ luồng hoạt động Báo động

- Thiết bị kích hoạt còi báo động (Buzzer) và đèn LED cảnh báo tại chỗ.

- Đồng thời, vi xử lý đóng gói và gửi bản tin cảnh báo khẩn cấp (Uplink Alarm Packet) qua mạng LoRaWAN tới Server ngay lập tức mà không chờ chu kỳ gửi tin định kỳ.

- Sau khi gửi cảnh báo, thiết bị duy trì trạng thái báo động trong khoảng thời gian ALARM_PERIOD trước khi kiểm tra lại môi trường để quyết định có quay về trạng thái giám sát hay không.

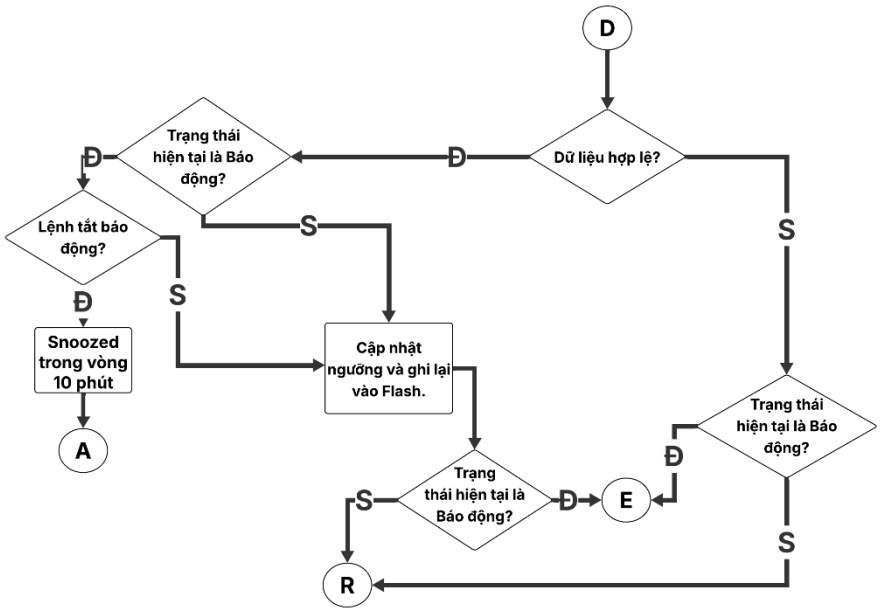
- Cơ chế truyền lại: Để khắc phục rủi ro mất gói tin (packet loss) trong môi trường vô tuyến, hệ thống thực hiện gửi lặp lại bản tin cảnh báo với chu kỳ ngắn (VD: mỗi 60 giây/lần) trong suốt thời gian báo động. Điều này đảm bảo tín hiệu khẩn cấp có xác suất cao nhất đến được Gateway và Server xử lý.

```
11:44:57.933 -> !!! FIRE DETECTED - ALARM ACTIVE !!!
11:44:58.069 -> Sent Packet: Type 3, Smoke 100%, Level 3, Bat 0%, adc2838
11:44:58.246 -> >>> Resending Alarm Packet...
11:44:59.749 -> +EVT:SEND_CONFIRMED_OK
```

Hình 2. 22 Serial log khi có cháy

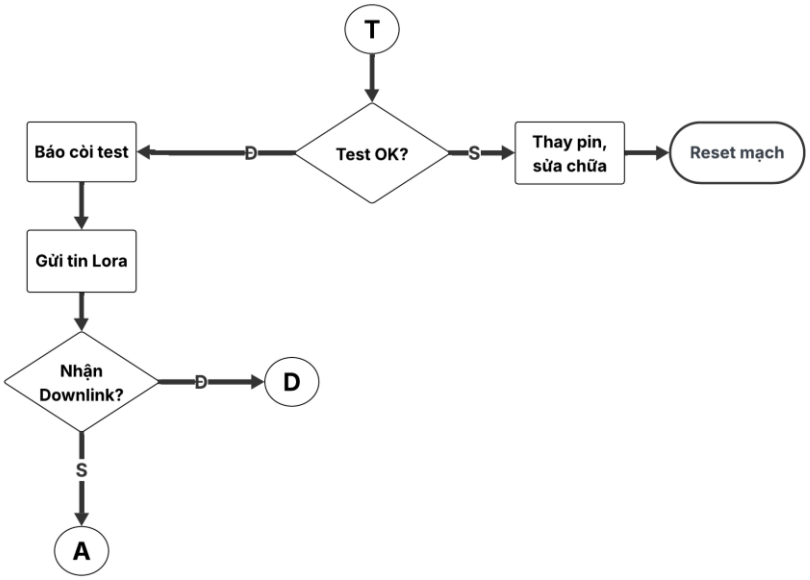
d. Các luồng hoạt động khác

- ***Luồng hoạt động Downlink***



Hình 2. 23 Lưu đồ luồng hoạt động Downlink

- ***Luồng hoạt động test báo cháy***



Hình 2. 24 Lưu đồ luồng hoạt động Test

2.3.2. Logic phát hiện sự kiện

Mục này trình bày các công thức toán học định nghĩa tín hiệu đầu vào cho máy trạng thái. Hệ thống sử dụng cơ chế kích hoạt dựa trên logic HOẶC (OR Logic) để đảm bảo phát hiện sớm mọi nguy cơ cháy, dù là cháy âm ỉ (nhiều khói) hay cháy bùng phát (nhiệt cao).

Phương trình kích hoạt tổng quát (Event Trigger Equation):

Biến trạng thái Trigger dùng để chuyển đổi hệ thống sang chế độ "Xác thực" (Confirming) được xác định như phương trình (2.1):

$$\textbf{Trigger} = \text{Cond}_{\text{Smoke}} \cup \text{Cond}_{\text{Heat}} = \text{TRUE} \quad (2.1)$$

Trong đó:

- $\text{Cond}_{\text{Smoke}}$: Điều kiện phát hiện khói (tính toán dựa trên thuật toán bù trừ nhiễu).
- $\text{Cond}_{\text{Heat}}$: Điều kiện phát hiện quá nhiệt.

Dưới đây là chi tiết cách xác định hai biến điều kiện này:

a. Xác định điều kiện phát hiện khói

Để loại bỏ sai số do dòng rò linh kiện hoặc ánh sáng môi trường, hệ thống sử dụng phương pháp Bù trừ nhiễu nền thay vì đọc trực tiếp ADC.

Công thức xử lý: Giá trị nồng độ khói thực tế ($\text{Val}_{\text{smoke}}$) được tính toán bằng hiệu số giữa giá trị đọc tức thời và giá trị nền chuẩn, được tính theo công thức (2.2):

$$\text{Val}_{\text{smoke}} = \text{ADC}_{\text{read}} - \text{Zero_Offset} \quad (2.2)$$

Trong đó:

- ADC_{read}: Giá trị điện áp thô đọc được từ LED thu (Photodiode) tại thời điểm lấy mẫu
- Zero_Offset: Giá trị điện áp nhiễu nền khi buồng khói sạch (được xác định tại thời điểm khởi động hệ thống).

Điều kiện ngưỡng khói ($Cond_{Smoke}$)được mô tả trong công thức (2.3):

$$Cond_{Smoke}=\begin{cases} TRUE\ nếu\ \text{Valsmoke} \geq \text{ALARM_ON_THRESHOLD} \\ FALSE\ nếu\ \text{Valsmoke} \leq \text{ALARM_ON_THRESHOLD} \end{cases} \quad (2.3)$$

(Trong đó ALARM_ON_THRESHOLD là ngưỡng kích hoạt báo khói)

b. Xác định điều kiện phát hiện nhiệt

Đối với cảm biến nhiệt độ SHTC3, hệ thống sử dụng ngưỡng cố định theo tiêu chuẩn phòng cháy chữa cháy. Điều kiện ngưỡng nhiệt $Cond_{Heat}$ được xác định theo công thức (2.4):

$$Cond_{Heat}=\begin{cases} TRUE\ nếu\ Tenv \geq 57°C \\ FALSE\ nếu\ Tenv \leq 57°C \end{cases} \quad (2.4)$$

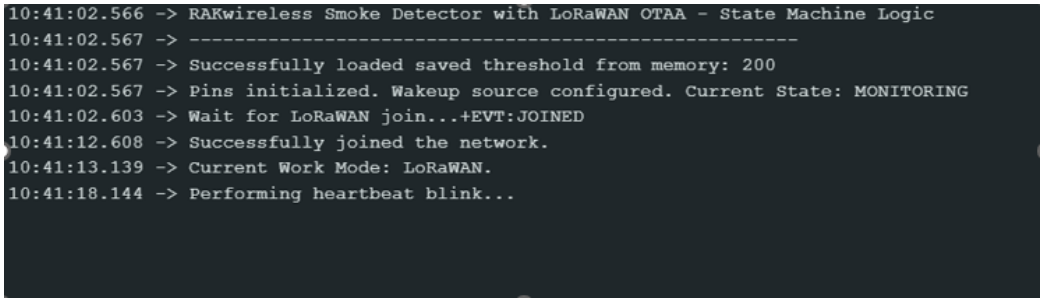
2.3.3. Triển khai Stack LoRaWAN và Cơ chế kích hoạt OTAA

Để đảm bảo khả năng tương thích với mạng lõi và tuân thủ các quy định về tần số vô tuyến tại Việt Nam, Firmware được xây dựng dựa trên ngăn xếp giao thức (Protocol Stack) LoRaWAN chuẩn, phiên bản 1.0.3.

a. Cấu hình tham số mạng

Trước khi khởi tạo kết nối, thiết bị được nạp các tham số cấu hình tĩnh (được lưu trong bộ nhớ Flash/EEPROM) để phù hợp với quy hoạch mạng của đồ án:

- **Vùng tần số (LoRaWAN Region): AS923-2** (Asian 923 MHz).
  - Lý do chọn: Đây là dải tần được Cục Tần số Vô tuyến điện cấp phép cho các thiết bị công suất thấp (LPWAN) tại Việt Nam.
  - Tần số Uplink chính: 923.2 MHz và 923.4 MHz.
- **Lớp thiết bị (Device Class): Class A**.
  - Đặc điểm: Thiết bị ở trạng thái ngủ hầu hết thời gian và chỉ mở cửa sổ thu (Receive Window) ngắn ngay sau khi gửi dữ liệu đi. Đây là chế độ tiết kiệm năng lượng nhất, phù hợp cho thiết bị báo cháy chạy pin.



Hình 2. 25 Serial log sau khi thiết bị kết nối thành công NetworkServer

b. *Thực thi quy trình Join mạng*

Khác với lý thuyết, việc triển khai OTAA trong firmware tập trung vào cơ chế quản lý trạng thái và xử lý lỗi kết nối để đảm bảo thiết bị không bị treo hoặc cạn kiệt pin khi Gateway gặp sự cố.

1. *Nạp khóa bảo mật:* Các khóa định danh không được nạp cứng (hard-coded) rải rác trong code mà được định nghĩa tập trung tại file header cấu hình hoặc nạp qua giao diện UART để lưu vào vùng nhớ an toàn:

    - uint8_t DEVEUI[8]: Định danh phần cứng duy nhất.
    - uint8_t APPEUI[8]: Định danh ứng dụng.
    - uint8_t APPKEY[16]: Khóa gốc dùng để mã hóa quá trình Join.

```
#define OTAA_DEVEUI   {0xEA, 0x58, 0xB1, 0xE8, 0x7D, 0xEA, 0x83, 0x00}
#define OTAA_APPEUI   {0xEA, 0x58, 0xB1, 0xE8, 0x7D, 0xEA, 0x83, 0x00}
#define OTAA_APPKEY   {0x1F, 0xA5, 0x75, 0xC8, 0xA9, 0x71, 0x8B, 0x96, 0x97, 0x0A, 0xE5, 0xAC, 0xBB, 0x01, 0x1D, 0x11}
```

Hình 2. 26 Các khóa bảo mật cần thiết

```
if (api.lorawan.njs.get() == 0) {
        unsigned long retry_interval;
        if (join_retry_count < 5) {
            retry_interval = 30000; // 5 lần đầu
        } else if (join_retry_count < 20) {
            retry_interval = 300000;
        } else {
            retry_interval = 3600000;
        }

        // Kiểm tra đến giờ thử lại
        if (millis() - last_join_try > retry_interval) {
            last_join_try = millis();
            join_retry_count++;

            Serial.printf(">>> Background Join Attempt #%d (Interval: %lu ms)\r\n", join_retry_count, retry_interval);
            api.lorawan.join();

        }
    } else {
        // Nếu đã có mạng, reset biến đếm
        join_retry_count = 0;
    }
    // 2. LOGIC BÁO CHÁY KÉP (SMOKE + HEAT)
```

Hình 2. 27 Cơ chế thử kết nối lại

2. *Xử lý sự kiện kết nối (Event Callback:* Firmware sử dụng cơ chế hướng sự kiện (Event-driven) từ thư viện RAK API để quản lý trạng thái kết nối

    - **Kích hoạt:** Khi khởi động, hàm api.lorawan.join() được gọi để bắt đầu quá trình Join.
    - **Sự kiện Thành công (LORAWAN_EVENT_JOIN_SUCCESS):** Hệ thống cập nhật trạng thái sang JOINED và bắt đầu chu trình hoạt động bình thường.

- **Sự kiện Thất bại (LORAWAN_EVENT_JOIN_FAILED):** Nếu Gateway không phản hồi, thiết bị sẽ tự động thực hiện lại quá trình Join sau mỗi **10 giây**. Cơ chế này đảm bảo thiết bị luôn nỗ lực tìm kiếm mạng ngay khi Gateway hoạt động trở lại.

c. Cơ chế hoạt động độc lập khi mất kết nối

Cơ chế hoạt động độc lập khi mất kết nối. Một yêu cầu quan trọng của hệ thống báo cháy là tính sẵn sàng cao ngay cả khi hạ tầng mạng gặp sự cố.

- Trường hợp mất mạng LoRaWAN:
  - Nếu thiết bị đã Join thành công nhưng đột ngột mất kết nối với Gateway (do Gateway mất điện hoặc tín hiệu yếu):
  - Firmware vẫn duy trì việc đọc cảm biến và xử lý logic phát hiện cháy bình thường.
  - Nếu phát hiện cháy: Hệ thống VẪN kích hoạt cảnh báo tại chỗ (Bật còi Buzzer và đèn LED) để cảnh báo người dân ngay lập tức.
  - Việc gửi gói tin Uplink sẽ thất bại (không nhận được ACK) nhưng không làm treo hệ thống xử lý chính. Điều này đảm bảo tính năng an toàn cốt lõi luôn được ưu tiên hàng đầu.

d. Chu trình Truyền - Nhận dữ liệu (RX/TX Cycle)

- Tính năng Uplink: Thiết bị thức dậy, gửi gói tin cảnh báo hoặc gói tin định kỳ lên Gateway.
- Tính năng Downlink: Thiết bị được lập trình để mở cửa sổ thu (Receive Window) sau mỗi lần gửi Uplink, cho phép nhận lệnh điều khiển từ xa từ ứng dụng di động (ví dụ: lệnh tắt còi báo động giả, lệnh Reset thiết bị hoặc cấu hình lại ngưỡng báo động).

```
10:46:16.126 -> Sent Packet: Type 1, Smoke 0%, Level 1, Bat 88%, adc0
10:46:18.417 -> Successfully sent packet
10:46:18.417 -> +EVT:TX_DONE
```

Hình 2. 28 Serial log gửi tin thành công

```
1:25:26.767 -> Sent Packet: Type 1, Smoke 0%, Level 1, Bat 85%, adc0
1:25:27.899 -> Successfully sent packet
1:25:27.899 -> +EVT:TX_DONE
1:25:27.899 -> +EVT:RX_1:-110:-1:UNICAST:10:00c8
1:25:27.899 -> Downlink received!
1:25:27.899 -> SUCCESS: New alarm threshold set to 200 and saved to memory.
```

Hình 2. 29 Serial log khi nhận downlink

2.4 Kết luận chương

Tổng kết lại, Chương 2 đã đi sâu vào quá trình hiện thực hóa phân hệ Thiết bị (Device), từ việc xây dựng sơ đồ nguyên lý, tối ưu hóa mạch in (PCB) cho đến hoàn thiện sản phẩm phần cứng thực tế. Song song với đó, các giải pháp phần mềm nhúng (Firmware) cũng đã được xây dựng hoàn chỉnh, bao gồm việc thiết lập các luồng thuật toán xử lý tín hiệu và tích hợp ngăn xếp giao thức LoRaWAN/OTAA. Những kết quả đạt được tại đây đóng vai trò nền tảng cốt lõi để tiếp tục xây dựng ứng dụng giám sát và thực hiện các kịch bản kiểm thử toàn hệ thống trong chương tiếp theo.

.

CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI PHÂN HỆ MOBILE APPLICATION

3.1. Tổng quan và công cụ phát triển

3.1.1. Vai trò của phân hệ ứng dụng

Trong hệ thống IoT cảnh báo cháy, ứng dụng di động (Mobile App) đóng vai trò là giao diện tương tác cuối cùng (End-user Interface). Nó không chỉ hiển thị các thông số kỹ thuật khô khan (nhiệt độ, mức khói) mà còn phải đảm bảo tính năng quan trọng nhất: Đánh thức người dùng khi có sự cố khẩn cấp, bất kể điện thoại đang ở trạng thái nào.

3.1.2. Môi trường và Công nghệ phát triển

- Framework: Flutter (Google) - Cho phép phát triển ứng dụng đa nền tảng (Android/iOS) từ một mã nguồn duy nhất.
- Ngôn ngữ lập trình: Dart - Với ưu điểm hỗ trợ lập trình bất đồng bộ mạnh mẽ, phù hợp cho các tác vụ I/O thời gian thực.
- Môi trường phát triển (IDE): VSCode/AndroidStudio.
- Giao thức truyền thông:
  - MQTT (Message Queuing Telemetry Transport): Sử dụng thư viện mqtt_client để nhận dữ liệu khói và pin theo thời gian thực (Real-time) với độ trễ thấp.
  - HTTP/REST: Sử dụng thư viện http để giao tiếp với ChirpStack API phục vụ việc lấy danh sách thiết bị và gửi lệnh điều khiển (Downlink).
- Cơ chế chạy nền (Background Service): Ứng dụng tích hợp flutter_background_service để duy trì kết nối MQTT liên tục 24/7 ngay cả khi người dùng tắt màn hình, đảm bảo cảnh báo báo cháy luôn được kích hoạt tức thì.
- Lưu trữ cục bộ: Sử dụng SharedPreferences để lưu trữ cấu hình, cache danh sách thiết bị và lịch sử cảnh báo ngay trên điện thoại, giúp ứng dụng khởi động nhanh và hoạt động được khi mất kết nối mạng tạm thời.

3.1.3. Các thư viện chính sử dụng

Các thư viện nòng cốt sử dụng bao gồm:

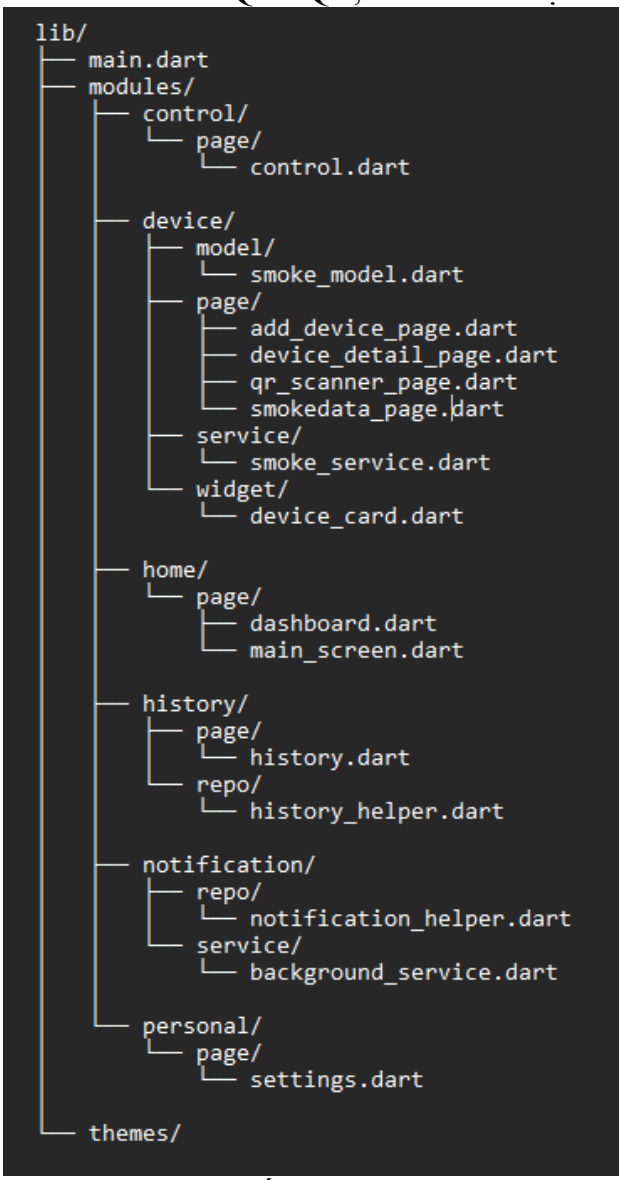
- flutter_local_notifications: Quản lý hiển thị thông báo đẩy khi có sự kiện cháy hoặc khi ứng dụng chạy nền.
- mqtt_client: Giao thức kết nối đến MQTT Broker để nhận dữ liệu thời gian thực từ Gateway với độ trễ thấp.
- flutter_background_service: Cho phép ứng dụng duy trì kết nối MQTT và lắng nghe cảnh báo ngay cả khi người dùng đã tắt màn hình (Background Task).
- mobile_scanner: Hỗ trợ camera quét mã QR Code để định danh và thêm nhanh thiết bị vào hệ thống.
- shared_preferences: Lưu trữ dữ liệu cục bộ (Local Storage) như lịch sử cảnh báo, cấu hình âm thanh và cache danh sách thiết bị.
- http: Thực hiện các giao thức mạng RESTful/gRPC-Web để gửi lệnh điều khiển xuống thiết bị (Downlink).
- audioplayers / flutter_ringtone_player: Quản lý và phát âm thanh cảnh báo, còi hú ưu tiên cao.

3.2. Kiến trúc phần mềm

Ứng dụng được thiết kế theo mô hình phân lớp (Layered Architecture) để đảm bảo

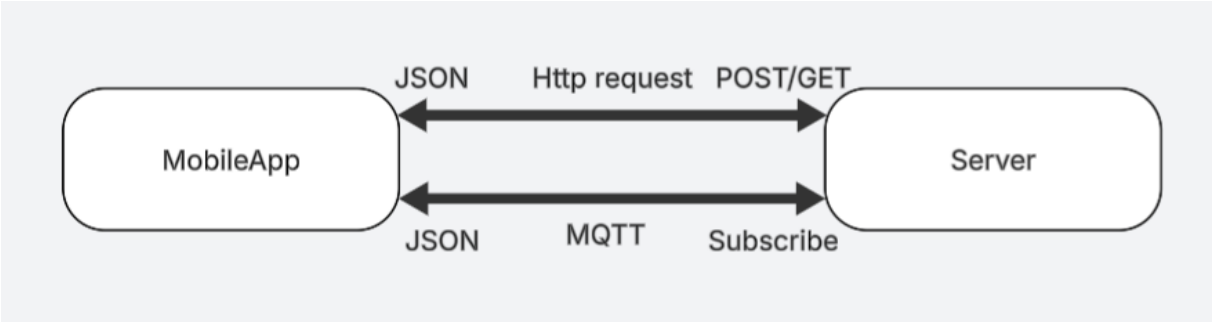
tính dễ bảo trì và mở rộng code. Cấu trúc project được tổ chức như sau:

- Layer Data (Models): Định nghĩa các cấu trúc dữ liệu ánh xạ từ JSON trả về của Server (ví dụ: smoke_model.dart chứa thông tin về nồng độ khói, nhiệt độ, trạng thái pin).
- Layer Service (Business Logic): Chứa các lớp xử lý nghiệp vụ và giao tiếp mạng.
- smoke_service.dart: Đảm nhận việc gọi RESTful API hoặc kết nối WebSocket để lấy dữ liệu thời gian thực.
- background_service.dart: Quản lý các tiến trình chạy nền, đảm bảo nhận cảnh báo ngay cả khi tắt màn hình.
- Layer Presentation (UI): Bao gồm các màn hình (pages/) và các widget tái sử dụng (widgets/).
- tabs/: Chứa các tab chính của giao diện Dashboard, History, Control.
- pages/: Các màn hình chi tiết như Quét QR, Thêm thiết bị.



Hình 3.1 Kiến trúc dự án flutter

3.3. Thiết kế cơ sở và mô hình dữ liệu



Hình 3.2 Sơ đồ tổng quan về giao tiếp giữa App và Server

MobiApp và Server Backend / Chirpstack trao đổi dữ liệu với nhau bằng dữ liệu dạng JSON (Phụ lục 3). Tuy nhiên, Dart là một ngôn ngữ không trực tiếp đọc được dữ liệu kiểu JSON mà cần qua bước giải mã trung gian (Dart đã hỗ trợ những hàm giải mã). Để thuận tiện trong việc đọc dữ liệu đó, MobiApp sẽ mô hình hoá những dữ liệu JSON đó thành một kiểu dữ liệu mới (dạng Class).

Mỗi dữ liệu JSON sẽ được biến thành một Class có tên phù hợp. Dựa trên file models/smoke_model.dart, cấu trúc dữ liệu được thiết kế như sau:

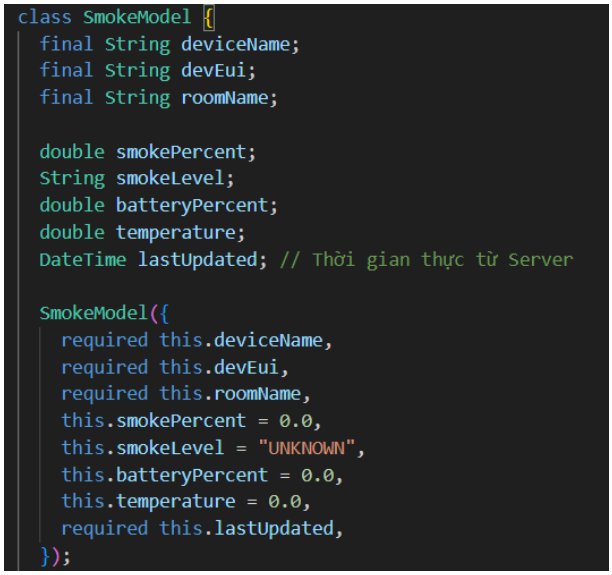
Class SmokeModel: Đối tượng đại diện cho một nút cảm biến báo cháy, bao gồm các thuộc tính:

- **devEui (String):** Định danh duy nhất của thiết bị LoRaWAN.
- **deviceName (String):** Tên gợi nhớ do người dùng đặt (VD: Phòng Khách, Kho hàng).
- **smokePercent (double):** Giá trị nồng độ khói hiện tại (%).
- **smokeLevel (String):** Trạng thái phân loại mức độ nguy hiểm (VD: FIRE, WARNING, SAFE).
- **temperature (double):** Thông số nhiệt độ môi trường.
- **batteryPercent (double):** Phần trăm pin còn lại.
- **lastUpdated (DateTime):** Thời gian cập nhật dữ liệu gần nhất từ Server.

Ví dụ dưới đây là một dữ liệu dạng JSON được Server trả về:


```
data: "AgAbIw=="
▼ object: {} 5 keys
    temperature: 36
    smoke_level: "LOW"
    type: "MANUAL_TEST"
    battery_percent: 79
    smoke_percent: 0
▼ rxInfo: [] 1 item
  ▼ 0: {} 10 keys
      gatewayId: "a84041ffff1ee248"
      uplinkId: 43303
      gwTime: "2025-12-26T11:46:32.060650+00:00"
      nsTime: "2025-12-26T11:46:32.062939771+00:00
      rssi: -106
      snr: 5.5
      channel: 3
      location: {} 0 keys
      context: "ZkDovA=="
      crcStatus: "CRC_OK"
```


Hình 3. 3 Dữ liệu JSON server trả về



```dart
class SmokeModel {
  final String deviceName;
  final String devEui;
  final String roomName;

  double smokePercent;
  String smokeLevel;
  double batteryPercent;
  double temperature;
  DateTime lastUpdated; // Thời gian thực từ Server

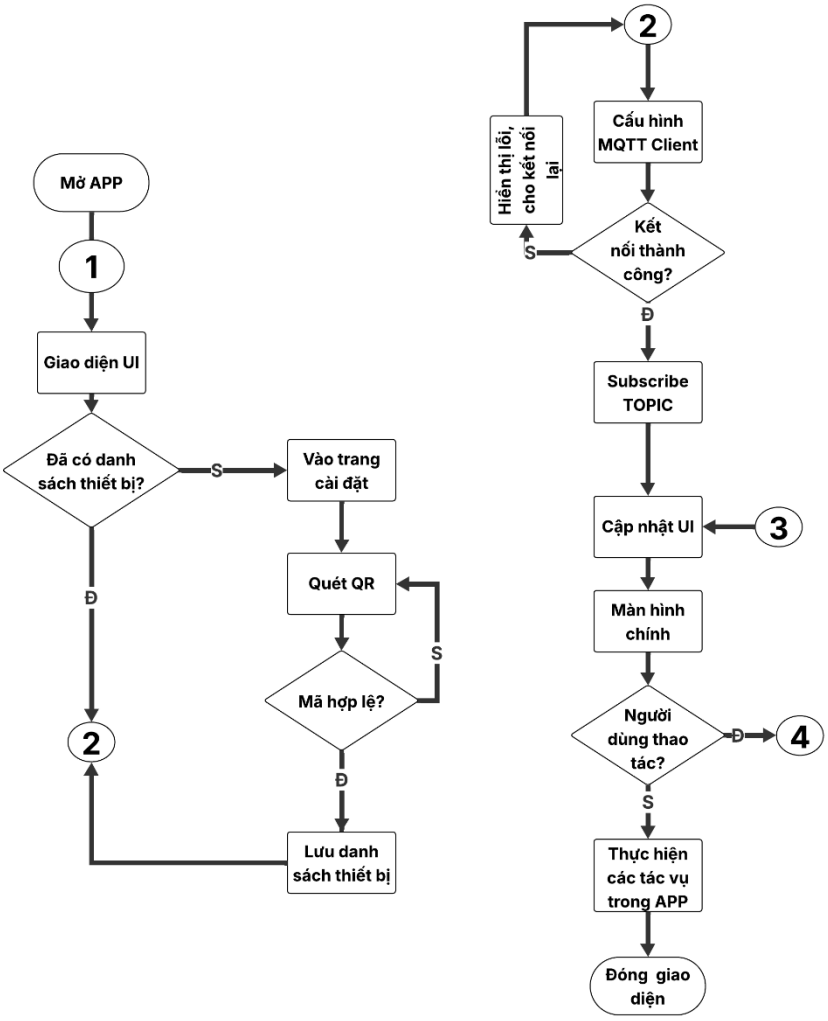
  SmokeModel({
    required this.deviceName,
    required this.devEui,
    required this.roomName,
    this.smokePercent = 0.0,
    this.smokeLevel = "UNKNOWN",
    this.batteryPercent = 0.0,
    this.temperature = 0.0,
    required this.lastUpdated,
  });
```

Hình 3. 4 Mô hình hóa dữ liệu JSON

Dữ liệu lịch sử cảnh báo được lưu trữ cục bộ thông qua HistoryHelper (lib/history_helper.dart) sử dụng SharedPreferences, giúp người dùng truy vết lại các sự kiện nguy hiểm trong quá khứ.

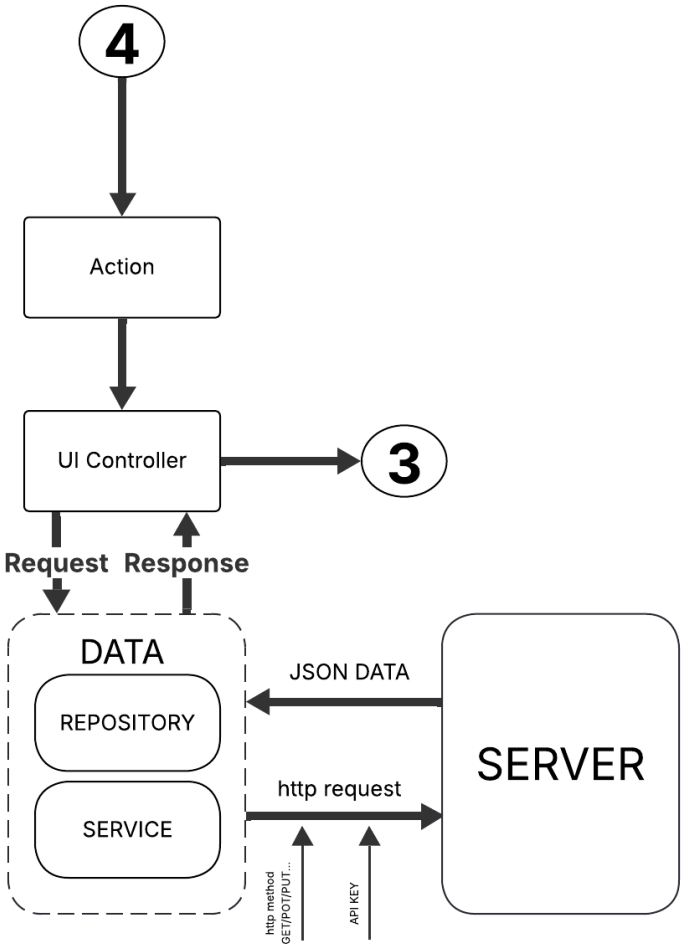
3.4. Thiết kế luồng hoạt động một số dịch vụ

3.4.1. Luồng hoạt động điều hướng màn hình khi mở ứng dụng



Hình 3. 5 Luồng điều hướng màn hình khi mở ứng dụng

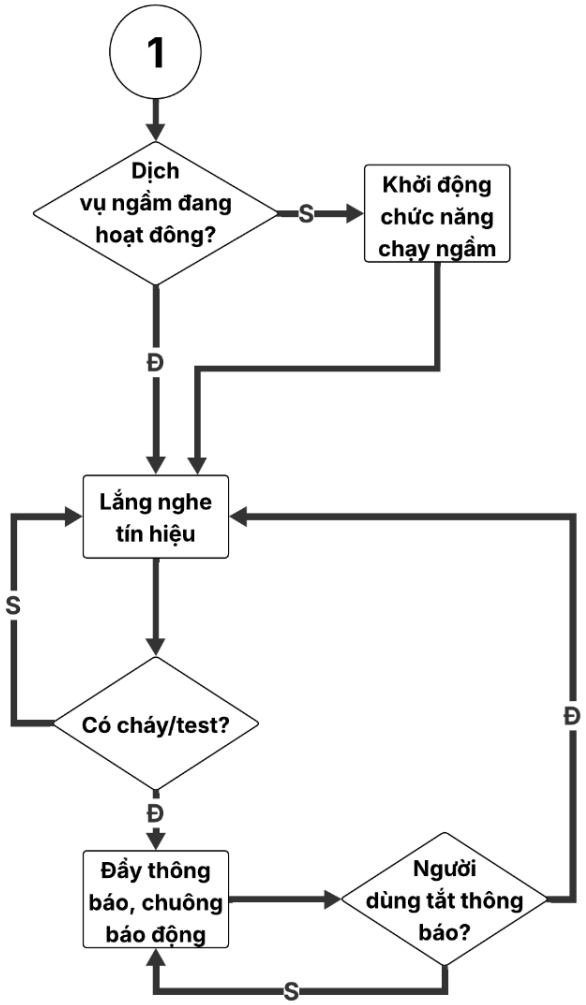
3.4.2. Luồng chung của một RestfulAPI



Hình 3. 6 Luồng chung của một RestfulAPI

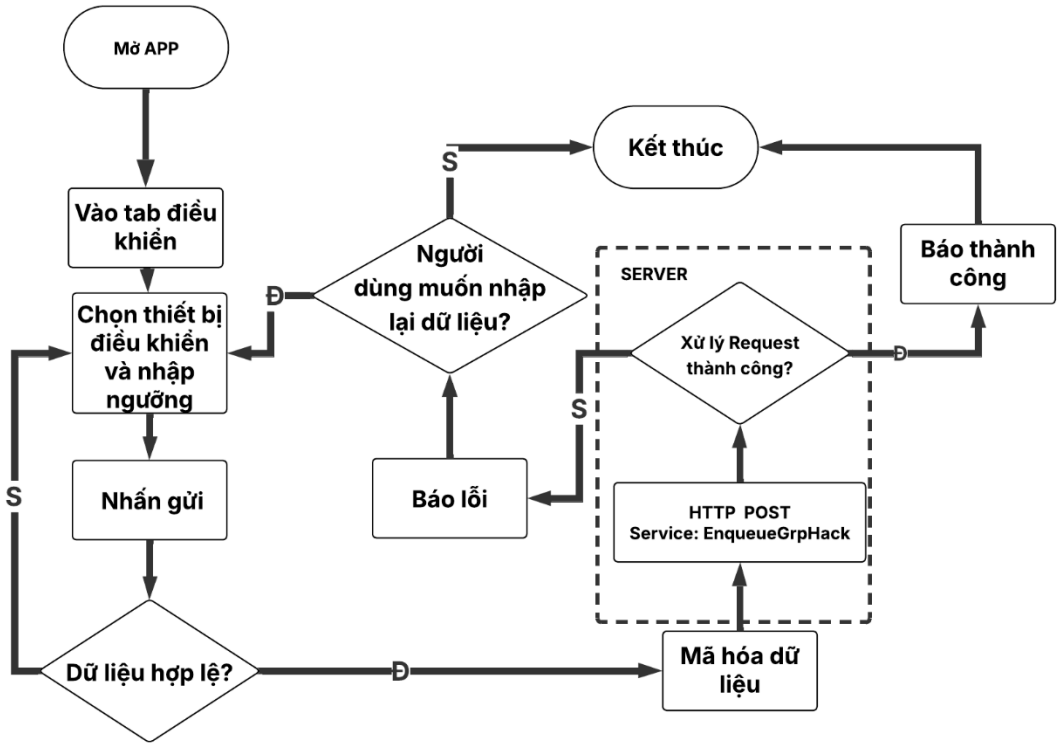
3.4.3. Luồng hoạt động khác

a. Luồng hoạt động ngầm (Background Service)



Hình 3. 7 Luồng hoạt động của Background Service

b. Luồng hoạt động của chế độ điều khiển

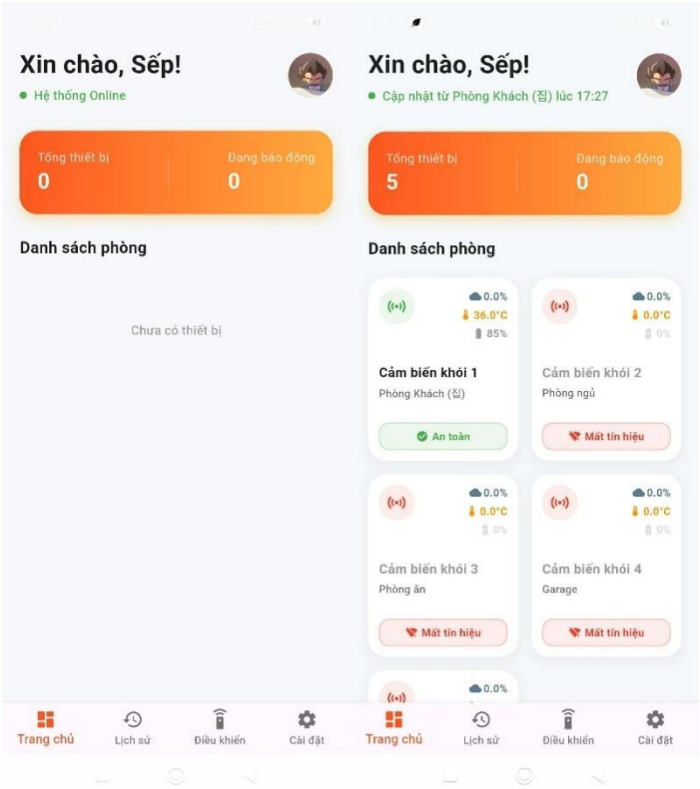


Hình 3. 8 Luồng hoạt động của chế độ điều khiển

3.5. Thiết kế giao diện người dùng (UI/UX)

3.5.1. Màn hình giám sát trung tâm (Dashboard):

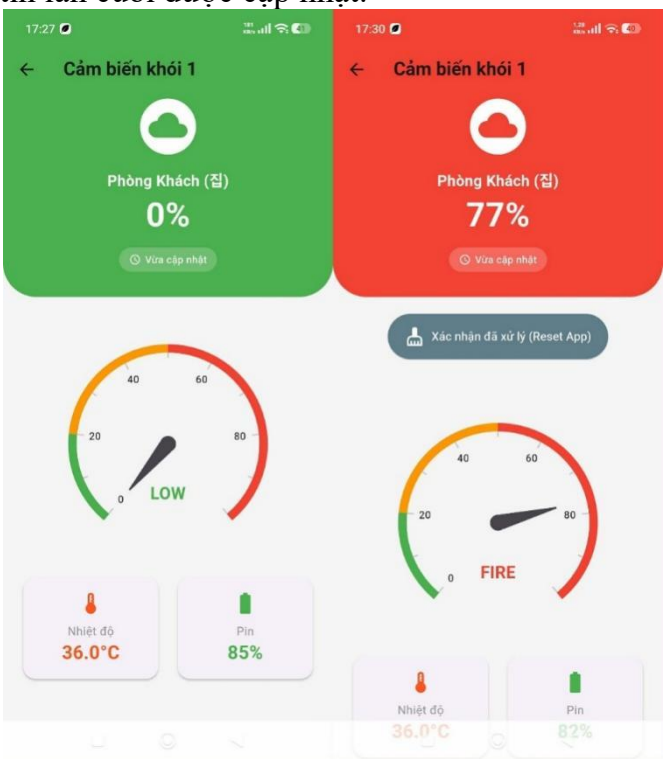
Được hiện thực tại lib/tabs/dashboard.dart. Sử dụng GridView để hiển thị các thẻ DeviceCard. Màu sắc thẻ thay đổi động: Đỏ (Báo cháy/Cảnh báo), Xanh (An toàn), Xám (Mất tín hiệu).



Hình 3. 9 Màn hình giám sát trung tâm

3.5.2. Màn hình chi tiết thiết bị

Được hiện thực tại lib/pages/device_detail_page.dart. Sử dụng đồng hồ đo (Gauge) để trực quan hóa nồng độ khói, kèm theo các thẻ thông tin về Nhiệt độ và Pin lẫn thời gian thông tin lần cuối được cập nhật.

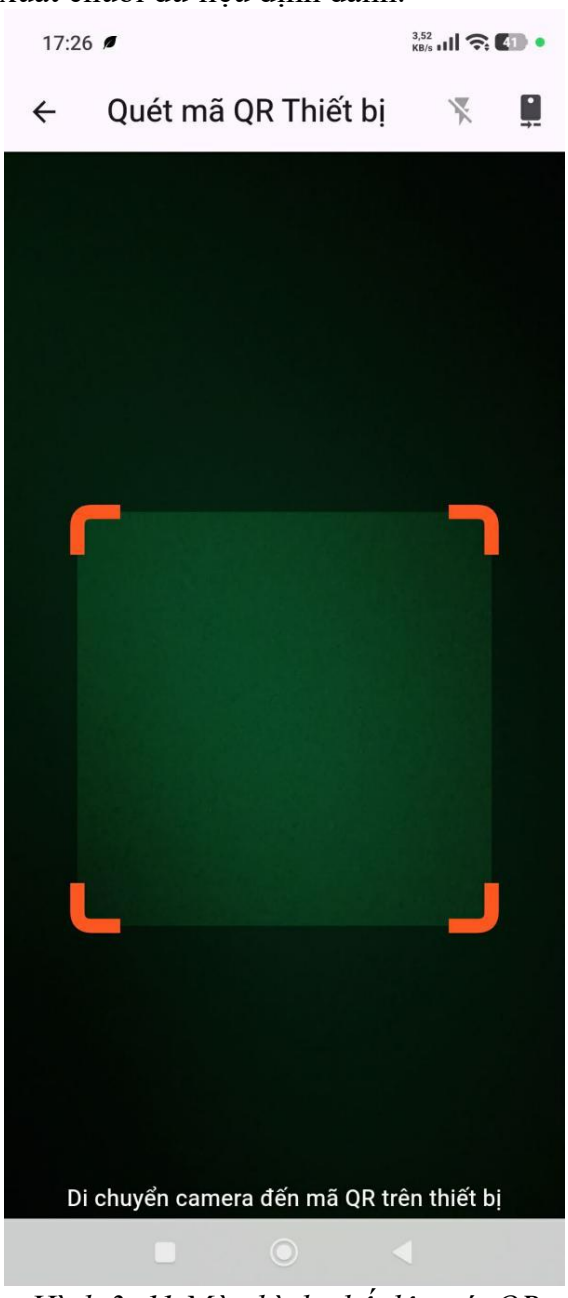


Hình 3. 10 Màn hình chi tiết thiết bị

3.5.3. Tính năng quét QR

Để đơn giản hóa quá trình triển khai lắp đặt, ứng dụng tích hợp tính năng quét mã QR (qr_scanner_page.dart).

- **Quy trình:**
  1. Người dùng chọn chức năng "Thêm thiết bị".
  2. Camera được kích hoạt để quét mã QR dán trên vỏ thiết bị (chứa thông tin DevID).
  3. Hệ thống tự động điền thông tin vào form (add_device_page.dart) và gửi yêu cầu đăng ký lên Server.
- **Kỹ thuật:** Sử dụng thư viện mobile_scanner hoặc tương đương để xử lý hình ảnh từ camera và trích xuất chuỗi dữ liệu định danh.



Hình 3. 11 Màn hình chế độ quét QR

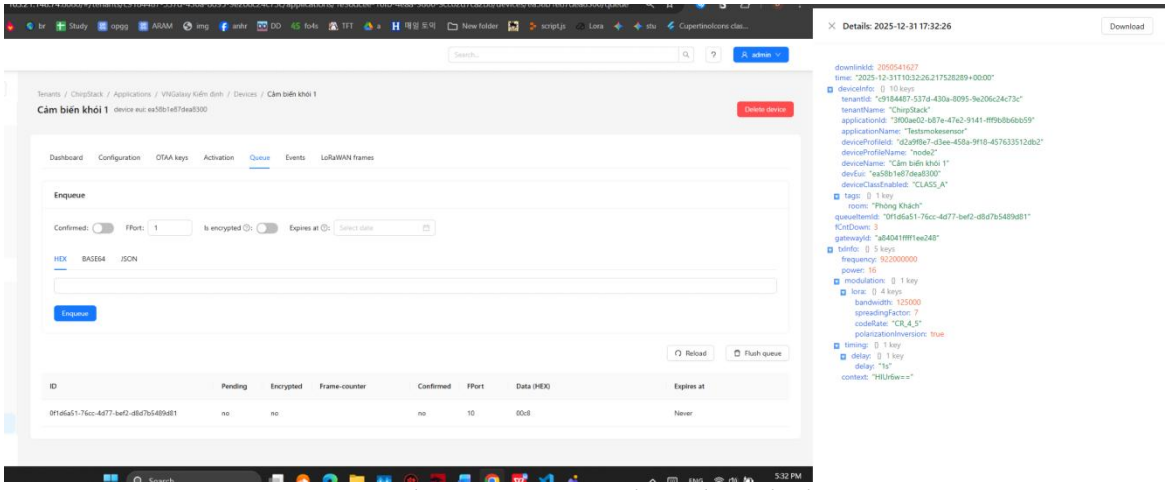
3.5.4. Màn hình điều khiển thiết bị (Downlink LoRaWAN)

Được hiện thực tại lib/tabs/control.dart, cho phép người dùng tương tác hai chiều (Downlink) với thiết bị:

- Lựa chọn thiết bị: Sử dụng Dropdown Menu để chọn thiết bị cần cấu hình từ danh sách quản lý.
- Thiết lập ngưỡng cảnh báo: Người dùng nhập giá trị ngưỡng khói mong muốn (từ 0% đến 100%) vào ô nhập liệu.
- Gửi lệnh Downlink: Khi nhấn nút "Gửi cấu hình", ứng dụng sẽ gọi phương thức enqueueGrpcHack trong SmokeService. Dữ liệu ngưỡng được mã hóa thành chuỗi Base64 và gửi xuống thiết bị thông qua giao thức gRPC-Web, giúp thay đổi độ nhạy của cảm biến theo thời gian thực mà không cần nạp lại code phần cứng.



Hình 3. 12 Màn hình chế độ điều khiển

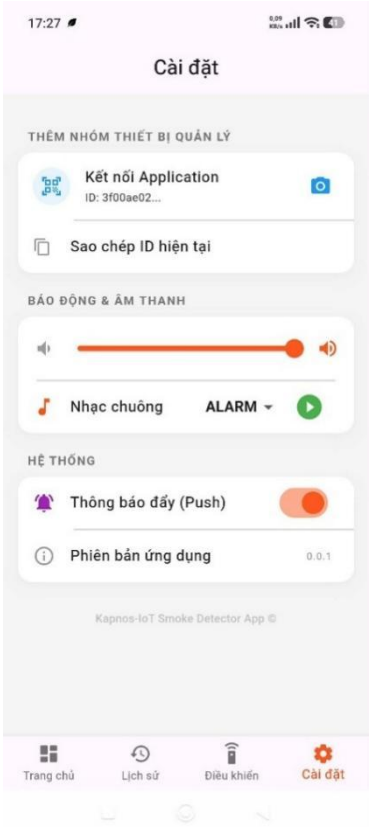


Hình 3. 13 Server nhận downlink

3.5.5. Màn hình cài đặt hệ thống

Được hiện thực tại lib/tabs/settings.dart, cung cấp khả năng cá nhân hóa trải nghiệm người dùng:

- ***Quản lý âm thanh báo động:***
    - ***Điều chỉnh âm lượng:*** Sử dụng thanh trượt (Slider) để tăng giảm âm lượng cảnh báo theo ý muốn.
    - ***Tùy chọn nhạc chuông:*** Cung cấp danh sách các file âm thanh cảnh báo (ví dụ: alarm.mp3, siren.mp3, beep.mp3). Người dùng có thể nghe t trực tiếp trên ứng dụng nhờ thư viện audioplayers.
- ***Lưu trữ cấu hình:*** Toàn bộ cài đặt về âm lượng và loại nhạc chuông được lưu trữ cục bộ bằng SharedPreferences. Điều này đảm bảo các thiết lập của người dùng được ghi nhớ ngay cả khi tắt và mở lại ứng dụng.
- ***Quản lý kết nối:*** Hỗ trợ quét mã QR để cập nhật nhanh Application ID mới, giúp ứng dụng linh hoạt chuyển đổi giữa các dự án hoặc tenant khác nhau trên ChirpStack.



Hình 3. 14 Màn hình chế độ cài đặt

3.5.6. Lịch sử và Truy xuất dữ liệu

Module history.dart và history_helper.dart cho phép người dùng xem lại nhật ký hoạt động.

Dữ liệu lịch sử (thời điểm báo cháy, giá trị nồng độ khói đỉnh điểm) được lưu trữ cục bộ (Local Storage) hoặc truy vấn từ Database server.



Hình 3. 15 Màn hình lịch sử

3.6. Xây dựng các module cốt lõi

3.6.1. Dịch vụ chạy ngầm

Đây là tính năng quan trọng nhất của phân hệ Mobile, đảm bảo người dùng không bỏ lỡ cảnh báo cháy ngay cả khi không mở ứng dụng.

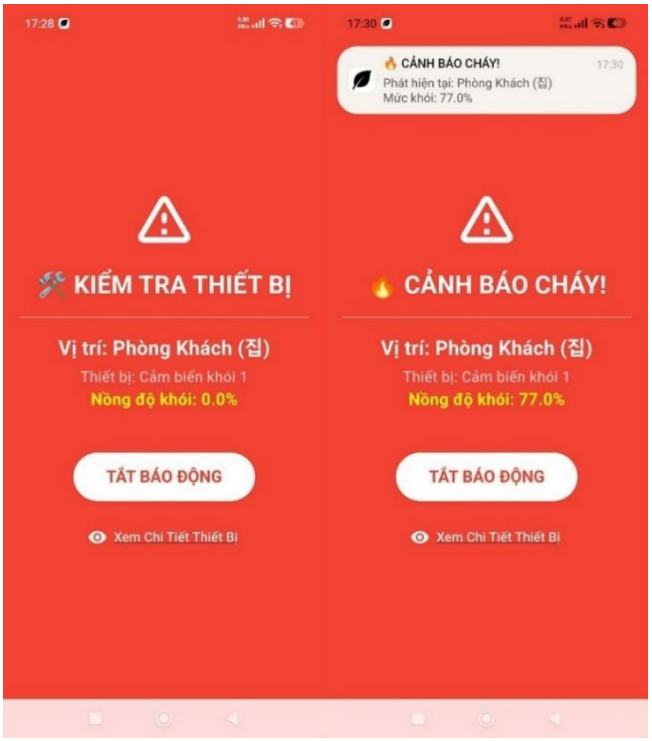
- **Thực thi:** File background_service.dart khởi tạo một Isolate (tiến trình riêng biệt) để duy trì kết nối hoặc định kỳ kiểm tra trạng thái từ Server.
- **Cơ chế thông báo:** Khi nhận được tín hiệu cảnh báo (Alarm Flag = True), module notification_helper.dart sẽ được kích hoạt để:
    - Đẩy thông báo (Push Notification) lên thanh trạng thái với mức độ ưu tiên cao nhất (High Priority).
    - Kích hoạt rung và âm thanh cảnh báo đặc biệt để thu hút sự chú ý của người dùng.



Hình 3. 16 Dịch vụ ngầm (Background Service)

3.6.2. Hệ thống cảnh báo

Hiện thực tại lib/notification_helper.dart. Sử dụng flutter_local_notifications với mức độ ưu tiên cao (Importance.max), kết hợp với flutter_ringtone_player để phát âm thanh còi hú khi nhận được tín hiệu báo cháy từ MQTT.



Hình 3. 17 Màn hình cảnh báo cháy/test

3.6.3. Tự động phát hiện thiết bị

Logic được xử lý tại lib/main_screen.dart. Ứng dụng lắng nghe gói tin MQTT (application/+/device/+/event/up), nếu phát hiện DevEUI mới chưa có trong danh sách, hệ thống sẽ tự động khởi tạo đối tượng SmokeModel (Hình 3. 4) và thêm vào danh sách giám sát mà không cần cấu hình thủ công.

3.7. Kết luận chương

Chương 3 đã trình bày chi tiết quy trình thiết kế và hiện thực hóa ứng dụng di động. Việc áp dụng kiến trúc phân tách giữa Giao diện (UI) và Dịch vụ nền (Background Service) đã giải quyết triệt để bài toán nhận cảnh báo thời gian thực. Đoạn mã xử lý tại phía Client cho thấy sự đồng bộ chặt chẽ với các định nghĩa gói tin từ phía thiết bị (Chương 3), tạo nên một hệ thống giám sát liền mạch và tin cậy.

CHƯƠNG 4: KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Kiểm thử tính năng cơ bản

4.1.1. Cấu hình phần cứng và thiết bị đo lường

a. Cấu hình phần cứng:

- Thiết bị đo (Device Node): 01 node cảm biến hoàn thiện (Module RAK3172, Cảm biến khói quang, Cảm biến nhiệt SHTC3)
- Gateway: Sử dụng Gateway LoRaWAN (Dragino) kết nối Internet, cấu hình tại băng tần AS923.
- Thiết bị giám sát: Webserver và điện thoại thông minh (Android/iOS) cài đặt ứng dụng "Kapnos" (tên dự kiến của App) được xây dựng bằng Flutter.

b. Công cụ hỗ trợ đo lường:

- Serial Monitor (Arduino IDE): Để theo dõi log debug thời gian thực từ cổng UART của thiết bị (Trạng thái máy trạng thái, giá trị ADC, sự kiện gửi gói tin)
- Tạo mẫu cháy: Khói giấy để thử nghiệm cảm biến quang.
- Máy khò nhiệt hoặc máy sấy tóc để thử nghiệm cảm biến nhiệt độ SHTC3.

4.1.2. Môi trường kiểm thử tính năng cơ bản

I. *Điều kiện kiểm thử 1*

- Địa điểm: Hành lang tầng 3-khu C-Đại học Bách khoa Đà Nẵng
- Điều kiện môi trường: Không gian trên mặt đất rộng rãi, thoáng, ít vật cản toà nhà.

II. *Điều kiện kiểm thử 2*

- Địa điểm: Tầng hầm Vincom Plaza Ngô Quyền
- Điều kiệm môi trường: Tầng hầm kín, sóng điện thoại, sóng LoRa kém

a) Kịch bản 1: Hoạt động giám sát định kỳ

- Mục đích: Kiểm tra tính năng gửi tin định kỳ để báo cáo phần trăm khói, nhiệt độ và dung lượng pin.
- Thực hiện: Để thiết bị chạy tự do trong 60 phút.
- Kết quả kỳ vọng:
  - Trên Serial Monitor: Thiết bị thức dậy, in ra Sending Periodic Uplink, sau đó ngủ Sleep.
  - Trên App và Webserver: Các thông số Phần trăm khói, Nhiệt độ và Pin được cập nhật mới nhất.

b) Kịch bản 2: Cảnh báo cháy

- Mục đích: Kiểm tra ngưỡng kích hoạt khói và nhiệt và logic xác thực (Confirming).
- Thực hiện: Phun khói vào buồng đo.
- Kết quả kỳ vọng:
  - Firmware phát hiện giá trị ADC hoặc nhiệt độ tăng vọt vượt ngưỡng.
  - Chuyển trạng thái sang CONFIRMING, sau đó là ALARMING.
  - Serial log: Fire Comfirming!!! Fire Detected!
  - Còi kêu, App nhận thông báo "Phát hiện khói/nhiệt" sau khoảng 5 giây.
  - Nhấn nút Button_User trên thiết bị, tạm thời tắt còi báo động và về trạng thái giám sát trong vòng 10 phút

c) Kịch bản 3: Điều khiển từ xa

- Mục đích: Kiểm tra khả năng nhận lệnh thay đổi cấu hình từ App.
- Thực hiện: Trên App, gửi lệnh thay đổi ngưỡng báo khói từ 180 lên 250.
- Kết quả kỳ vọng:
  - Thiết bị nhận gói tin Downlink tại cửa sổ RX1/RX2 sau lần gửi tin tiếp theo.
  - Serial Log hiển thị: Downlink received. New Threshold: 250.

4.1.4. Kết quả kiểm thử tính năng cơ bản

Đối với điều kiện kiểm thử 1, yêu cầu hệ thống có thể thực hiện được tất cả chức năng cảnh báo được đề ra trong yêu cầu chức năng, phi chức năng, với tỉ lệ lỗi chủ quan khoảng 10-15%

Đối với điều kiện kiểm thử 2, có thể thực hiện được các chức năng như điều kiện một, với tỉ lệ lỗi chủ quan khoảng 10-20%. Đối với tính năng gửi gói tin chấp nhận mọi tỉ lệ lỗi.

Kết quả đạt được:

ĐKKT 1:

ĐKKT1			
Lần test	Kịch bản 1	Kịch bản 2	Kịch bản 3
1	FAIL	FAIL	OK
2	OK	OK	OK
3	OK	OK	OK
4	OK	OK	OK
5	OK	OK	OK
6	OK	OK	OK
7	FAIL	FAIL	OK
8	OK	OK	OK
9	OK	OK	OK
10	OK	OK	OK
11	OK	OK	OK
12	FAIL	FAIL	OK
13	OK	OK	OK
14	OK	OK	OK
15	OK	OK	OK
16	OK	OK	OK
17	OK	OK	OK
18	OK	OK	OK
19	OK	OK	OK
20	OK	OK	OK
Tỉ lệ lỗi từng kịch bản	15.00%	15.00%	0.00%
	Tỉ lệ lỗi ĐKKT1		
	Tỉ lệ lỗi (20 lần test)	10.00%	
	Tổng lỗi	6	

Hình 4. 1 Kết quả của ĐKKT 1

ĐKKT 2:

ĐKKT2			
Lần test	Kịch bản 1	Kịch bản 2	Kịch bản 3
1	FAIL	FAIL	OK
2	OK	OK	OK
3	FAIL	FAIL	OK
4	OK	OK	OK
5	OK	OK	OK
6	OK	OK	OK
7	FAIL	FAIL	OK
8	FAIL	FAIL	OK
9	FAIL	FAIL	OK
10	OK	OK	OK
11	FAIL	FAIL	OK
12	FAIL	FAIL	OK
13	OK	OK	OK
14	OK	OK	OK
15	FAIL	FAIL	OK
16	OK	OK	OK
17	FAIL	FAIL	OK
18	FAIL	FAIL	OK
19	FAIL	FAIL	OK
20	OK	OK	OK
Tỉ lệ lỗi từng kịch bản	55.00%	55.00%	0.00%

Tỉ lệ lỗi ĐKKT2	
Tỉ lệ lỗi (20 lần test)	36.67%
Tổng lỗi	22

Hình 4. 2 Kết quả của ĐKKT 2

4.1.5 Nhận xét kết quả kiểm thử

– Thể hiện tỉ lệ lỗi trên 20 lần kiểm thử:

– Lỗi xuất hiện tại ĐKKT2 cao hơn ĐKKT1 (RSSI trung bình thấp hơn)

– Lỗi xuất hiện chủ yếu ở kịch bản 1: "Hoạt động giám sát định kỳ" dẫn tới kịch bản 2 "Cảnh báo cháy" bị lỗi theo (Lỗi chỉ xuất hiện ở việc gửi thông báo tới app, còn báo động tại chỗ vẫn thực hiện tốt)

– Nguyên nhân: Nguyên nhân khách quan là do vị trí lắp đặt tại ĐKKT2 nằm trong vùng chịu ảnh hưởng bởi hiệu ứng đa đường và vật cản, gây khó khăn cho việc duy trì kết nối liên tục đối với các gói tin.

4.2. Kiểm thử năng lượng tiêu thụ

4.2.1. Cấu hình phần cứng và dụng cụ đo lường

a. ***Cấu hình phần cứng:*** Thiết bị đo (Device Node): 01 node cảm biến hoàn thiện (Module RAK3172, Cảm biến khói quang, Cảm biến nhiệt SHTC3)

b. ***Dụng cụ đo lường:*** Máy phân tích nguồn **Qoitech Otii Arc Power Analyser** để đo dòng tiêu thụ tức thời.

4.2.2. Các hoạt động chính của thiết bị

Thông số đầu vào:

- Dung lượng pin $\mathbf{C_{BAT}}$**:** 1000 mAh.
- Chu kỳ hoạt động chính $\mathbf{T_{Cycle}}$**:** 15 phút = 900 giây.
- Dòng ngủ nền $\mathbf{I_{Sleep}}$**:** 31.855 µA = 0.031855 mA.

Trong 1 chu kỳ 15 phút (900s), thiết bị sẽ thực hiện:

a. Đọc cảm biến

Trong 15 phút (900s), số lần đọc cảm biến là: $\mathbf{T_{READ}}$=900 / 10 = 90 lần.

- $\mathbf{I_{Sensor}}$= 3.773189 mA
- $\mathbf{t_{Sensor}}$ = 57.855 ms = 0.057855 s
- Tiêu thụ điện tích cho 1 lần đọc:

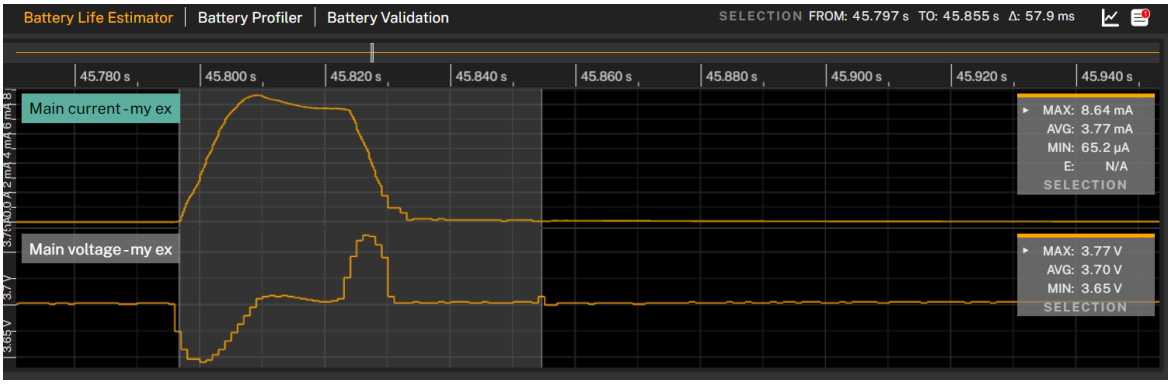
$$\mathbf{q_{Sensor}} = \mathbf{I_{Sensor}} \times \mathbf{t_{Sensor}} \quad (4.1)$$

$$\mathbf{q_{Sensor}} = 3.773189 \times 0.05984 \approx 0.218297 \text{mAs (mili-Ampere giây).}$$

- Tổng tiêu thụ đọc cảm biến trong 15 phút:

$$\mathbf{Q_{Sensor_total}} = \mathbf{T_{READ}} \times \mathbf{q_{Sensor}} \quad (4.2)$$

$$\mathbf{Q_{Sensor_total}} = 90 \times 0.218513 \approx 19.6468 \text{ mAs}$$



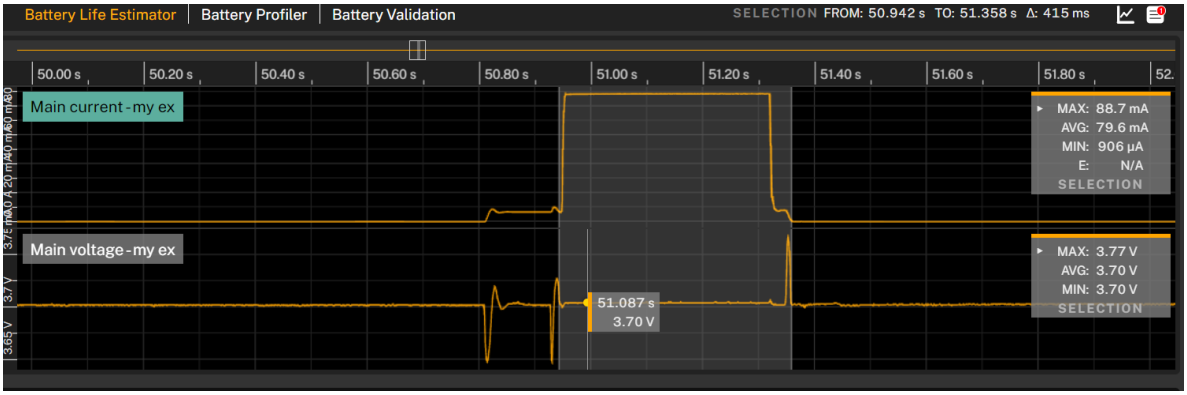
Hình 4. 3 Dòng tiêu thụ khi ở chế độ đọc cảm biến

b. Gửi tin định kỳ

- $\mathbf{I_{tx}}$ = 79.598669 mA
- $\mathbf{t_{tx}}$ = 415.322 ms = 0.415322 s
- Tổng tiêu thụ gửi tin trong 15 phút:

$$\mathbf{Q_{tx}} = \mathbf{I_{tx}} \times \mathbf{t_{tx}} \quad (4.3)$$

$$\mathbf{Q_{tx}} = 79.598669 \times 0.415322 \approx 33.059 \text{mAs}$$



Hình 4. 4 Dòng tiêu thụ khi ở chế độ gửi tin định kỳ

c. *Mở cửa sổ RX*

- $\mathbf{I_{rx}}$= 4.514784mA
- $\mathbf{t_{rx}}$= 444.113ms=0.444113 s
- Tổng thời gian tiêu thụ nhận tin trong 15 phút:

$$\mathbf{Q_{rx}=I_{rx}\times t_{rx}} \quad (4.4)$$

$$\mathbf{Q_{rx}}= 5.013134 \times 0.397851\text{s} \approx 2.005 \text{ mAs}$$



Hình 4. 5 Dòng tiêu thụ khi ở chế độ mở cửa sổ RX

d. *Ngủ*

- Thời gian hoạt động thực tế trong 15 phút:

$$\mathbf{t_{active}= T_{READ}\times t_{Sensor} + t_{tx} + t_{rx}} \quad (4.5)$$

$$\mathbf{t_{active}}= (90\times 0.057855) + 0.415322 + 0.444113$$

$$= 6.066385 \text{ s}$$

- Thời gian ngủ thực tế trong 15 phút:

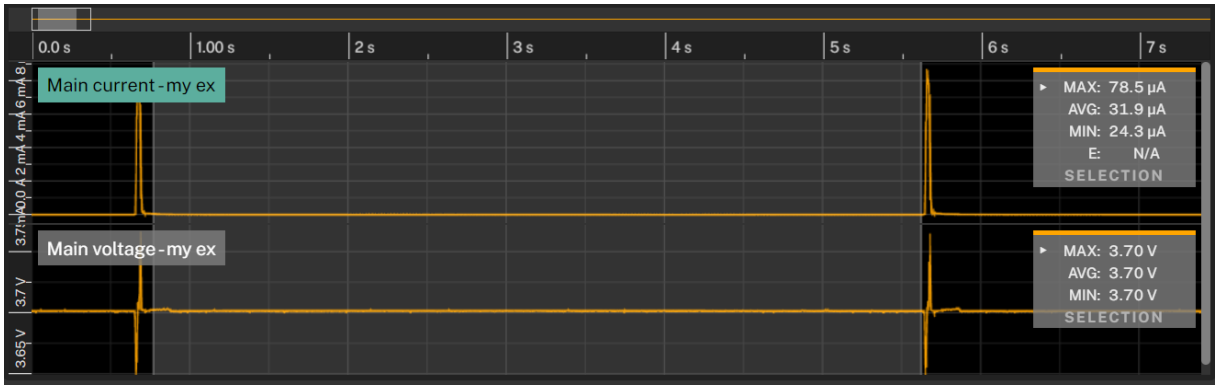
$$\mathbf{T_{sleep}=T_{Cycle}-t_{active}} \quad (4.6)$$

$$\mathbf{T_{sleep}}=900-6.066385 \approx 893.933\text{s}$$

- Tổng tiêu thụ khi ngủ trong 15 phút:

$$\mathbf{Q_{sleep}=I_{Sleep}\times T_{sleep}} \quad (4.7)$$

$$\mathbf{Q_{sleep}}=0.031855 \times 893.933 \approx 28.4762 \text{ mAs}$$



Hình 4. 6 Dòng tiêu thụ khi ở chế độ ngủ

4.2.3. Tính thời gian sử dụng pin

a. Tổng hợp dòng tiêu thụ trung bình:

- Tổng điện lượng tiêu thụ trong 1 chu kỳ 15 phút (900s)

$$\mathbf{Q_{total}=Q_{Sensor_total}+Q_{tx}+Q_{rx}+Q_{Sleep}} \quad (4.8)$$

$$\mathbf{Q_{total}}= 19.6468 +33.059+ 2.005 + 28.4762 = 83.187 \text{ mAs}$$

- Dòng điện trung bình của toàn hệ thống:

$$\mathbf{I_{avg}} = \frac{\boldsymbol{Qtotal}}{\boldsymbol{TCycle}} \quad (4.9)$$

$$\mathbf{I_{avg}} = \frac{83.187 \text{ mAs}}{900\ s} \approx 0.09243 \text{ mA hay } 92.43\mu A$$

b. Tính toán thời lượng sử dụng pin:

Với pin dung lượng 1000 mAh và hệ số xả (thực tế pin không bao giờ xả hết 100%, thường lấy hiệu suất khoảng 85% cho pin Alkaline hoặc NiMH giá rẻ)→Dung lượng khả dụng: 1000 mAh × 0.85 = 850 mAh.

Thời gian hoạt động

$$\mathbf{T_{hour}}=\frac{\mathbf{CBAT}}{\mathbf{Iavg}} \quad (4.10)$$

$$\mathbf{T_{hour}}=\frac{850 \text{ mAh}}{0.09243} \approx 9196.14 \text{ (giờ)} \approx 383.17 \text{ (ngày)} \approx 12.58 \text{(tháng)}$$

4.2.4. Nhận xét kết quả kiểm thử

Với cấu hình dung lượng pin hiện tại (1000mAh):

Thời gian hoạt động ước tính là hơn 12 tháng

Thành phần tiêu thụ:

- Dòng ngủ: chiếm khoảng 34%. Điều này cho thấy dòng ngủ 31.855 µA hiện tại tạm chấp nhận so với chuẩn Low-Power
- Gửi Packet (TX): Chiếm khoảng 39.7%
- Đọc cảm biến: Chiếm khoảng 23.6%
- Nhìn chung hệ thống đã đáp ứng được yêu cầu thời lượng pin từ 8 tháng tới 1

năm

- Đây chỉ là điều kiện kiểm thử, nếu vào thực tiễn ta phải giảm tần suất gửi tin xuống để có thể tối ưu thời lượng pin một cách hiệu quả (VD: giảm tần suất gửi tin xuống thành 2 tiếng/lần gửi ta có được thời lượng pin khoảng 19.9 tháng chưa kể việc tăng dung lượng pin)

4.4. Kết luận chương

Như vậy, chương 4 đã đưa ra một số kịch bản, điều kiện, kỳ vọng kiểm thử của một số chức năng trong hệ thống. Từ đó, có thể dựa vào để đánh giá khả năng hoạt động ổn định của chức năng, hệ thống.

KẾT LUẬN

1. Đánh giá hệ thống

Sau quá trình thiết kế, thi công và thử nghiệm, Thiết kế và triển khai hệ thống cảnh báo cháy sử dụng công nghệ LoraWAN đã đạt được những kết quả khả quan. Tôi xin đưa ra những đánh giá khách quan về ưu điểm và nhược điểm của hệ thống như sau:

- ***Ưu điểm***
  - Về khả năng truyền thông: Hệ thống đã ứng dụng thành công công nghệ LoRaWAN, cho phép truyền dữ liệu ở khoảng cách xa (theo thử nghiệm thực tế) và khả năng xuyên vật cản tốt hơn so với các công nghệ tầm ngắn như Wi-Fi hay Zigbee. Điều này phù hợp với yêu cầu giám sát tại các khu vực rộng lớn hoặc các tòa nhà cao tầng.
  - Về tối ưu năng lượng: Firmware của thiết bị đo (End-node) đã được thiết kế tối ưu với cơ chế Sleep Mode (ngủ sâu). Thiết bị chỉ thức dậy khi đến chu kỳ gửi tin hoặc khi có sự kiện bất thường (ngưỡng khói hoặc nhiệt độ vượt mức cài đặt). Dòng tiêu thụ ở chế độ ngủ đạt mức thấp (~31 µA), giúp kéo dài thời gian hoạt động của pin.
  - Về độ tin cậy của cảnh báo: Thuật toán phát hiện sự cố được xây dựng dựa trên sự kết hợp dữ liệu đa cảm biến bao gồm nhiệt độ (SHTC3) và nồng độ khói. Cơ chế kiểm tra 2 bước (Monitoring → Confirming → Alarming) trong mã nguồn giúp giảm thiểu tình trạng báo cháy giả do nhiễu môi trường.
  - Về trải nghiệm người dùng: Ứng dụng di động (Mobile App) được phát triển trên nền tảng Flutter hoạt động mượt mà trên Android. Giao diện trực quan, tính năng quét QR Code để thêm thiết bị và lịch sử cảnh báo giúp người dùng dễ dàng quản lý và giám sát.
- ***Nhược điểm***
  - Độ trễ truyền tin (Latency): Do đặc thù của giao thức LoRaWAN hoạt động ở Class A (ưu tiên tiết kiệm năng lượng), việc gửi lệnh điều khiển (Downlink) từ App xuống thiết bị (ví dụ: tắt còi báo động từ xa hoặc thay đổi ngưỡng) không được thực thi ngay lập tức mà phải đợi đến chu kỳ gửi tin tiếp theo của thiết bị.
  - Độ chính xác của cảm biến: Do có chế độ ngủ (thức giấc mới đọc cảm biến) nên đôi lúc sẽ cảnh báo chậm hơn so với các cảm biến có sử dụng nguồn trực tiếp.
  - Phụ thuộc hạ tầng mạng: Hoạt động của hệ thống phụ thuộc vào sự ổn định của Gateway LoRaWAN (đối với thiết bị) và kết nối Internet (đối với Mobile App). Nếu mất kết nối tại Gateway, dữ liệu sẽ không thể đẩy lên Server.
  - Thời lượng sử dụng: Do hạn chế về mặt phần vỏ thiết bị, dung lượng pin hiện tại còn thấp ($\approx 1000mAh$)

2. Kết luận chung

Đồ án "Thiết kế và triển khai hệ thống cảnh báo cháy sử dụng công nghệ LoraWAN" đã hoàn thành các mục tiêu cơ bản đề ra ban đầu:

1. **Xây dựng thành công phần cứng:** Thiết kế và chế tạo node cảm biến tích hợp module LoRaWAN, cảm biến nhiệt độ SHTC3 và cảm biến khói, hoạt động ổn định bằng nguồn pin.
2. **Làm chủ công nghệ truyền thông:** Thiết lập thành công mạng LoRaWAN, cấu hình kết nối OTAA (Over-The-Air Activation) đảm bảo tính bảo mật và toàn vẹn dữ liệu.
3. **Phát triển hệ thống phần mềm:** Xây dựng hoàn thiện ứng dụng di động với đầy đủ các chức năng giám sát thời gian thực, cảnh báo qua thông báo đẩy (Push Notification) và lưu trữ lịch sử dữ liệu thông qua giao thức MQTT.
4. **Chi phí:** Chi phí hoàn thiện phần cứng Device khi làm số lượng nhỏ: khoảng 500 nghìn vnđ trên một Device. Nếu như có khả tối ưu linh kiện phần cứng cũng như sản xuất với số lượng lớn, chi phí dự kiến 1 Device là 350-400 nghìn vnđ. Thấp hơn nhiều so với các thiết bị báo khói khác có kết nối trên thị trường (Giá dao động hơn 800 nghìn)

Kết quả thử nghiệm cho thấy hệ thống hoạt động đúng theo kịch bản thiết kế, thời gian đáp ứng khi có sự cố cháy nằm trong ngưỡng chấp nhận được, khẳng định tính khả thi của việc ứng dụng LoRaWAN trong các bài toán giám sát an toàn cháy nổ dân dụng.

3. Hướng phát triển đề tài

Để hệ thống có thể hoàn thiện hơn và hướng tới thương mại hóa hoặc triển khai thực tế quy mô lớn, tôi xin đề xuất các hướng phát triển tiếp theo:

- **Nâng cấp phần cứng:**
  - Thiết kế lại vỏ hộp mới ABS chống cháy, kích thước phù hợp cho pin với dung lượng lớn hơn.
  - Tích hợp Module SIM, dự phòng trong trường hợp mất kết nối Internet/LoRaWAN.
  - Cải tiến thiết bị có thể kết nối với loa độc lập với công suất âm thanh lớn hơn.
  - Triển khai mô hình LoRa Mesh để mở rộng phạm vi giám sát xuống các tầng hầm sâu hoặc các khu vực bị che khuất sóng trong các tòa nhà cao tầng.
- **Cải tiến thuật toán và phần mềm:**
  - Tại Node**:** Tối ưu hóa thuật toán "Listen Before Talk" (Thiết bị phải nghe kênh truyền có rảnh hay không trước khi thực sự truyền dữ liệu) để giảm xung đột gói tin khi mở rộng số lượng node.
  - Xây dựng cơ chế tự động Join lại mạng khi Gateway bị sự cố.
  - Xây dựng Web Dashboard dành cho người quản lý (Admin) để giám sát tập trung nhiều khu vực/tòa nhà cùng lúc trên bản đồ số.

- Thay thế API gán cứng bằng cách thiết kế màn hình đăng nhập cho người dùng. Sau khi xác thực thành công, Server cấp Token riêng cho từng tài khoản. Token được lưu trữ an toàn trong nội bộ ứng dụng thay vì gắn cứng trong mã nguồn.

TÀI LIỆU THAM KHẢO

[1] Cục Cảnh sát Phòng cháy, Chữa cháy và Cứu nạn cứu hộ, "Thông cáo báo chí về tình hình PCCC và CNCH năm 2023," Bộ Công an, 08/01/2024. [Trực tuyến]. Xem tại: https://canhsatpccc.gov.vn/vi/news/thong-cao-bao-chi/thong-cao-bao-chi-ve-pccc-va-cnch-nam-2023-3827. [Ngày truy cập: 08/11/2025].

[2] Cục Cảnh sát Phòng cháy, Chữa cháy và Cứu nạn cứu hộ, "Thông cáo báo chí về công tác PCCC và CNCH năm 2024," Bộ Công an, 11/01/2025. [Trực tuyến]. Xem tại: https://canhsatpccc.gov.vn/vi/news/thong-cao-bao-chi/thong-cao-bao-chi-ve-phong-chay-chua-chay-va-cuu-nan-cuu-ho-nam-2024-368. [Ngày truy cập: 11/11/2025].

[3] Actility, "LoRaWAN Architecture & Network Server," Actility. [Trực tuyến]. Xem tại: https://www.actility.com/lorawan-architecture-network-server/. [Ngày truy cập: 15/10/2025].

[4] RAKwireless, "RAK3172 Module Quick Start Guide," RAKwireless Docs. [Trực tuyến]. Xem tại: https://docs.rakwireless.com/product-categories/wisduo/rak3172-module/quickstart/. [Ngày truy cập: 15/10/2025].

[5] JSON.org, "Introducing JSON," json.org. [Trực tuyến]. Xem tại: https://www.json.org/json-en.html. [Ngày truy cập: 02/12/2025].

[6] Mann Howie, "What is a REST API? A Complete Guide," 24/03/2023. [Trực tuyến]. Xem tại: https://mannhowie.com/rest-api. [Ngày truy cập: 09/12/2025].

[7] Thanh Thư, "MQTT là gì & Vai trò của MQTT trong IoT," Viblo.asia, 09/11/2020. [Trực tuyến]. Xem tại: https://viblo.asia/p/mqtt-la-gi-vai-tro-cua-mqtt-trong-iot-V3m5WL3bKO7. [Ngày truy cập: 09/12/2025].

[8] Nguyen Hong Son, "JWT từ cơ bản đến chi tiết," Viblo.asia, 13/05/2019. [Trực tuyến]. Xem tại: https://viblo.asia/p/jwt-tu-co-ban-den-chi-tiet-LzD5dXwe5jY. [Ngày truy cập: 09/12/2025].

[9] Etinco, "UART là gì? Tổng quan về UART & ứng dụng trong kỹ thuật," Etinco.vn, 16/09/2024. [Trực tuyến]. Xem tại: https://etinco.vn/uart-la-gi/. [Ngày truy cập: 02/12/2025].

[10] Digital Logic & Programming, "14.3(a) - Serial Communication on the MSP430: I2C - What is I-Squared C and why the Resistors?," YouTube, 06/05/2020. [Được ghi hình]. Xem tại: https://www.youtube.com/watch?v=kWz6ekvoNbw. [Ngày truy cập: 14/04/2024].

[11] TEKTELIC, "Over-The-Air Activation (OTAA) in LoRaWAN," TEKTELIC Communications, 2024. [Trực tuyến]. Xem tại: https://tektelic.com/what-it-is/over-the-air-activation/. [Ngày truy cập: 15/10/2025].

[12] Grant Maloy Smith, "What is an ADC Converter?," Dewesoft Blog, 13/02/2024. [Trực tuyến]. Xem tại: https://dewesoft.com/blog/what-is-adc-converter. [Ngày truy cập: 09/11/2025].

[13] quocbao, "BIT_MATH — Các phép toán thao tác trên bit," Arduino.vn, 11/07/2014. [Trực tuyến]. Xem tại: http://arduino.vn/reference/bit-math-cac-

phep-toan-thao-tac-tren-bit [Ngày truy cập: 20/10/2025].
[14] İpek Yavuz KONAK, "How Optical Smoke Detectors Work," Mavili, 04/12/2017. [Trực tuyến]. Xem tại: https://mavili.com.tr/en/how-optical-smoke-detectors-work/. [Ngày truy cập: 09/11/2025].
[15] Farnell element14, "Power Calculations for IoT," element14, 15/08/2018. [Trực tuyến]. Xem tại: https://in.element14.com/power-calculations-for-iot. [Ngày truy cập: 13/12/2025].

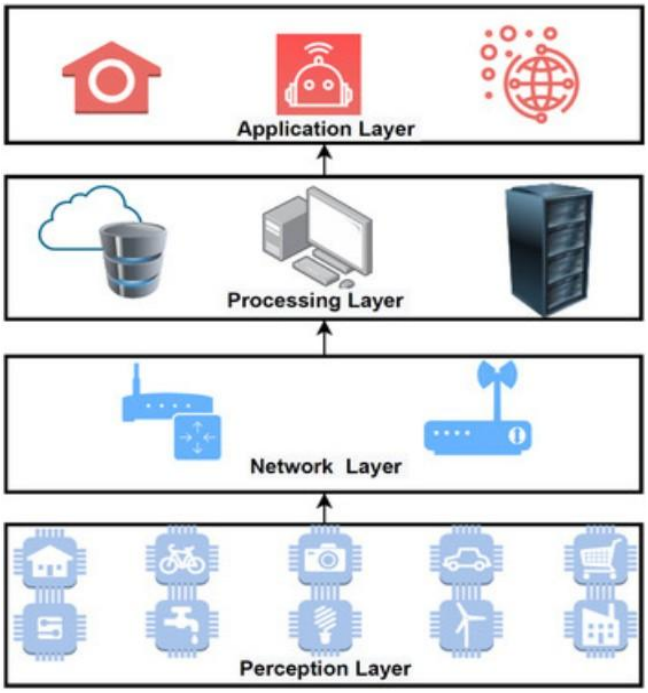
PHỤ LỤC

Phụ luc 1. Hệ thống IOT

Hệ thống IoT (Internet of Things) là một mạng lưới kết nối các đối tượng vật lý, thiết bị thông minh và cảm biến thông qua internet, cho phép chúng tương tác và trao đổi dữ liệu với nhau. Một hệ thống IoT thường bao gồm các lớp sau

- Perception Layer: lớp đầu tiên trong hệ thống IoT. Nó bao gồm các cảm biến và thiết bị đo lường để thu thập dữ liệu từ môi trường vật lý. Các cảm biến có thể đo nhiệt độ, độ ẩm, ánh sáng, chuyển động, định vị GPS, cảm biến khí, cảm biến hình ảnh và nhiều loại cảm biến khác. Nhiệm vụ của lớp này là thu thập dữ liệu và chuyển tiếp nó đến lớp tiếp theo.
- Network Layer: lớp trung gian xử lý và truyền dữ liệu giữa Perception và các lớp tiếp theo. Lớp này xử lý việc chuyển tiếp, mã hóa và giải mã dữ liệu thu thập từ lớp Perception. Nó đảm bảo rằng dữ liệu được truyền đi một cách an toàn và hiệu quả. Lớp Network sử dụng các giao thức mạng như Wi-Fi, Bluetooth, Zigbee, LoRaWAN và các giao thức mạng khác để kết nối các thiết bị IoT với nhau và với internet.
- Processing Layer: đóng vai trò nhận dữ liệu từ lớp Network và tiến hành xử lý, phân tích và lọc dữ liệu. Lớp này có thể áp dụng các thuật toán học máy, trí tuệ nhân tạo và xử lý dữ liệu để tạo ra thông tin hữu ích từ dữ liệu thu thập được. Nhiệm vụ của lớp Processing là biến dữ liệu thành thông tin có ý nghĩa và cung cấp dữ liệu đã được xử lý cho lớp Application.

Application Layer: nhận dữ liệu đã được xử lý từ lớp Processing Layer và cung cấp các dịch vụ, ứng dụng và giao diện người dùng. Lớp này cho phép người dùng tương tác với hệ thống IoT, quản lý và điều khiển các thiết bị IoT, và tận dụng các thông tin hữu ích từ dữ liệu thu thập được. Các ứng dụng IoT



Hình 0. 1 Các lớp trong hệ thống IOT

Phụ lục 2. JSON DATA

JSON (JavaScript Object Notation) [5] là một định dạng trao đổi dữ liệu nhẹ, dựa trên văn bản được thiết kế để lưu trữ và truyền tải cấu trúc dữ liệu có tính phân cấp giữa các hệ thống khác nhau trong môi trường mạng. JSON lấy cảm hứng từ cú pháp đối tượng của JavaScript nhưng hoàn toàn độc lập với ngôn ngữ lập trình và được hỗ trợ bởi hầu hết ngôn ngữ hiện đại cho việc phân tích và sinh dữ liệu.

Dữ liệu JSON được cấu trúc theo các cặp khóa-giá trị (key-value), trong đó giá trị có thể thuộc một trong các kiểu dữ liệu cơ bản sau: chuỗi (string), số (number), logic (boolean), null, đối tượng (object) hoặc mảng (array). Điều này cho phép JSON thể hiện các cấu trúc phức tạp và lồng nhau theo dạng cây.

Ví dụ

```
{
type:"PERIODIC"
smoke_level:"LOW"
temperature:36
smoke_percent:4
battery_percent:0
}
```

JSON thường được sử dụng rộng rãi trong các API và hệ thống web-services để trao đổi dữ liệu giữa máy chủ và máy khách, cũng như giữa các dịch vụ backend với nhau do tính dễ đọc, dễ phân tích và hỗ trợ đa ngôn ngữ.

Phụ lục 3. Tiêu chuẩn JWT

Tiêu chuẩn JWT (JSON Web Token) [8] là một phương tiện định dạng và trao đổi thông tin an toàn dựa trên định dạng JSON giữa các bên. JWT là một chuỗi được mã hóa chứa thông tin về người dùng hoặc thông tin xác thực, được sử dụng trong việc xác thực và phân quyền trong các ứng dụng web và dịch vụ web.

JWT được định nghĩa trong các tài liệu tiêu chuẩn của IETF (Internet Engineering Task Force) với mục đích cụ thể là RFC 7519. Nó là một tiêu chuẩn mở, linh hoạt và bảo mật, cho phép các bên trao đổi thông tin an toàn theo cơ chế phát hành token.

JWT bao gồm ba phần chính, mỗi phần cách nhau bởi dấu chấm (.):

- Header: Chứa các thông tin mô tả về loại token và thuật toán mã hóa được sử dụng. Header thường chứa hai thuộc tính quan trọng là "typ" (kiểu) và "alg" (algorithm).

- Payload: Chứa các thông tin cần thiết và có thể truyền tải, chẳng hạn như thông tin người dùng, quyền hạn, thông tin liên quan đến phiên làm việc và nhiều thuộc tính khác. Payload không được mã hóa và có thể dễ dàng đọc được bởi bất kỳ ai.

- Signature: Được tạo bằng cách ký số (được mã hóa) của phần header và payload bằng một khóa bí mật duy nhất chỉ được biết đến bởi server. Nó được sử dụng để xác minh tính toàn vẹn của token và đảm bảo rằng token không bị sửa đổi khi được truyền tải.

Khi một ứng dụng xác thực người dùng, nó gửi JWT cho người dùng sau khi họ đăng nhập thành công. Sau đó, JWT sẽ được gửi kèm với mỗi yêu cầu từ người dùng đến máy chủ. Máy chủ sẽ xác thực token và kiểm tra tính hợp lệ của nó trước khi cho phép truy cập vào các tài nguyên hoặc chức năng cần được bảo mật.

Trong mô hình xác thực, khi người dùng đăng nhập thành công (hoặc sử dụng khóa API được cấp trước), máy chủ sẽ trả về một JWT. Khi người dùng muốn truy cập vào các tài nguyên hoặc đường dẫn (Routes) được bảo vệ, User Agent (ví dụ: Mobile App) sẽ gửi JWT này kèm theo, thường là trong Authorization header với lược đồ Bearer. Máy chủ sẽ xác thực chữ ký của token, nếu hợp lệ sẽ cấp quyền truy cập.

```
// Key API
static const String apiKey =
    'eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJjaGlycHN0YWNrIiwiaXNzIjoiY2hpcnBzdGFjayIsInN1YiI6IjkyN2ZiYzZkLTg2M2MtNDg1Yi04MTMxLTk3Ym
```

Hình 0. 3 API key được sử dụng trong dự án

Phụ lục 4. Tiêu chuẩn REST và RESTful API

Tiêu chuẩn REST (Representational State Transfer) [6] là một kiến trúc dựa trên các nguyên tắc thiết kế cho việc xây dựng các ứng dụng phân tán và các dịch vụ web. Nó được giới thiệu bởi Roy Fielding trong luận văn tiến sĩ vào năm 2000 và đã trở thành một trong những phương pháp phổ biến để thiết kế các API (Application Programming Interface) trong việc tạo ra các dịch vụ web.

Một số điểm quan trọng của kiến trúc REST bao gồm:

- Cơ sở dữ liệu đại diện (Representational): Mỗi tài nguyên (resource) trong hệ thống được đại diện bởi một URI (Uniform Resource Identifier), chẳng hạn như URL (Uniform Resource Locator). Ví dụ, tài nguyên có thể là một bài viết, người dùng, hoặc sản phẩm.

- Trạng thái (State): Trạng thái của tài nguyên được trả về trong phản hồi khi yêu cầu xem tài nguyên. Các tài nguyên có thể có nhiều trạng thái khác nhau, và REST đảm bảo rằng các thao tác CRUD (Create, Read, Update, Delete) có thể được thực hiện dễ dàng

- Truyền tải thông tin qua các phương thức HTTP: REST sử dụng các phương thức HTTP như GET (lấy thông tin), POST (tạo mới tài nguyên), PUT (cập nhật tài nguyên), DELETE (xóa tài nguyên) để quản lý tài nguyên và thực hiện các thao tác trên chúng.

- Stateless (Không lưu trạng thái): Server không lưu trữ thông tin trạng thái của client giữa các yêu cầu. Mỗi yêu cầu từ client phải chứa đủ thông tin cần thiết để xử lý yêu cầu này mà không cần dựa vào thông tin từ yêu cầu trước đó.

- Quy định các cấu trúc dữ liệu đơn giản: REST thường sử dụng các định dạng dữ liệu như JSON hoặc XML để truyền tải thông tin giữa client và server.

- Kiến trúc REST cho phép các ứng dụng có thể hoạt động độc lập và liên kết với nhau thông qua các API. Điều này giúp giảm sự ràng buộc giữa các thành phần trong hệ thống, làm cho ứng dụng linh hoạt và dễ dàng mở rộng. Nó cũng là một tiêu chuẩn rất phổ biến trong việc xây dựng các dịch vụ web và ứng dụng web hiện đại.

- RESTful API là một cách triển khai của tiêu chuẩn REST. Nó là một API (Application Programming Interface) được thiết kế và triển khai theo các nguyên tắc và tiêu chuẩn của REST. Một API được gọi là RESTful nếu nó tuân thủ các nguyên tắc cơ bản của REST.

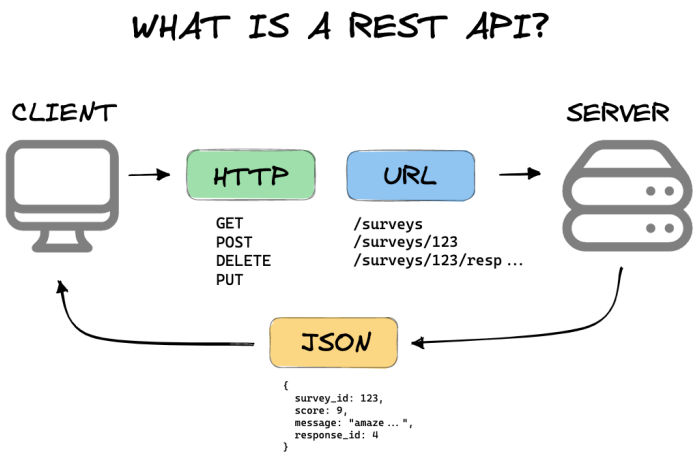
- Một API RESTful sử dụng các phương thức HTTP chuẩn như GET, POST, PUT và DELETE để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên các tài nguyên. Các tài nguyên được xác định bằng các URI (Uniform Resource Identifier) và trạng thái của tài nguyên được truyền tải qua các phản hồi từ server. Các phương thức HTTP thường được sử dụng trong RESTful API bao gồm:

- GET: Lấy thông tin về một tài nguyên hoặc danh sách tài nguyên.

- POST: Tạo mới một tài nguyên.

- PUT: Cập nhật tài nguyên đã tồn tại.

- DELETE: Xóa một tài nguyên.



Hình 0. 3 Mô hình RestAPI

Phụ lục 5. Mô hình giao tiếp Client – Server

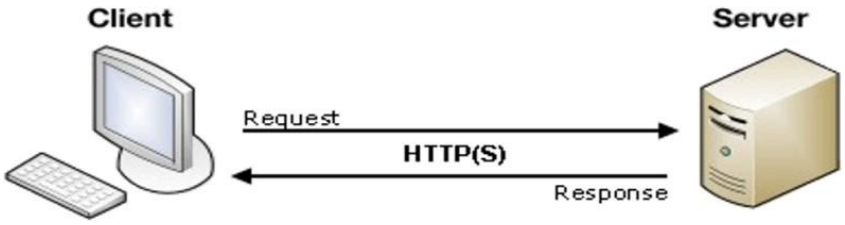
Mô hình Client-Server là một kiến trúc phần mềm phổ biến trong công nghệ thông tin, nơi ứng dụng hoạt động theo cơ chế phân tách thành hai phần: client (người dùng hoặc ứng dụng người dùng) và server (máy chủ). Các phần này tương tác với nhau thông qua mạng hoặc giao thức mạng như HTTP.

Mô hình Client-Server có hai thành phần chính:

- Client: Đây là phần sẽ được cài đặt và chạy trên máy tính hoặc thiết bị của người dùng cuối (như máy tính cá nhân, Smartphone, máy tính bảng, trình duyệt web, v.v.). Nhiệm vụ của client là gửi các yêu cầu đến server và chờ phản hồi từ server. Client có trách nhiệm hiển thị dữ liệu hoặc thông tin được nhận từ server và cung cấp giao diện người dùng để tương tác với ứng dụng.

- Server: Đây là phần mềm hoặc máy chủ được cài đặt trên các máy tính hoặc hệ thống máy chủ chuyên dụng. Server nhận các yêu cầu từ client, xử lý yêu cầu và trả về phản hồi chứa dữ liệu hoặc thông tin được yêu cầu. Server chịu trách nhiệm xử lý các tác vụ phức tạp và thường có tài nguyên cao hơn để đáp ứng nhiều yêu cầu từ client cùng một lúc.

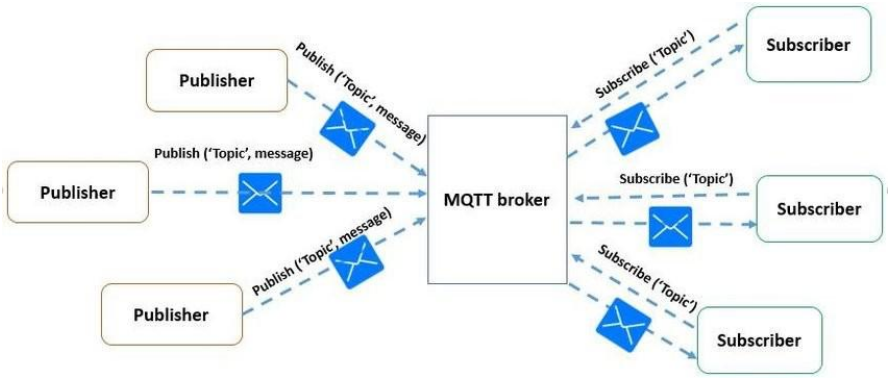
Trong mô hình này, client và server tương tác qua mạng, thông qua giao thức truyền thông như HTTP, TCP/IP, v.v. Client gửi yêu cầu đến server, sau đó server xử lý yêu cầu và trả về phản hồi cho client. Mô hình Client-Server giúp phân tách trách nhiệm giữa client và server, giúp hệ thống dễ dàng mở rộng, bảo trì và phát triển thêm các tính năng mới.



Hình 0. 5 Mô hình Client-Server

Phụ lục 6: Giao thức truyền thông MQTT & Mô hình MQTT Broker

MQTT (Message Queuing Telemetry Transport) [7] là một giao thức truyền thông được phát triển ban đầu bởi IBM vào những năm 1999 và sau đó trở thành một tiêu chuẩn công cộng. MQTT được thiết kế để hỗ trợ truyền tải dữ liệu giữa các thiết bị và ứng dụng trong môi trường Internet of Things (IoT) và các hệ thống giám sát từ xa. Giao thức này chủ yếu được sử dụng để giao tiếp giữa các thiết bị có băng thông thấp, mạng không ổn định và yêu cầu tiêu thụ năng lượng thấp.



Hình 0. 6 Mô hình MQTT

Các thành phần trong mô hình MQTT

- MQTT Client: là Publisher/Subcriber, là các thiết bị/ứng dụng Client kết nối đến Broker để thực hiện truyền nhận dữ liệu. Client thì được chia thành hai nhóm là Publisher và Subscriber. Một Client có thể có 1 trong 2 nhiệm vụ hoặc cả 2. Publisher là thiết bị gửi bản tin lên MQTT Broker, Subscriber là người nhận bản tin mỗi khi có bản tin mới gửi lên MQTT Broker.

- MQTT Broker: MQTT Broker là trung tâm quản lý thông điệp và kết nối giữa các thiết bị. Nhiệm vụ chính của Broker là nhận thông điệp (message) từ Publisher, xếp vào hàng đợi rồi chuyển đến một địa điểm cụ thể. Nhiệm vụ phụ của Broker là nó có thể đảm nhận thêm một vài tính năng liên quan tới quá trình truyền thông như: bảo mật message, lưu trữ message, logs v.v. Một số dịch vụ MQTT Broker đang cung cấp như: HiveMQ, Mosquitto, MQTTRoute, Jmqtt v.v.

- Topics: Mỗi MQTT Client thực hiện truyền/nhận dữ liệu với nhau thông qua các Topic

được quản lý bởi MQTT Broker. Để nhận dữ liệu từ Publisher, đầu tiên Subcriber phải subscribe đến một Topic, sau đó bất cứ Client nào publish dữ liệu đến đúng Topic, thì Broker sẽ lọc và chuyển tiếp gói tin đến đúng Subscriber đó. Một Client có thể subscribe hoặc publish đến nhiều Topic khác nhau.

Phụ lục 7: Cross-platform Mobile Application

Cross-platform Mobile application là một loại ứng dụng di động được phát triển để chạy trên nhiều hệ điều hành và nền tảng khác nhau. Thay vì phải phát triển các ứng dụng riêng biệt cho mỗi hệ điều hành như iOS (Apple) và Android(Google), việc sử dụng công nghệ này cho phép nhà phát triển tạo ra một phiên bản duy nhất, đồng nhất của ứng dụng, sau đó có thể chạy trên nhiều nền tảng khác nhau.

Các công nghệ phổ biến để xây dựng mobile app cross platform bao gồm React Native, Flutter và Xamarin… Các công nghệ này cho phép nhà phát triển sử dụng ngôn ngữ lập trình và framework chung để xây dựng ứng dụng cho cả iOS và Android. Điều này giúp tiết kiệm thời gian, công sức và tài nguyên so với việc phát triển hai ứng dụng riêng biệt cho mỗi nền tảng.

Phụ lục 8: Ngôn ngữ lập trình Dart

1. Giới thiệu về ngôn ngữ lập trình Dart

Dart là một ngôn ngữ lập trình hiện đại, được phát triển bởi Google vào năm 2011 và phát hành chính thức phiên bản đầu tiên (1.0) vào năm 2013.

Ý định ban đầu của Google khi phát triển Dart là để thay thế Javascript – ngôn ngữ chạy trên trình duyệt Google Chrome của họ. Tuy nhiên, dự án đã thất bại, Dart không còn được sử dụng nhiều cho đến khi Google triển khai Flutter, và phát hành phiên bản Flutter đầu tiên vào năm 2018 (Framework sử dụng Dart).

Dart ngày nay được biết đến với nhiều khả năng mạnh mẽ, nó giúp cải thiện quá trình phát triển các ứng dụng trên mọi nền tảng, mọi màn hình trên các thiết bị.

Dart là một ngôn ngữ hướng đối tượng nguyên thủy (hay ngôn ngữ hướng đối tượng hoàn toàn) – Pure OOP language (Complete OOP language), có nghĩa là mọi thứ trong Dart đều là Object (đối tượng). Dart không có các kiểu dữ liệu nguyên thủy (Ví dụ: int, char, float…), những kiểu dữ liệu trong Dart đều là Object, thuộc Class num.

2. Một số đặc điểm của ngôn ngữ Dart

Dart là một ngôn ngữ lập trình "type safe", có nghĩa là những thao tác, hành động được thực hiện trên dữ liệu nào thì cần được cho phép bởi kiểu dữ liệu đó

Dart cũng khá mạnh trong việc phân tích lỗi trong quá trình viết mã (Dart hỗ trợ cả 2 công nghệ Static analyzer, Runtime check)

Dart có khả năng cảnh báo "null safety", có nghĩa là những biến không thể là null trừ khi ta khai báo nó là null

Dart hỗ trợ các khái niệm Just In Time (JIT) compiler và Ahead of Time (AOT) compiler. Điều đó cho phép lập trình viên thuận tiện hơn khi viết các đoạn mã trong các giai đoạn của quá trình phát triển một sản phẩm phần mềm:

- Development phase: JIT, dartdevc
- Production phase: AOT, dart2js

Phụ lục 9: Framework Flutter

Flutter là một framework mã nguồn mở được phát triển bởi Google, cho phép xây dựng các ứng dụng di động nhanh chóng và dễ dàng trên nhiều nền tảng, bao gồm cả iOS và Android. Flutter sử dụng ngôn ngữ lập trình Dart, cung cấp một cách tiếp cận khác biệt so với các framework khác như React Native hay Xamarin.

Một số điểm mạnh của Flutter:

- Tốc độ phát triển: Flutter cho phép các nhà phát triển tạo các ứng dụng di động nhanh chóng và dễ dàng, giúp tiết kiệm thời gian và tăng năng suất làm việc.
- Đồ họa đẹp: Flutter có một số tính năng tuyệt vời cho đồ họa, bao gồm cả tích hợp widget tùy chỉnh và các hiệu ứng hình ảnh, giúp các ứng dụng được trình bày một cách hấp dẫn và chuyên nghiệp.
- Tính di động: Flutter cung cấp cho các nhà phát triển tính di động cao, giúp tạo ra các ứng dụng di động đáp ứng được các yêu cầu của người dùng trên nhiều nền tảng.
- Widget linh hoạt: Flutter sử dụng widget để tạo giao diện người dùng, cho phép các nhà phát triển dễ dàng tùy chỉnh các phần tử trên giao diện một cách linh hoạt và nhanh chóng.
- Tích hợp API: Flutter hỗ trợ tích hợp API và các tính năng của hệ thống, giúp cho các ứng dụng di động có thể kết nối với các dịch vụ bên ngoài một cách dễ dàng.

Phụ lục 10: Xử lý tác vụ nền (Background Services)

Background Service là một thành phần ứng dụng có thể thực hiện các hoạt động chạy ngầm mà không cần giao diện người dùng (UI). Trong lập trình Mobile (Flutter/Android/iOS), điều này cực kỳ quan trọng đối với các ứng dụng IoT báo động, vì sự cố (cháy) có thể xảy ra bất cứ lúc nào, ngay cả khi người dùng đã tắt màn hình

hoặc thoát ứng dụng.

Các thành phần trong mô hình Background Service:

- **Service Instance:** Là một tiến trình (process) độc lập được hệ điều hành cấp phát để chạy mã lệnh tách biệt với giao diện chính. Trong dự án, FlutterBackgroundService khởi tạo một instance này để duy trì kết nối.

- **Isolates (Đa luồng):** Flutter chạy đơn luồng (single thread). Để chạy nền mà không làm đơ ứng dụng, Service thường chạy trên một Isolate riêng biệt. Nó có bộ nhớ riêng và giao tiếp với luồng chính (Main Isolate) thông qua các cổng (Ports).

- **Foreground Notification (Thông báo tiền cảnh):** Đối với Android 8.0 trở lên, để một Service không bị hệ điều hành "giết" (kill) nhằm tiết kiệm pin, nó buộc phải hiển thị một thông báo dính (sticky notification) trên thanh trạng thái để báo hiệu cho người dùng biết "Ứng dụng đang chạy ngầm bảo vệ...".

Ứng dụng trong dự án: File lib/service/background_service.dart khởi tạo một kết nối MQTT riêng biệt nằm trong hàm onStart. Service này tự động lắng nghe topic báo cháy (ALARM_FIRE) và kích hoạt loa/rung ngay lập tức mà không cần người dùng mở App.

```dart
@pragma('vm:entry-point')
void onStart(ServiceInstance service) async {
  // 1. Đảm bảo plugin hoạt động trong Background Isolate
  DartPluginRegistrant.ensureInitialized();

  // 2. Cấu hình service cho Android
  if (Platform.isAndroid) {
    service.on('setAsForeground').listen((event) {
      if (service is AndroidServiceInstance) {
        service.setAsForegroundService();
      }
    });

    service.on('setAsBackground').listen((event) {
      if (service is AndroidServiceInstance) {
        service.setAsBackgroundService();
      }
    });
```

```dart
class BackgroundServiceHelper {
  static Future<void> initializeService() async {
    final service = FlutterBackgroundService();
    await service.configure(
      androidConfiguration: AndroidConfiguration(
        onStart: onStart,
        autoStart: true,
        isForegroundMode: true,
        notificationChannelId: 'fire_alarm_channel_v4_silent',
        initialNotificationTitle: 'Kapnos',
        initialNotificationContent: 'Đang chạy ngầm bảo vệ...',
        foregroundServiceNotificationId: 888,
      ),
      iosConfiguration: IosConfiguration(
        autoStart: true,
        onForeground: onStart,
      ),
    );
  }
}
```

Hình 0. 7 Khởi tạo tiến trình chạy ngầm

Phụ lục 11: Lưu trữ cục bộ & Caching

Lưu trữ cục bộ là kỹ thuật lưu dữ liệu trực tiếp trên bộ nhớ vật lý của thiết bị người dùng thay vì chỉ dựa vào Server. Kỹ thuật này giúp ứng dụng hoạt động nhanh hơn, giảm tải cho Server và quan trọng nhất là hỗ trợ trải nghiệm "Offline-first" (dùng được khi mất mạng).

Các thành phần trong mô hình Lưu trữ cục bộ:

- **Key-Value Store:** Là dạng lưu trữ đơn giản nhất, lưu dữ liệu dưới dạng cặp Khóa: Giá trị. Trong Flutter, SharedPreferences là đại diện tiêu biểu cho mô hình này trên

Android/iOS.

- **Tuần tự hóa / Giải tuần tự:** Quá trình chuyển đổi các đối tượng phức tạp (Object, Class) thành chuỗi văn bản (JSON String) để lưu trữ và ngược lại.
- **Cache Strategy:** Chiến thuật quyết định khi nào dùng dữ liệu cũ và khi nào tải mới. Ví dụ: "Hiển thị cache trước, tải mới sau".

Ứng dụng trong dự án: Sử dụng SharedPreferences ở nhiều nơi:

- history_helper.dart: Lưu danh sách lịch sử cảnh báo.
- main_screen.dart: Lưu cấu hình cached_devices để khi mở App lên là thấy ngay danh sách thiết bị cũ mà chưa cần đợi tải từ Server.

```dart
class HistoryHelper {
  static Future<void> addEvent(String deviceName, String type, double smoke,
      int battery, double temperature) async {
    final prefs = await SharedPreferences.getInstance();

    List<String> historyList = prefs.getStringList('history_data') ?? [];

    Map<String, dynamic> newEvent = {
      'time': DateFormat('HH:mm:ss dd/MM/yyyy').format(DateTime.now()),
      'device': deviceName,
      'type': type,
      'smoke': smoke,
      'battery': battery,
      'temp': temperature,
    };
    historyList.insert(0, json.encode(newEvent));
    if (historyList.length > 50) {
      historyList = historyList.sublist(0, 50);
    }
    await prefs.setStringList('history_data', historyList);
  }

static Future<List<Map<String, dynamic>>> getEvents() async {
  final prefs = await SharedPreferences.getInstance();
  List<String> historyList = prefs.getStringList('history_data') ?? [];

  return historyList
      .map((item) => json.decode(item) as Map<String, dynamic>)
      .toList();
}
```

Hình 0. 8 Khởi tạo chức năng lưu lịch sử

Phụ lục 12: Thông báo cục bộ (Local Notification)

Khác với Push Notification (được gửi từ Server như Firebase), Local Notification được lên lịch và kích hoạt bởi chính mã nguồn chạy bên trong ứng dụng (từ Background Service hoặc Main App). Đây là cách nhanh nhất để báo động vì không phụ thuộc vào độ trễ của đường truyền Internet bên thứ 3.

Các thành phần trong mô hình Local Notification:

- **Kênh thông báo (Notification Channel):** Từ Android 8.0, mọi thông báo phải thuộc về một kênh. Kênh này quy định mức độ quan trọng, âm thanh, rung. Ví dụ: Kênh "Cảnh báo cháy" sẽ có mức Importance.max để bung cửa sổ (Heads-up) ngay lập tức.

- **Gói tin (Payload):** Dữ liệu đính kèm ẩn bên trong thông báo. Khi người dùng bấm vào thông báo, Payload này được dùng để điều hướng ứng dụng đến đúng màn hình chi tiết thiết bị đang báo cháy.
- **Hành động (Actions):** Các hành động tương tác nhanh, ví dụ như nút "Tắt báo động" ngay trên thanh thông báo.

Ứng dụng trong dự án: File lib/notification_helper.dart cấu hình kênh fire_alarm_channel_v3_popup với độ ưu tiên cao nhất (Priority.high) để đảm bảo thông báo luôn xuất hiện trên cùng màn hình khóa khi có cháy.

```dart
// Tạo kênh thông báo
final androidImplementation =
    _notifications.resolvePlatformSpecificImplementation<
        AndroidFlutterLocalNotificationsPlugin>();

const AndroidNotificationChannel alarmChannel = AndroidNotificationChannel(
  'fire_alarm_channel_v4_silent',
  'Cảnh Báo Cháy (Visual Only)',
  description: 'Kênh hiển thị thông báo, âm thanh do App xử lý',
  importance: Importance.max,
  playSound: false,
);
//hiển thị thông báo
static Future<void> showNotification(String title, String body,
    {Map<String, dynamic>? payload}) async {
  const AndroidNotificationDetails androidDetails =
      AndroidNotificationDetails(
    'fire_alarm_channel_v3_popup',
    'Cảnh Báo Cháy Khẩn Cấp',
    importance: Importance.max,
    priority: Priority.high,
    enableVibration: true,
    playSound:
        false,
    color: Color(0xFFFF0000),
    fullScreenIntent: true,
    icon: '@mipmap/ic_launcher',
    styleInformation: BigTextStyleInformation(''),
  ); // AndroidNotificationDetails
```

Hình 0. 9 Tạo và hiển thị thông báo

Phụ lục 13: Thao tác Bit (Bitwise Operations)

Thao tác Bit [13] là kỹ thuật xử lý dữ liệu ở mức nhị phân (0 và 1). Đây là kỹ thuật nâng cao thường thấy trong lập trình nhúng (Embedded) và IoT để tối ưu hóa băng thông truyền tải. Thay vì gửi một chuỗi văn bản dài "200", ta có thể gửi 1 byte giá trị 200.

Các thành phần trong mô hình Thao tác Bit:

- **Bit Shifting (Dịch bit):** Phép toán >> (dịch phải) hoặc << (dịch trái) để di chuyển các bit. Thường dùng để tách một số lớn (VD: 16-bit integer) thành các mảnh nhỏ (bytes) để truyền đi.
- **Masking (Mặt nạ Bit):** Sử dụng phép & (AND) với 0xFF để lấy ra giá trị của 8 bit

cuối cùng, loại bỏ các bit thừa.

Ứng dụng trong dự án: Trong lib/service/smoke_service.dart, hàm enqueueGrpcHack sử dụng: List<int> bytes = [(val >> 8) & 0xFF, val & 0xFF]; Mục đích là chia giá trị ngưỡng (threshold) thành 2 bytes (High byte và Low byte) để gửi qua giao thức LoRaWAN xuống thiết bị phần cứng, vì LoRaWAN giới hạn kích thước gói tin rất nhỏ.

```
Future<bool> enqueueGrpcHack(String devEui, double thresholdVal) async {
  final String url = '$baseUrl/devices/$devEui/queue';

  try {
    int val = thresholdVal.toInt();
    // Mã hóa 2 bytes Big Endian
    List<int> bytes = [(val >> 8) & 0xFF, val & 0xFF];
    String dataBase64 = base64Encode(bytes);
```

Hình 0. 10 Dùng phép dịch bit

Phụ lục 14: Giao thức truyền thông UART

Giao thức truyền thông UART (Universal Asynchronous Receiver-Transmitter) là một phương thức phổ biến trong việc truyền và nhận dữ liệu giữa các thiết bị điện tử. Nó cho phép truyền dữ liệu không đồng bộ (asynchronous), tức là không cần một tín hiệu clock chung như trong một số giao thức khác như SPI hoặc I2C. Thay vào đó, dữ liệu được truyền một bit sau một bit và được đồng bộ hóa thông qua các bit khởi đầu (start bit), dữ liệu (data bits), và các bit kết thúc (stop bits) [9] .

Dưới đây là các thành phần chính của giao thức truyền thông UART:

- **Start Bit**: Đây là bit đầu tiên của mỗi khung dữ liệu và thường có giá trị logic thấp (0). Nó được sử dụng để báo hiệu cho người nhận rằng một khung dữ liệu đang bắt đầu.
- **Data Bits**: Đây là các bit chứa dữ liệu cần truyền. Số bit dữ liệu trong mỗi khung UART có thể linh hoạt, thường từ 5 đến 9 bit. Số lượng bit này thường được cài đặt trước để người nhận biết được cách tạo ra các khung dữ liệu.
- **Parity Bit** (optional): Đây là một bit kiểm tra thêm để kiểm tra lỗi truyền thông. Parity bit có thể là bit chẵn (even parity) hoặc bit lẻ (odd parity), tùy thuộc vào tổng số bit dữ liệu ở giá trị logic cao trong khung. Tùy chọn này giúp phát hiện một số lỗi truyền thông như mất bit.
- **Stop Bits**: Đây là một hoặc nhiều bit có giá trị logic cao (1) sau khi dữ liệu và parity bit (nếu có) để báo hiệu cho người nhận rằng khung dữ liệu đã kết thúc.



Hình 0. 11 Bản tin truyền nhận trong UART

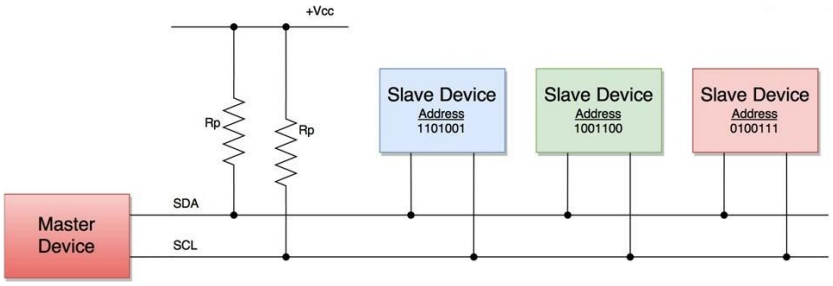
Phụ lục 15: Giao thức truyền thông I2C

I2C, viết tắt của "Inter-Integrated Circuit," [10] là một giao thức truyền thông số học được phát triển bởi Philips (nay là NXP Semiconductors) vào những năm 1980. I2C

là một giao thức truyền thông đồng bộ dựa trên hai dây (SDA và SCL) để kết nối các thiết bị điện tử với nhau. Giao thức này thường được sử dụng để kết nối các linh kiện như cảm biến, bộ nhớ EEPROM, vi điều khiển, các IC điều khiển màn hình, và nhiều thiết bị điện tử khác.

I2C hoạt động dựa trên nguyên tắc gửi và nhận dữ liệu thông qua hai dây chung

- SDA (Serial Data Line): Dây này dùng để truyền dữ liệu giữa các thiết bị. Dữ liệu được truyền theo định dạng chuỗi các bit.
- SCL (Serial Clock Line): Dây này dùng để đồng bộ hóa việc truyền dữ liệu giữa các thiết bị. Tín hiệu clock điều khiển tốc độ truyền dữ liệu.



Hình 0. 12 Mô hình I2C

Giao thức I2C hoạt động trong mô hình master-slave, trong đó một thiết bị đóng vai trò "*master*" điều khiển các thiết bị "*slave*". Thiết bị *master* sẽ khởi tạo và điều khiển quá trình truyền dữ liệu, còn các thiết bị *slave* sẽ phản hồi và thực hiện các thao tác theo yêu cầu của *master*.

Mỗi thiết bị có một địa chỉ duy nhất trong mạng I2C, cho phép *master* xác định thiết bị nào sẽ giao tiếp. Giao thức I2C cũng hỗ trợ nhiều tốc độ truyền khác nhau, thường từ vài kilobits đến vài megabits mỗi giây.

I2C rất phổ biến trong các ứng dụng nhúng vì nó đơn giản và tiết kiệm dây kết nối, nhưng cũng có hạn chế về khoảng cách truyền tải và tốc độ truyền dữ liệu so với các giao thức khác.

Phụ lục 16: Bộ chuyển đổi tương tự - số (ADC)

ADC (Analog-to-Digital Converter) [12] hay bộ chuyển đổi tương tự sang số là một mạch điện tử dùng để chuyển đổi tín hiệu analog liên tục (như điện áp đo từ cảm biến) thành giá trị nhị phân (digital) mà các bộ vi điều khiển và hệ thống số có thể hiểu và xử lý. Đây là thành phần quan trọng vì thế giới thực xung quanh ta thường là tín hiệu analog, trong khi hầu hết các thiết bị xử lý hiện đại hoạt động với dữ liệu số.

ADC lấy mẫu tín hiệu analog trong các khoảng thời gian định sẵn và báo giá trị đó dưới

dạng số nguyên tương ứng với mức tín hiệu.

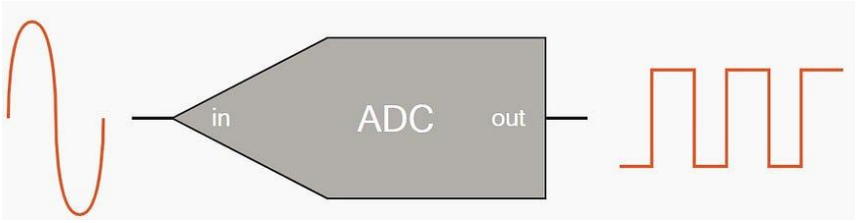
Mỗi giá trị analog được ánh xạ sang giá trị số dựa trên điện áp tham chiếu và độ phân giải bit của ADC; ví dụ ADC 10-bit có thể tạo ra 1024 mức khác nhau

Các thông số quan trọng:

- Độ phân giải: số bit đầu ra (10-bit, 12-bit, 16-bit …) quyết định độ nhỏ nhất của bước đo.
- Tốc độ lấy mẫu: số mẫu analog được ADC chuyển đổi mỗi giây, đo bằng samples/s.
- Điện áp tham chiếu (Vref): xác định dải điện áp analog đầu vào tối đa mà ADC có thể đo.

Các loại ADC:

- Flash ADC – nhanh nhưng phức tạp.
- ADC kiểu SAR (Successive Approximation) – cân bằng giữa tốc độ và độ chính xác.
- ADC dạng tích hợp đếm/độ dốc – đơn giản nhưng thường chậm hơn.



Hình 0. 13 Mô hình hoạt động của ADC

Phụ lục 17: Giao thức OTAA (Over-The-Air Activation)

OTAA [13] là cơ chế kích hoạt thiết bị trong mạng LoRaWAN, cho phép node (end-device) tự động đăng ký và xác thực với Network Server thông qua quá trình trao đổi khóa bảo mật qua mạng không dây. Đây là phương thức kích hoạt linh hoạt và bảo mật cao, được khuyến nghị sử dụng trong các hệ thống LoRaWAN hiện đại.

Khi sử dụng OTAA, thiết bị sẽ gửi một bản tin Join Request đến Network Server. Máy chủ sau đó xác thực thiết bị và phản hồi bằng Join Accept, đồng thời cấp phát các khóa phiên bảo mật cần thiết cho quá trình truyền thông dữ liệu.

Thành phần định danh trong OTAA:

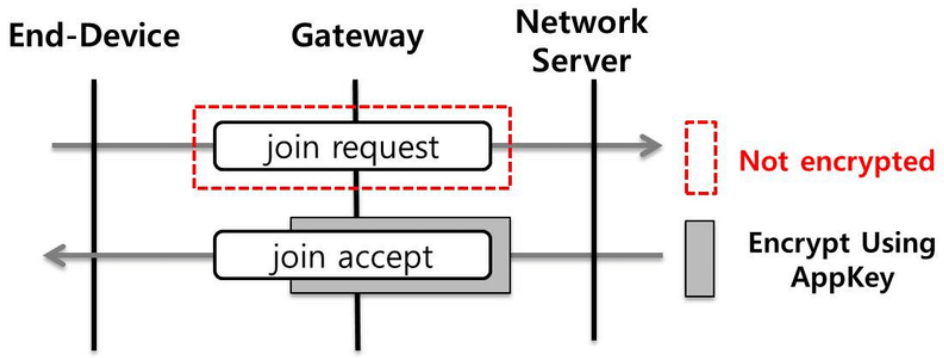
- **DevEUI:** Mã định danh duy nhất của thiết bị
- **AppEUI (hoặc JoinEUI):** Mã định danh của ứng dụng
- **AppKey:** Khóa bảo mật dùng để tạo khóa phiên

Các khóa được sinh ra sau khi Join thành công:

- **NwkSKey:** Khóa bảo mật cho lớp mạng

- **AppSKey:** Khóa bảo mật cho dữ liệu ứng dụng
- **DevAddr:** Địa chỉ động của thiết bị trong mạng

Quy trình OTAA: Thiết bị gửi Join Request (chứa DevEUI, AppEUI, nonce)→Network Server xác thực thiết bị→Server gửi Join Accept→Thiết bị sinh khóa phiên (NwkSKey, AppSKey)→Thiết bị chính thức tham gia mạng LoRaWAN



Hình 0. 14 Mô hình giao thức OTAA

Phụ lục 18: Module RAK3172 WisDuo LoRaWAN

RAK3172 WisDuo là một module LoRaWAN công suất thấp được phát triển bởi RAKwireless, dựa trên vi điều khiển STM32WLE5 tích hợp sẵn transceiver LoRa. Module được thiết kế tối ưu cho các ứng dụng IoT tầm xa, tiêu thụ năng lượng thấp, đặc biệt phù hợp cho các hệ thống giám sát môi trường, nông nghiệp thông minh, thành phố thông minh và các ứng dụng LoRaWAN khác.

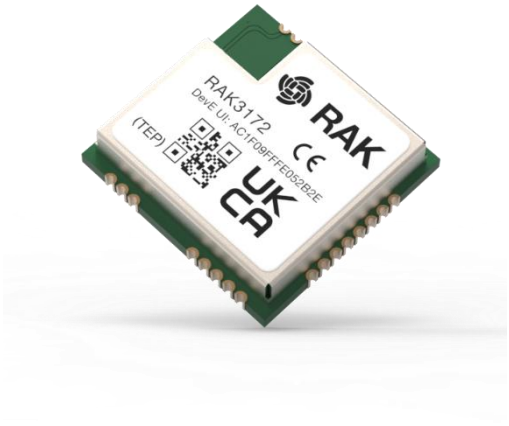
RAK3172 hỗ trợ đầy đủ chuẩn LoRaWAN Class A/B/C, hoạt động trên nhiều băng tần khác nhau (EU868, US915, AS923, v.v.) và có khả năng truyền dữ liệu ổn định với khoảng cách xa trong khi vẫn đảm bảo mức tiêu thụ năng lượng rất thấp.

Một số thông số kỹ thuật RAK3172 WisDuo

- **IC chính**: STM32WLE5 SoC tích hợp LoRa RF Transceiver
- **Điện áp sử dụng:** 1.8V ~ 3.6V DC
- **Dòng điện tiêu thụ:**
  - Chế độ ngủ (Sleep): ~1.5 µA
  - Chế độ hoạt động: ~30–50 mA
  - Truyền LoRa (TX): lên đến ~120 mA (tùy công suất phát)
- **Nhân xử lý trung tâm:** ARM Cortex-M4 32-bit, xung nhịp lên đến 48 MHz
- **Bộ nhớ:**
  - Flash: 256 KB
  - SRAM: 64 KB
- **LoRa / LoRaWAN:**
  - Hỗ trợ LoRaWAN Class A / B / C
  - Băng tần: EU868, US915, AS923, AU915, KR920, IN865

- Công suất phát: lên đến +22 dBm
- Khoảng cách truyền: lên đến 10–15 km (môi trường mở)

- **Giao tiếp:** UART, SPI, I2C, GPIO, ADC, PWM
- **Chuẩn RF:**
  - LoRa modulation
  - FSK, GFSK
  - Hỗ trợ CRC, AES-128
- **Bảo mật:**
  - Mã hóa AES-128
  - Hỗ trợ bảo mật LoRaWAN end-to-end
- **Nhiệt độ hoạt động:** -40°C ~ +85°C
- **Kích thước module:** Nhỏ gọn, phù hợp cho thiết bị IoT pin



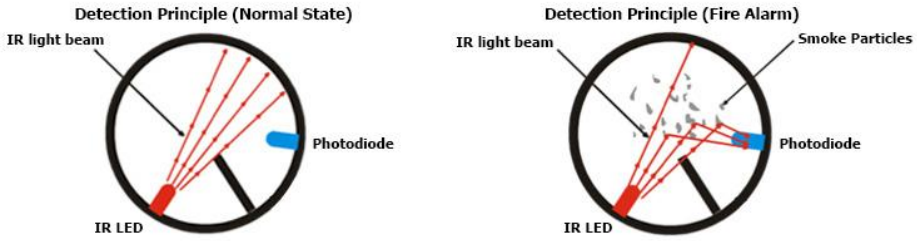
Hình 0. 15 Module RAK3172

Phụ lục 19: Tổng quan buồng quang cảm biến khói

Buồng quang cảm biến khói [14] là bộ phận quan trọng trong các cảm biến khói quang học, có nhiệm vụ phát hiện sự xuất hiện của khói dựa trên hiện tượng tán xạ ánh sáng. Nguyên lý hoạt động của buồng quang dựa trên việc sử dụng một nguồn phát ánh sáng (thường là LED hồng ngoại) và một cảm biến thu quang (photodiode hoặc phototransistor) được bố trí theo một góc xác định bên trong buồng kín.

Trong điều kiện bình thường, khi không có khói, ánh sáng từ LED không chiếu trực tiếp vào cảm biến thu do cấu trúc hình học của buồng quang đã được thiết kế để tránh ánh sáng phản xạ trực tiếp. Khi khói xâm nhập vào buồng, các hạt khói sẽ làm tán xạ chùm ánh sáng, khiến một phần ánh sáng bị phản xạ về phía cảm biến thu. Tín hiệu quang này sau đó được chuyển đổi thành tín hiệu điện, khuếch đại và xử lý bởi mạch điện tử để xác định mức độ khói.

Buồng quang thường được thiết kế với kết cấu chống bụi, chống côn trùng và hạn chế ánh sáng môi trường, nhằm giảm nhiễu và tăng độ ổn định của phép đo. Nhờ nguyên lý phát hiện dựa trên tán xạ ánh sáng, cảm biến khói quang có khả năng phát hiện sớm các đám cháy âm ỉ – loại cháy tạo ra nhiều khói nhưng ít ngọn lửa.



Hình 0. 16 Cấu tạo buồng quang khói

Phụ lục 20: Cảm biến nhiệt ẩm SHTC3

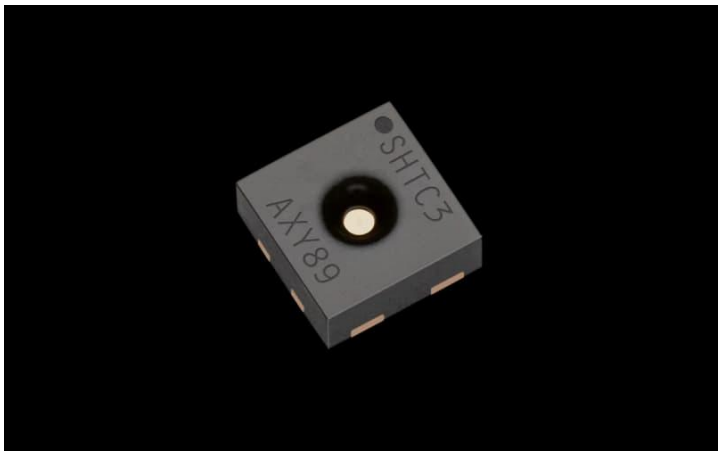
SHTC3 là cảm biến đo nhiệt độ và độ ẩm kỹ thuật số do Sensirion (Thụy Sĩ) sản xuất. Thiết bị được thiết kế với kích thước siêu nhỏ, tiêu thụ năng lượng thấp và độ chính xác cao, phù hợp cho các ứng dụng IoT, thiết bị đeo, hệ thống giám sát môi trường và các thiết bị chạy pin.

SHTC3 tích hợp sẵn cảm biến độ ẩm điện dung, cảm biến nhiệt độ và mạch ADC, cho phép xuất dữ liệu dạng số thông qua giao tiếp I2C, giúp giảm nhiễu và tăng độ ổn định của phép đo.

Một số thông số kỹ thuật của SHTC3

- **Hãng sản xuất:** Sensirion (Thụy Sĩ)
- **Loại cảm biến:** Nhiệt độ & Độ ẩm kỹ thuật số
- **Điện áp hoạt động:** 1.62V ~ 3.6V
- **Dòng tiêu thụ:**
  - Chế độ đo: ~180 µA
  - Chế độ ngủ: ~0.3 µA
- **Giao tiếp:** I2C
- **Dải đo nhiệt độ:** -40°C ~ +125°C
- **Độ chính xác nhiệt độ:** ±0.2°C (ở 25°C)
- **Dải đo độ ẩm:** 0% ~ 100% RH
- **Độ chính xác độ ẩm:** ±2% RH

- **Độ phân giải:**
  - Nhiệt độ: 14-bit
  - Độ ẩm: 12-bit
- **Thời gian đáp ứng:** < 5 giây
- **Kích thước:** 2.0 × 2.0 × 0.75 mm
- **Tích hợp:** Cảm biến RH, cảm biến nhiệt, ADC, bộ hiệu chuẩn nội



Hình 0. 17 Module SHTC3

Phụ lục 21: IC ổn áp 9013-33GB

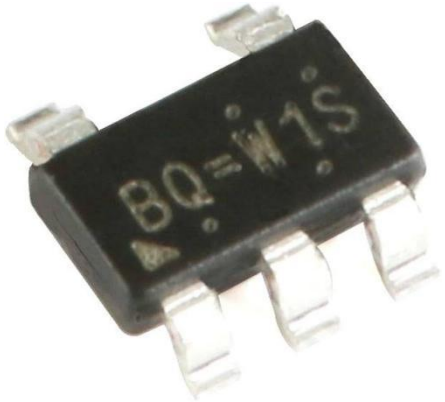
RT9013-33GB là một bộ ổn áp Linear Low-Dropout (LDO) regulator do hãng Richtek phát triển, cung cấp điện áp đầu ra cố định 3.3 V với dòng tối đa lên tới 500 mA, được thiết kế cho các ứng dụng yêu cầu nguồn ổn định, nhiễu thấp và tiêu thụ năng lượng thấp trong các mạch điện tử nhúng và IoT.

LDO này phù hợp để sử dụng trong các thiết bị bộ điều khiển, module không dây, cảm biến và hệ thống điện áp 3.3 V, đặc biệt trong các bo mạch chạy pin hoặc yêu cầu kích thước nhỏ gọn.

Một số thông số kỹ thuật RT9013-33GB

- **IC chính:** Linear voltage regulator LDO
- **Điện áp đầu vào:** 2.2 V ~ 5.5 V
- **Điện áp đầu ra cố định:** 3.3 V
- **Dòng đầu ra tối đa:** 500 mA
- **Gói IC:** SOT-23-5 (Surface Mount)
- **Độ sụt áp thấp (Dropout):** ~250 mV @ 500 mA

- **Dòng không tải (Quiescent Current):** ~25 µA
- **PSRR cao:** tăng ổn định nguồn và giảm nhiễu trên rail đầu ra
- **Tính năng bảo vệ:** bao gồm giới hạn dòng, bảo vệ quá nhiệt
- **Điều khiển ENABLE:** cho phép bật/tắt LDO bằng logic điều khiển
- **Nhiệt độ hoạt động:** −40 °C ~ +85 °C
- **Chứng nhận:** RoHS / Lead-free compliant



Hình 0. 18 IC 9013-33GB

Phụ lục 22: BJT BC847B-NPN

Loại Transistor: NPN

Dòng Ic max: 100 mA

Điện áp Vce max: 45 V

Điện áp Vce(sat) max @ Ib, Ic: 400mV @ 5mA, 100mA

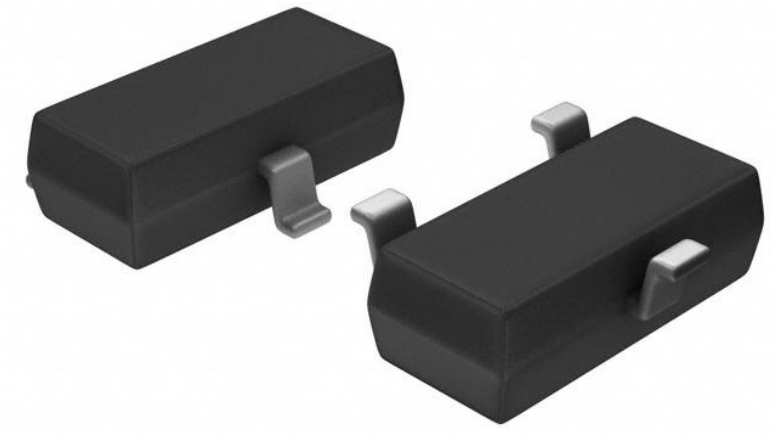
Dòng điện Cutoff Ic max: 15nA

Độ lợi dòng điện DC (hFE) min @ Ic, Vce: 200 @ 2mA, 5V

Công suất max: 250 mW

Tần số chuyển đổi: 100MHz

Nhiệt độ hoạt động: 150°C (TJ)



Hình 0. 19 BJT BC847B

Phụ lục 23: BJT 2SA1015LT1-PNP

Loại Transistor: PNP

Dòng Ic max: 150 mA

Điện áp Vce max: 50 V

Điện áp Vce(sat) max @ Ib, Ic: 300mV @ 10mA, 100mA

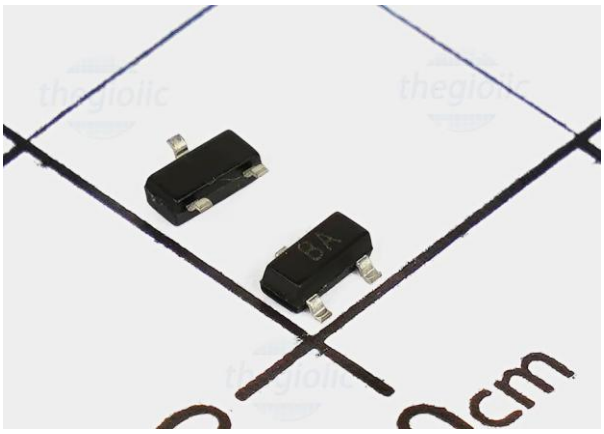
Dòng điện Cutoff Ic max: 100nA

Độ lợi dòng điện DC (hFE) min @ Ic, Vce: 70 @ 2mA, 6V

Công suất max: 200 mW

Tần số chuyển đổi: 80MHz

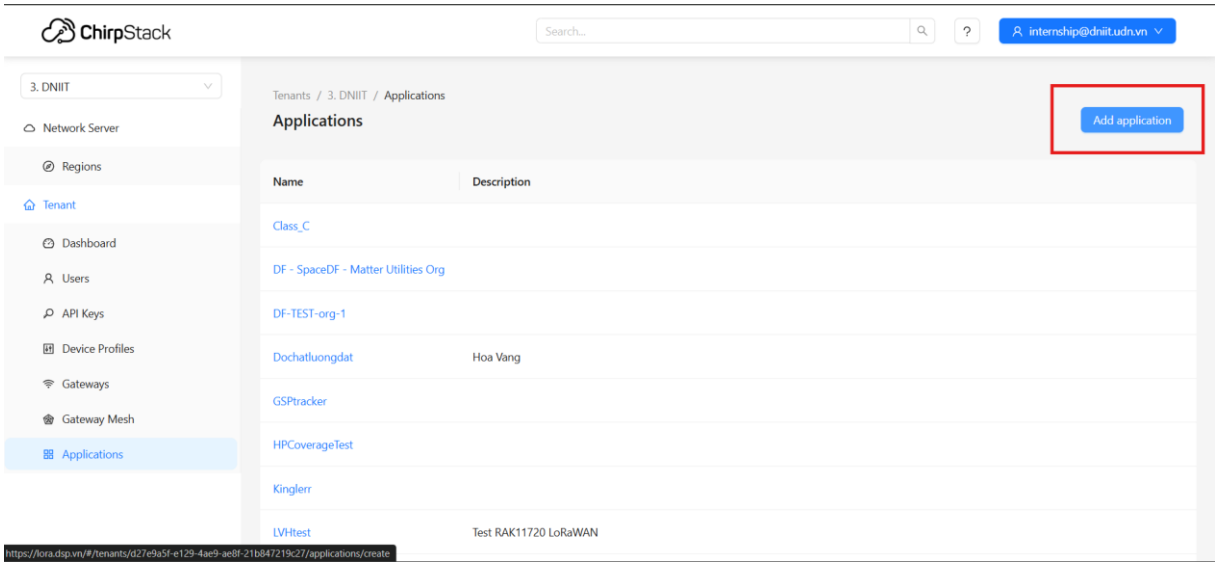
Nhiệt độ hoạt động: -55°C ~ 150°C



Hình 0. 20 BJT 2SA1015LT1

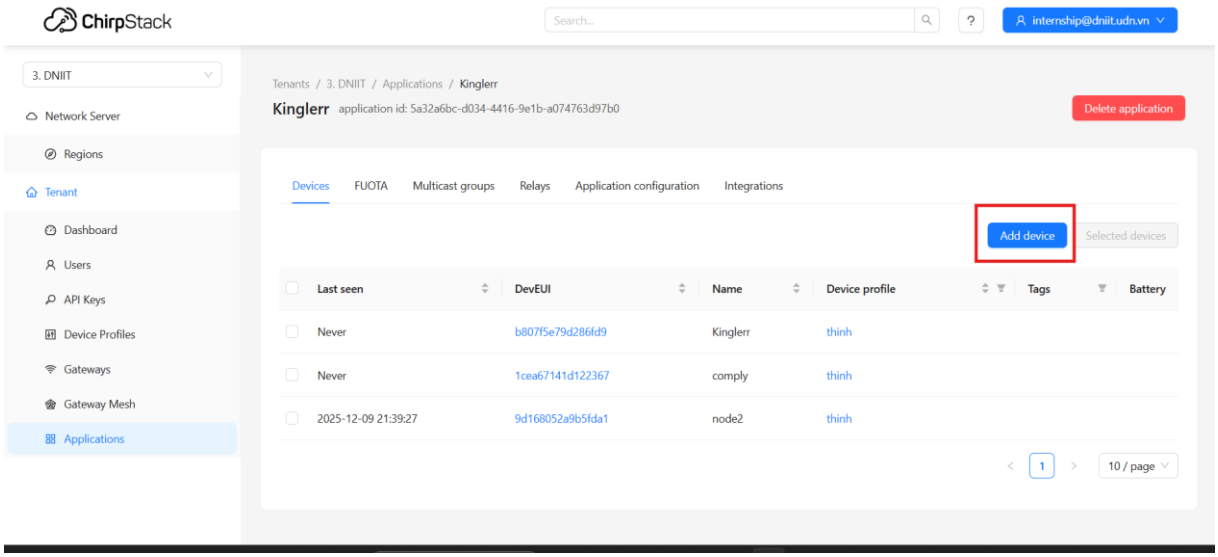
Phụ lục 24: Cách cấu hình một node trên ChirpStack Server

B1: Vào mục Application, chọn "Add Application"



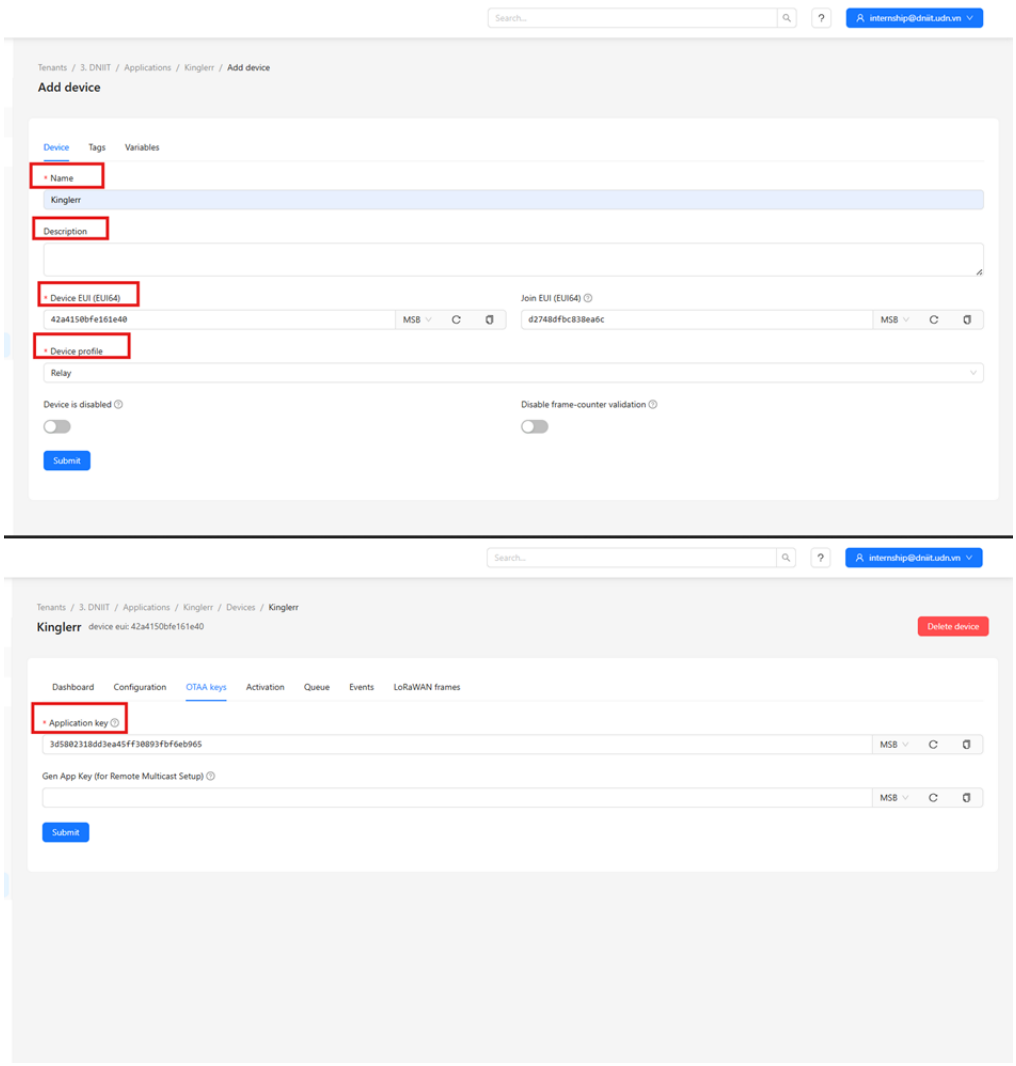
Hình 0. 21 Tạo mục Application

B2: Tạo Device bằng cách chọn mục "Add device".



Hình 0. 22 Tạo Device

B3: Thiết lập các thông số cần thiết của Node



Hình 0. 23 Các thông số cần thiết của Device

B4: Sau đó ta dùng các key cần thiết như OTAA key, DevEUI để gán vào phần code firmware.

```
#define OTAA_BAND     (RAK_REGION_AS923_2)
#define OTAA_DEVEUI   {0xEA, 0x58, 0xB1, 0xE8, 0x7D, 0xEA, 0x83, 0x00}
#define OTAA_APPEUI   {0xEA, 0x58, 0xB1, 0xE8, 0x7D, 0xEA, 0x83, 0x00}
#define OTAA_APPKEY   {0x1F, 0xA5, 0x75, 0xC8, 0xA9, 0x71, 0x8B, 0x96, 0x97, 0x0A, 0xE5, 0xAC, 0xBB, 0x01, 0x1D, 0x11}
```

Hình 0. 24 Các thông số được gán vào phần code firmware